

Algorithmic Adventures, Cracking Puzzles with Code

Ed Shah

July 22, 2024

Contents

<i>Array and String Manipulation</i>	9
<i>Introduction to Arrays</i>	10
<i>Sliding Window Technique</i>	16
<i>Best Time to Buy and Sell Stock</i>	22
<i>Longest Substring Without Repeating Characters</i>	25
<i>Longest Substring with At Most Two Distinct Characters</i>	28
<i>Minimum Size Subarray Sum</i>	31
<i>Contains Duplicate</i>	34
<i>Valid Anagram</i>	37
<i>Group Anagrams</i>	40
<i>Longest Repeating Character Replacement</i>	43

<i>Maximum Product Subarray</i>	47
<i>Non-overlapping Intervals</i>	51
<i>Merge Intervals</i>	55
<i>Insert Interval</i>	58
<i>Find Minimum in Rotated Sorted Array</i>	62
<i>Search in Rotated Sorted Array</i>	65
<i>Meeting Rooms</i>	68
<i>Meeting Rooms II</i>	72
<i>Rotate Image</i>	76
<i>Rotate Image: Spin That Matrix!</i>	79
<i>Container With Most Water</i>	82
<i>Product of Array Except Self</i>	86
<i>Maximum Subarray</i>	90
<i>Number of Islands</i>	94
<i>Word Break</i>	99

<i>Word Search</i>	103
<i>Unique Paths</i>	107
<i>Climbing Stairs</i>	110
<i>Min Stack</i>	113
<i>647. Palindromic Substrings</i>	116
<i>Jump Game I</i>	119
<i>Skyline Problem</i>	122
<i>Decode Ways</i>	125
<i>Longest Increasing Subsequence</i>	128
<i>Longest Consecutive Sequence</i>	131
<i>Minimum Window Substring</i>	135
<i>Set Matrix Zeroes</i>	139
<i>Merge k Sorted Lists</i>	144
<i>Coin Change</i>	148
<i>Sum of Two Integers Without Using Plus or Minus Operators</i>	152

<i>Missing Number</i>	156
<i>N-Queens Problem</i>	159
<i>Encode and Decode Strings</i>	163
<i>Combination Sum</i>	166
<i>Pacific Atlantic Water Flow</i>	169
<i>Linked List Problems</i>	172
<i>Remove Nth Node From End of List</i>	173
<i>Linked List Cycle</i>	177
<i>Reverse Linked List</i>	180
<i>Reorder List</i>	183
<i>Reorder List</i>	187
<i>Merge Two Sorted Lists</i>	190
<i>Add Two Numbers</i>	193
<i>Tree and Graph Problems</i>	196
<i>100. Same Tree</i>	197

Graph Valid Tree 201

Number of Connected Components in an Undirected Graph 205

Clone Graph 208

Binary Tree Maximum Path Sum 212

Validate Binary Search Tree 215

Alien Dictionary 219

Alien Dictionary 222

Trie (Prefix Tree) 225

Course Schedule 229

Dynamic Programming 233

Climbing Stairs 234

Decode Ways 237

Unique Paths 240

House Robber 243

House Robber II 246

<i>Coin Change</i>	250
<i>Longest Increasing Subsequence</i>	254
<i>Longest Common Subsequence</i>	258
<i>Bit Manipulation</i>	261
<i>Number of 1 Bits</i>	262
<i>Reverse Bits</i>	265
<i>231. Power of Two</i>	268
<i>Counting Bits</i>	272
<i>Counting Bits</i>	275
<i>Search and Sort</i>	278
<i>Valid Anagram</i>	279
<i>Contains Duplicate</i>	282
<i>Group Anagrams</i>	285
<i>Merge Intervals</i>	288
<i>Find Minimum in Rotated Sorted Array</i>	291

<i>Search in Rotated Sorted Array</i>	294
<i>Merge k Sorted Lists</i>	298
<i>Miscellaneous</i>	302
<i>Happy Number</i>	303
<i>933. Number of Recent Calls</i>	306
<i>362. Design Hit Counter</i>	309
<i>994. Rotting Oranges</i>	312
<i>295. Find Median from Data Stream</i>	315
<i>Design Add and Search Words Data Structure</i>	318
<i>Kth Smallest Element in a BST</i>	323

Array and String Manipulation

Introduction to Arrays

Creating an Array in Python

Arrays are the backbone of efficient data handling in many programming scenarios¹, and Python, with its versatile capabilities, offers a unique approach to arrays. Unlike lists, which can hold items of varying data types, arrays in Python are designed to store items of the same data type². This distinction allows arrays to be more memory efficient and faster for certain operations³.

To work with arrays in Python, you need to use the array module⁴, which provides a space-efficient way to store basic data types. One important thing to remember is that arrays in Python must contain real numbers like integers and floats—no strings allowed!⁵ This ensures that all elements are of the same type, which optimizes memory usage and processing speed⁶.

Here's how you can create an integer array in Python to store account balances:

```
import array

# Creating an array to store account balances
balance = array.array('i', [350, 420, 180])

print(balance)
```

In this code snippet, we import the array module and then create an integer array ('i' stands for integer) to store three account balances⁷. The array module requires you to specify the type code, which is 'i' for integers. The balance array now contains the values 350, 420, and 180, representing the account balances of your friends. When you print the balance array, it shows the stored values in a neat and organized fashion⁸.

Types of Variables in Arrays

In Python's array module, each array must be declared with a specific type code that represents the type of elements it can hold⁹. Below are the most common type codes and their corresponding data types, along with examples:

- **Type Code 'i':** This type code represents signed integers¹⁰. For instance:

¹ Efficient data handling

² Uniform data type

³ Memory efficiency and speed

⁴ Using the array module

⁵ Numeric types only

⁶ Optimized memory usage

⁷ Creating an integer array

⁸ Print the array

⁹ Type codes

¹⁰ Signed integers

```
import array
int_array = array.array('i', [150, 230, 310])
print(int_array) # Output: array('i', [150, 230, 310])
```

- **Type Code 'f':** This type code is used for floating-point numbers¹¹. For example:

¹¹ Floating-point numbers

```
import array
float_array = array.array('f', [1.7, 2.8, 3.9])
print(float_array) # Output: array('f', [1.7, 2.8, 3.9])
```

- **Type Code 'd':** This represents double-precision floating-point numbers¹². Example:

¹² Double-precision floating-point

```
import array
double_array = array.array('d', [1.25, 2.50, 3.75])
print(double_array) # Output: array('d', [1.25, 2.50, 3.75])
```

Choosing the Right Type

Choosing the right type for your array is crucial¹³. It impacts both the memory efficiency and the performance of your program. For numerical data where precise floating-point arithmetic is required, use 'd' for double precision. For integers, 'i' is suitable. Always consider the nature of the data you are working with to select the most appropriate type code¹⁴.

¹³ Choosing the right type

By understanding these type codes and their uses, you can create more efficient and effective arrays in Python, tailored to the specific needs of your applications¹⁵.

¹⁴ Consider the data nature

¹⁵ Tailored efficiency

Array Operations

Arrays support a variety of operations that make data manipulation easy and efficient¹⁶. Here are a few common operations:

¹⁶ Array operations

- **Accessing Elements:** You can access elements in an array using their index, just like with lists¹⁷.

¹⁷ Accessing elements

```
print(balance[0]) # Output: 350
```

- **Updating Elements:** Updating elements in an array is straightforward¹⁸.

¹⁸ Updating elements

```
balance[1] = 275
print(balance) # Output: array('i', [350, 275, 180])
```

- **Appending Elements:** You can add elements to the end of the array¹⁹.

¹⁹ Appending elements

```
balance.append(490)
print(balance) # Output: array('i', [350, 275, 180, 490])
```

- **Removing Elements:** Removing elements is also possible, using methods like `remove()`²⁰.

²⁰ Removing elements

```
balance.remove(180)
print(balance) # Output: array('i', [350, 275, 490])
```

- **Insert:** You can insert one or more items into an array at any given index²¹. This method expects two arguments: index and value.

²¹ Insert elements

```
balance.insert(1, 220)
print(balance) # Output: array('i', [350, 220, 275, 490])
```

- **Delete:** You can delete one item from an array by value²². This method accepts only one argument: value.

²² Delete elements

```
balance.remove(220)
print(balance) # Output: array('i', [350, 275, 490])
```

- **Search:** You can search for an item in an array based on its value²³. This method accepts only one argument: value.

²³ Search elements

```
print(balance.index(275)) # Output: 1
```

- **Update:** This operation replaces the existing value at the given index²⁴.

²⁴ Update elements

```
balance[1] = 315
print(balance) # Output: array('i', [350, 315, 490])
```

- **Traverse:** You can traverse a python array using loops²⁵.

²⁵ Traverse array

```
for x in balance:
    print(x)
```

Common Terms

When working with arrays, there are a few terms you will frequently encounter²⁶:

Subarray: A contiguous segment of an array²⁷.

Example: for `[2, 3, 6, 1, 5, 4]`, `[3, 6, 1]` is a subarray.

Subsequence: A sequence derived from another sequence by deleting some elements without changing the order of the remaining elements²⁸.

Example: for `[2, 3, 6, 1, 5, 4]`, `[3, 1, 5]` is a subsequence.

Time Complexity

Understanding the time complexity of various operations on arrays is crucial for writing efficient code. Here's a breakdown of common operations and their time complexities:

- **Access:** $O(1)$ ²⁹ Accessing an element by its index in an array is instantaneous because the memory address of each element can be directly calculated. ²⁹ Constant time access
- **Search:** $O(n)$ ³⁰ In an unsorted array, searching for an element requires checking each element one by one until the desired element is found or the end of the array is reached. ³⁰ Linear search
- **Search (sorted array):** $O(\log n)$ ³¹ In a sorted array, you can use binary search, which repeatedly divides the array in half to locate an element, resulting in logarithmic time complexity. ³¹ Binary search
- **Insert:** $O(n)$ ³² Inserting an element at a specific position requires shifting all subsequent elements to the right to make space for the new element. ³² Insert with shifting
- **Insert (at the end):** $O(1)$ ³³ Adding an element at the end of the array does not require any shifting, making it a constant time operation. ³³ Append operation
- **Remove:** $O(n)$ ³⁴ Removing an element requires shifting all subsequent elements to the left to fill the gap, which takes linear time. ³⁴ Remove with shifting
- **Remove (at the end):** $O(1)$ ³⁵ Removing the last element is a constant time operation because no elements need to be shifted. ³⁵ Remove last element

Slicing and Concatenating Arrays

Be mindful about slicing or concatenating arrays in your code. Typically, slicing and concatenating arrays take $O(n)$ time³⁶. Use start and end indices to demarcate a subarray or range where possible to avoid unnecessary overhead.

- **Slicing:** $O(n)$ ³⁷ Creating a slice of an array involves copying the specified range of elements, which takes linear time proportional to the number of elements being copied. ³⁷ Slicing complexity

²⁶ Common terms

²⁷ Subarray

²⁸ Subsequence

²⁹ Constant time access

³⁰ Linear search

³¹ Binary search

³² Insert with shifting

³³ Append operation

³⁴ Remove with shifting

³⁵ Remove last element

³⁶ Slicing and concatenating

³⁷ Slicing complexity

- **Concatenating:** $O(n)$ ³⁸ Concatenating two arrays requires creating a new array and copying all elements from both arrays into it, resulting in linear time complexity.

³⁸ Concatenating complexity

By understanding these time complexities, you can make informed decisions about which operations to use and how to structure your code for optimal performance³⁹.

³⁹ Optimizing performance

Key Considerations During Interviews

When solving array problems in interviews, it's important to clarify a few points:

- **Duplicate Values:** Confirm if the array can contain duplicate values⁴⁰. Understand how duplicates might affect the solution – whether they simplify or complicate the problem.
- **Index Boundaries:** Ensure that you manage indices properly⁴¹ when iterating through the array to avoid accessing elements outside its bounds.

⁴⁰ Check for duplicates⁴¹ Avoid out-of-bounds errors

Common Corner Cases to Test

When testing your array solutions, make sure to consider the following corner cases:

- **Empty Array:** How does your solution handle an array with no elements?⁴²
- **Small Arrays:** Consider arrays with only one or two elements⁴³.
- **Repeated Elements:** Ensure your solution works correctly when the array contains repeated elements⁴⁴.
- **Duplicate Values:** Verify that your solution correctly handles arrays with duplicate values⁴⁵.

⁴² Test empty array⁴³ Test small arrays⁴⁴ Handle repeats⁴⁵ Test duplicates

Techniques Applicable to Arrays and Strings

Remember that many techniques used for arrays can also be applied to strings, as strings are essentially arrays of characters⁴⁶. This crossover can help you apply array manipulation strategies to string problems as well.

⁴⁶ Arrays and strings

Conclusion

Arrays are a fundamental data structure that can significantly optimize your data handling capabilities in Python⁴⁷. They offer a compelling blend of memory efficiency, performance, and type safety, making them an excellent choice for handling large datasets of uniform data types. By understanding how to create and manipulate arrays, you can harness their full potential and improve the efficiency of your programs⁴⁸.

⁴⁷ Fundamental data structure⁴⁸ Optimizing efficiency

So, the next time you find yourself managing a list of homogeneous items—be it account balances, sensor readings, or scores—consider using arrays for a streamlined and efficient solution⁴⁹.

⁴⁹ Use arrays for efficiency

Sliding Window Technique

Introduction to Sliding Window

The sliding window technique is a powerful method used to solve various sub-array and substring problems⁵⁰. This approach involves maintaining a subset of items within a given range or "window" and moving this window across the data structure to achieve the desired result. The sliding window can be of fixed or variable length depending on the problem requirements.

⁵⁰ Sliding window method

History of Sliding Window Technique

The concept of the sliding window has roots in both computer science and signal processing⁵¹. In signal processing, sliding windows are used for smoothing data and calculating moving averages. In computer science, the sliding window technique gained prominence for its efficiency in solving problems related to strings and arrays.

⁵¹ Historical context

How the Sliding Window Technique Works

In a sliding window, two pointers are used to represent the boundaries of the window⁵². Typically, both pointers start at the beginning of the data structure. The window is expanded or contracted by moving these pointers to explore different parts of the data.

⁵² Two pointers

The key idea is that the two pointers move in the same direction and never overtake each other⁵³. This ensures that each value is only visited at most twice, resulting in a time complexity of $O(n)$ ⁵⁴.

⁵³ Pointers move in same direction

⁵⁴ Time complexity is $O(n)$

- **Fixed-length Window:** In problems where the window size is fixed, one pointer advances while the other remains static until the window is fully formed, then both pointers move together.
- **Variable-length Window:** For problems where the window size can change, both pointers move independently to expand or contract the window based on specific conditions.

Applications of Sliding Window Technique

The sliding window technique is versatile and can be applied to a variety of problems⁵⁵. Here are a few common applications:

⁵⁵ Versatile technique

Maximum Sum Subarray of Size k

Introduction

The problem of finding the Maximum Sum Subarray of Size k is a classic example of using the sliding window technique. In this problem, you are given an array of integers and a number k . Your task is to find the maximum sum of any contiguous subarray of length k .

Python Implementation

Here's the Python code to solve the problem using the sliding window technique:

```
def max_sum_subarray(arr, k):
    n = len(arr)
    if n < k:
        return "Invalid"

    # Calculate the sum of the first window
    window_sum = sum(arr[:k])
    max_sum = window_sum

    # Slide the window over the array
    for i in range(k, n):
        window_sum += arr[i] - arr[i - k]
        max_sum = max(max_sum, window_sum)

    return max_sum

# Example usage:
arr = [2, 1, 5, 1, 3, 2]
k = 3
print(max_sum_subarray(arr, k)) # Output: 9
```

By maintaining a window of elements and updating the sum as the window moves, you ensure each element is processed only a few times, resulting in an efficient $O(n)$ solution.

Longest Substring with At Most k Distinct Characters

Introduction

A more complex problem is finding the length of the longest substring with at most k distinct characters. Given a string s and an integer k , the task is to determine the longest substring that contains at most k distinct characters. This

problem is a bit more challenging as it involves dynamically changing the window size based on the number of distinct characters.

Python Implementation

Here's the Python code to solve this problem using the sliding window technique:

```
def length_of_longest_substring_k_distinct(s, k):
    from collections import defaultdict
    n = len(s)
    left = 0
    max_length = 0
    char_count = defaultdict(int)

    for right in range(n):
        char_count[s[right]] += 1

        # Ensure there are at most k distinct characters in the
        # ↪ window
        while len(char_count) > k:
            char_count[s[left]] -= 1
            if char_count[s[left]] == 0:
                del char_count[s[left]]
            left += 1

        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
s = "eceba"
k = 2
print(length_of_longest_substring_k_distinct(s, k)) # Output: 3
```

In this solution, a dictionary is used to count the frequency of characters in the current window. The window expands by moving the right pointer, and contracts by moving the left pointer whenever there are more than k distinct characters. The size of the window is adjusted dynamically, ensuring that the solution is efficient with a time complexity of $O(n)$. The `defaultdict(int)` is a specialized dictionary from Python's `collections` module that initializes any new key with a default integer value of 0⁵⁶. This feature simplifies the code by eliminating the need to check if a key exists before incrementing its value, making the character count management more efficient.

⁵⁶ `defaultdict(int)`

In the implementation, two pointers (`left` and `right`) are used to represent the sliding window⁵⁷. The `right` pointer expands the window by moving through each character in the string `s`, while the `left` pointer contracts the window when the number of distinct characters exceeds k .

⁵⁷ Two pointers

Here's a step-by-step breakdown of the process:

1. Initialize the `left` pointer at the start of the string and the `max_length`

variable to keep track of the longest valid substring found⁵⁸.

⁵⁸ Initialization

2. Use a `for` loop to iterate through the string with the `right` pointer, expanding the window by including `s[right]` and updating the `char_count` dictionary⁵⁹.
3. Inside the loop, use a `while` loop to check if the number of distinct characters in `char_count` exceeds k . If it does, decrement the count of the character at the `left` pointer and move the `left` pointer to the right, effectively shrinking the window until the constraint is satisfied⁶⁰.
4. After adjusting the window, update `max_length` with the size of the current window if it is larger than the previously recorded maximum⁶¹.

⁵⁹ Expanding the window

⁶⁰ Shrinking the window

⁶¹ Updating maximum length

The window size dynamically adjusts based on the number of distinct characters, ensuring the algorithm efficiently finds the longest substring that meets the condition. Each character is processed at most twice—once when the `right` pointer includes it in the window and once when the `left` pointer excludes it—resulting in a linear time complexity of $O(n)$ ⁶².

⁶² Linear time complexity

Minimum Window Substring

Introduction

Given two strings s and t , the task is to find the minimum window in s which will contain all the characters in t . This problem is more complex as it involves dynamically adjusting the window size based on the presence of required characters.

Python Implementation

Here's the Python code to solve this problem using the sliding window technique:

```
def min_window_substring(s, t):
    from collections import Counter
    t_count = Counter(t)
    current_count = Counter()
    start = 0
    min_length = float('inf')
    left = 0

    for right, char in enumerate(s):
        current_count[char] += 1

        # Ensure the window contains all characters of t
        while all(current_count[c] >= t_count[c] for c in t_count):
            if right - left + 1 < min_length:
                start = left
                min_length = right - left + 1
            current_count[s[left]] -= 1
            if current_count[s[left]] == 0:
                del current_count[s[left]]
            left += 1
```

```

        left += 1

    return "" if min_length == float('inf') else s[start:start +
    ↪ min_length]

# Example usage:
s = "ADOBECODEBANC"
t = "ABC"
print(min_window_substring(s, t)) # Output: "BANC"

```

Detailed Explanation

The `Counter` from the `collections` module is used to keep track of the frequency of characters in the target string t and the current window of the string s ⁶³. This helps in efficiently managing and checking the required characters.

In the implementation, two pointers (`left` and `right`) are used to represent the sliding window⁶⁴. The `right` pointer expands the window by moving through each character in the string s , while the `left` pointer contracts the window when the window contains all characters of t .

Here's a step-by-step breakdown of the process:

1. Initialize `left` pointer at the start of the string and set `min_length` to infinity to keep track of the smallest valid window found⁶⁵.
2. Use a `for` loop to iterate through the string with the `right` pointer, expanding the window by including `s[right]` and updating the `current_count` dictionary⁶⁶.
3. Inside the loop, use a `while` loop to check if the current window contains all characters of t by comparing `current_count` with `t_count`⁶⁷. If the condition is satisfied, attempt to minimize the window by moving the `left` pointer to the right, updating `current_count` accordingly.
4. Whenever a valid window is found (i.e., it contains all characters of t), update `start` and `min_length` if the current window is smaller than the previously recorded minimum window⁶⁸.

Explanation of Loops and Window Sizes

The outer `for` loop iterates through each character in s with the `right` pointer, expanding the window one character at a time⁶⁹. The `while` loop checks if the current window contains all characters required by t . If so, it tries to minimize the window by incrementing the `left` pointer until the window no longer satisfies the condition⁷⁰.

The dynamic adjustment of the window size ensures that the smallest possible window containing all characters of t is found. Each character in s is processed at most twice—once when the `right` pointer includes it and once when the `left` pointer excludes it—resulting in a linear time complexity of $O(n)$ ⁷¹.

⁶³ Using Counter

⁶⁴ Two pointers

⁶⁵ Initialization

⁶⁶ Expanding the window

⁶⁷ Checking window contents

⁶⁸ Updating minimum length

⁶⁹ Outer loop

⁷⁰ Inner loop

⁷¹ Linear time complexity

Conclusion

The sliding window technique is a fundamental tool in the programmer's toolkit⁷². By efficiently managing a subset of data within a "window" and moving this window across the dataset, many complex problems can be solved in linear time. Understanding and mastering this technique will greatly enhance your ability to tackle array and string problems effectively⁷³.

⁷² Fundamental tool

⁷³ Mastering the technique

Best Time to Buy and Sell Stock

The "Best Time to Buy and Sell Stock" problem is a common interview question that assesses a candidate's ability to work with arrays and apply dynamic programming concepts. The problem requires finding the optimal buy and sell times to maximize profit from a series of daily stock prices.

Problem Statement

You are given an array `prices`, where `prices[i]` is the price of a given stock on the i th day. Your goal is to determine the maximum profit obtainable by buying on one day and selling on a later day. If no profit can be made, return 0.



Figure 1: Best Time to Buy and Sell Stock.

Input: An array `prices`, with `prices[i]` representing the stock price on the i th day.

Output: The maximum profit that can be achieved.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6),

```
profit = 6 - 1 = 5.
```

Example 2:

```
Input: prices = [7,6,4,3,1]
```

```
Output: 0
```

```
Explanation: No transaction can yield a profit, hence the max profit = 0.
```

Algorithmic Approach

The common approach to solving this problem is to use a single pass. Iterate through the `prices` array while tracking the minimum price encountered so far. For each price, calculate the profit if the stock were sold at that price, updating the maximum profit as necessary.

Complexities

- **Time Complexity:** The time complexity of this algorithm is $O(n)$, where n is the number of days, since it requires a single pass through the array.
- **Space Complexity:** The space complexity is $O(1)$, as only a constant amount of extra space is needed for variables such as `min_price` and `max_profit`.

Python Implementation

The Python implementation uses the above algorithm to find the maximum profit with a single pass through the stock prices:

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        min_price = float('inf') # Initialize to positive infinity
        max_profit = 0

        # Loop through all stock prices
        for price in prices:
            # Updating the minimum price seen so far
            if price < min_price:
                min_price = price
            # Computing potential profit and update max profit
            elif price - min_price > max_profit:
                max_profit = price - min_price
        return max_profit
```

This approach initializes the minimum price to infinity and the maximum profit to 0 and iterates over the `prices` array. For each stock price, it checks if the price is lower than the current minimum price. If it is, the minimum price is updated. Otherwise, it calculates the profit one would get if one sold at the current price and updates the maximum profit if the current profit is larger. After iterating through all the prices, the maximum profit is returned.

Why this approach

This approach is chosen for its efficiency in both time and space. It navigates the array only once, ensuring a linear time complexity, which is optimal in this case because you have to examine each price at least once to determine the profit.

Alternative approaches

Alternative methods might use more complex data structures, such as segment trees, to perform many queries on the range of days. However, this is overkill for the given problem and would not improve the best-case time complexity.

Similar problems to this one

Similar problems include variations that allow for multiple transactions, incorporate a cooldown period between sales, or limit the number of transactions.

Things to keep in mind and tricks

One trick for problems like this is to consider the running difference between the current and minimum prices as the "current profit" and to update the maximum profit if a higher current profit is found.

Corner and Special Cases to Test When Writing the Code

Test cases should include: - Increasing and decreasing prices, ensuring profit is calculated correctly or confirmed as zero. - Large input arrays to verify performance. - Edge cases with the minimum input size (e.g., an array of just one price).

Explanation of Sliding Window Concept

In this problem, the sliding window is not explicitly represented by two pointers, but the concept is similar. The window dynamically adjusts as we iterate through the `prices` array, always keeping track of the lowest buying price and the highest potential selling price after that buying day. This ensures that each price is considered only once, resulting in a linear time complexity of $O(n)$ ⁷⁴.

⁷⁴ Linear time complexity

Longest Substring Without Repeating Characters

The "Longest Substring Without Repeating Characters" problem is a frequently addressed question in coding interviews, focused on strings and the utilization of data structures to optimize search and retrieval operations. The task is to sift through the sequence of characters in a given string, identifying the lengthiest possible segment that consists purely of non-repeating characters.



Figure 2: A scientist studying the Mayan alphabet to decode the longest substring without repeating characters.

Problem Statement

Given a string s , the challenge lies in discerning the length of the longest substring devoid of recurrent characters. For example, within the string "abcabcbb", the longest substring that meets the criteria is "abc", which constitutes 3 characters. In contrast, for a string like "bbbb", the longest such substring is simply "b", reflecting a length of 1. The crux of the matter is to efficiently navigate through the string, implementing a strategy that maximizes the window of unique characters whilst conforming to the constraints set forth.

Algorithmic Approach

To unravel this problem, a sliding window technique is employed, enhancing the algorithm's capability to dynamically adjust the extent of the observed substring⁷⁵. The primary objective is to expand this window by traversing the string and keeping a vigilant eye for duplicating elements. The moment a recurring character manifests, it necessitates a recalibration of the window's scope, excising the previously occurring instance of said character before progressing. To effectuate this methodology with efficacy, data structures such as hash maps or sets, or even arrays tailored for indexing, are instrumental in maintaining a ledger of character occurrences along with their respective positions.

⁷⁵ Sliding window technique

Complexities

- **Time Complexity:** The overarching time complexity of the solution pivots around $O(n)$, as the algorithm meticulously processes each character in the string a singular time — a testament to the efficiency of the sliding window methodology⁷⁶.
- **Space Complexity:** The auxiliary space complexity, contingent upon the breadth of the character set employed in the hash map or set, invariably approaches $O(m)$, where m represents the size of the character alphabet (in the case of ASCII, potentially up to 128)⁷⁷.

⁷⁶ Time complexity $O(n)$

⁷⁷ Space complexity $O(m)$

Below is the complete Python code for solving the "Longest Substring Without Repeating Characters" problem by employing a sliding window approach:

Algorithm Explanation

This implementation uses a dictionary, `char_index_map`, to record the most recent index of each character encountered in the string. It initially sets the starting point of the window, `start`, to 0. As it iterates over the string, the algorithm updates the start index if a repeated character is found within the current window. It then calculates the current window size and updates `max_length` if a larger window is found.

```
def length_of_longest_substring(s):
    char_index_map = {}
    max_length = 0
    start = 0

    for i, char in enumerate(s):
        if char in char_index_map and char_index_map[char] >= start:
            ↪ :
            start = char_index_map[char] + 1
        char_index_map[char] = i
        max_length = max(max_length, i - start + 1)
```

```

    return max_length

# Example usage:
s = "abcabcbb"
print(length_of_longest_substring(s)) # Output: 3

```

Why this approach

The sliding window technique is favored in this context due to its real-time responsiveness to evolving conditions within the string. It streamlines operations by eliminating the need for redundant inspections of previously reviewed characters and concentrates computational efforts solely on the changing window edges, thereby enhancing the procedure's efficiency⁷⁸.

⁷⁸ Efficiency of sliding window

Alternative approaches

An alternative approach to the sliding window technique could involve brute force, which entails inspecting every possible substring for duplicates, a method that is computationally exhaustive and less feasible for longer strings. Another potential approach might utilize dynamic programming to keep track of the longest substring without repeating characters up to each index, but this is generally less efficient than the adopted sliding window strategy⁷⁹.

⁷⁹ Alternative approaches

Similar problems to this one

There are several problems akin to the "Longest Substring Without Repeating Characters," such as "Longest Substring with At Most Two Distinct Characters," "Longest Repeating Character Replacement," and "Substring with Concatenation of All Words." Each of these presents unique requisites that necessitate slight modifications to the basic sliding window construct⁸⁰.

⁸⁰ Similar problems

Things to keep in mind and tricks

When dealing with sliding window problems, an imperative trick is to adequately manage the window boundaries, especially when the window contracts. It's crucial to ensure that the starting boundary does not revert backwards, as this would lead to incorrect computations⁸¹.

⁸¹ Managing window boundaries

Corner and special cases to test when writing the code

One should be alert to scenarios involving extensive sequences of identical characters, strings consisting solely of unique characters, or strings with alternating patterns. Additionally, edge cases such as an empty string should be contemplated. It is advisable to conduct thorough testing against these circumstances to affirm the robustness of the algorithm⁸².

⁸² Corner cases

Longest Substring with At Most Two Distinct Characters

The "Longest Substring with At Most Two Distinct Characters" problem is an excellent example of using the sliding window technique to handle constraints involving substrings with character limits.

Problem Statement

Given a string s , find the length of the longest substring that contains at most two distinct characters.

Input: A string s .

Output: The length of the longest substring containing at most two distinct characters.

Example:

Input: $s = \text{"eceba"}$

Output: 3

Explanation: The substring is "ece" with length 3.

Example 2:

Input: $s = \text{"ccaabbb"}$

Output: 5

Explanation: The substring is "aabbb" with length 5.

Algorithmic Approach

To solve this problem, we use the sliding window technique. We maintain a window that expands to include new characters and contracts when the number of distinct characters exceeds two. By using a hash map to count character frequencies within the window, we can efficiently manage the character counts and ensure the window contains at most two distinct characters.

Complexities

- **Time Complexity:** The time complexity is $O(n)$, where n is the length of the string. Each character is processed at most twice, making the solution linear in time.

- **Space Complexity:** The space complexity is $O(1)$ because the hash map will store at most three distinct characters at any time.

Python Implementation

Below is the complete Python code for solving the "Longest Substring with At Most Two Distinct Characters" problem using the sliding window technique:

```
def length_of_longest_substring_two_distinct(s):
    char_count = {}
    max_length = 0
    start = 0

    for end in range(len(s)):
        if s[end] in char_count:
            char_count[s[end]] += 1
        else:
            char_count[s[end]] = 1

        while len(char_count) > 2:
            char_count[s[start]] -= 1
            if char_count[s[start]] == 0:
                del char_count[s[start]]
            start += 1

        max_length = max(max_length, end - start + 1)

    return max_length

# Example usage:
s = "eceba"
print(length_of_longest_substring_two_distinct(s)) # Output: 3
```

This implementation maintains a hash map, `char_count`, to record the frequency of each character in the current window. It expands the window by including new characters and contracts it when there are more than two distinct characters, updating the maximum length of valid substrings as needed.

Why this approach

The sliding window technique is chosen here for its efficiency in handling character constraints. It ensures that each character is processed at most twice, which makes the solution optimal and straightforward.

Alternative approaches

An alternative approach could involve brute force, where you check all possible substrings and count the distinct characters. However, this method would be

computationally expensive with a time complexity of $O(n^3)$, making it impractical for longer strings.

Similar problems to this one

Similar problems include "Longest Substring with At Most K Distinct Characters" and "Longest Substring Without Repeating Characters," where the sliding window technique is also applied to find substrings under different constraints.

Things to keep in mind and tricks

When dealing with sliding window problems, a useful trick is to use a hash map to keep track of the counts of characters within the window. This helps efficiently manage the window's constraints and ensures that the solution remains optimal.

Corner and Special Cases to Test When Writing the Code

Test cases should include:

- Strings with no characters or a single character.
- Strings where all characters are the same.
- Strings where each character is unique.
- Large strings to verify performance.
- Edge cases such as an empty string or strings with fewer than two distinct characters.

Minimum Size Subarray Sum

The "Minimum Size Subarray Sum" problem is another excellent example of the sliding window technique in action. This problem challenges one to find the smallest contiguous subarray whose sum is at least a given target.

Problem Statement

Given an array of positive integers `nums` and a positive integer `s`, find the minimal length of a contiguous subarray of which the sum is greater than or equal to `s`. If there isn't one, return 0 instead.

Input: An integer `s` and an array of positive integers `nums`.

Output: The minimal length of a contiguous subarray with a sum $\geq s$.

Example:

Input: `s = 7, nums = [2,3,1,2,4,3]`

Output: 2

Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

Algorithmic Approach

To solve this problem, we employ the sliding window technique. The idea is to expand the window by adding elements to the sum until it is greater than or equal to `s`. Once the sum is sufficient, we contract the window from the left to find the minimal length while still maintaining the sum $\geq s$.

Complexities

- **Time Complexity:** The time complexity of this algorithm is $O(n)$, where n is the length of the array. This is because each element is added and removed from the window at most once.
- **Space Complexity:** The space complexity is $O(1)$ as we are using only a few extra variables for the window sum, start pointer, and the minimum length.

Python Implementation

Below is the complete Python code for solving the "Minimum Size Subarray Sum" problem using the sliding window technique:

```
def min_subarray_len(s, nums):
    n = len(nums)
    min_length = float('inf')
    current_sum = 0
    start = 0

    for end in range(n):
        current_sum += nums[end]

        while current_sum >= s:
            min_length = min(min_length, end - start + 1)
            current_sum -= nums[start]
            start += 1

    return 0 if min_length == float('inf') else min_length

# Example usage:
s = 7
nums = [2, 3, 1, 2, 4, 3]
print(min_subarray_len(s, nums)) # Output: 2
```

This implementation maintains a running sum of the current window. If the sum becomes greater than or equal to *s*, it tries to shrink the window from the left to find the minimum length. It updates the minimum length whenever the sum condition is satisfied.

Why this approach

The sliding window technique is chosen here for its efficiency in handling contiguous subarrays. It ensures that each element is processed at most twice, once when added to the window and once when removed, making it highly efficient.

Alternative approaches

An alternative approach could involve using binary search along with prefix sums to find the minimum subarray length. However, this approach would be more complex and less intuitive than the sliding window method.

Similar problems to this one

Similar problems include "Longest Substring with At Most Two Distinct Characters" and "Longest Repeating Character Replacement," where the sliding window technique is also applied to find subarrays or substrings under certain constraints.

Things to keep in mind and tricks

One important trick is to remember that as soon as the window sum exceeds s , you should try to shrink the window from the left to find the minimal length subarray. This ensures that the solution remains optimal.

Corner and Special Cases to Test When Writing the Code

Test cases should include: - Arrays where no subarray meets the sum condition, returning 0. - Large arrays to verify performance. - Edge cases with the minimum input size (e.g., an array with one element).

Contains Duplicate

The "Contains Duplicate" problem is a common coding challenge that requires a straightforward solution to determine if an array contains any repeated elements.

Problem Statement

Given an integer array *nums*, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Example 1:

Input: *nums* = [1,2,3,1]

Output: `true`

Example 2:

Input: *nums* = [1,2,3,4]

Output: `false`

Example 3:

Input: *nums* = [1,1,1,3,3,4,3,2,4,2]

Output: `true`

Algorithmic Approach

There are several approaches to solve this problem:

1. Sorting: Sort the array and check for consecutive elements being equal.
2. Hashing: Use a hash set to keep track of visited elements.

The solution can be implemented using either approach, but using a hash set typically results in a time-efficient and straightforward solution. The follow-up question challenges the possibility of not using any extra space, which would then likely lead us to modify the array in-place, such as performing a sort operation.

Complexities

- **Time Complexity:** The time complexity for the sorting approach is $O(n \log n)$ due to the sort operation. For the hashing approach, it is $O(n)$ in the average case, as set operations have a constant time complexity on average.
- **Space Complexity:** The space complexity is $O(n)$ for the hashing approach as it requires storing up to n elements in the set. For sorting, if the in-place sort is

used, then the space complexity can be $O(1)$.

Python Implementation

Below is the complete Python code implementing the hashing approach to determining if a list contains any duplicates:

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        seen = set()
        for number in nums:
            if number in seen:
                return True
            seen.add(number)
        return False
```

This implementation uses a set to store elements that have already been processed. As we iterate over the list, we check if each element is already present in the set, indicating a duplicate. If so, we return `true`; otherwise, we add the element to the set. If we finish iterating without finding duplicates, we return `false`.

Why this approach

The hashing approach was chosen because it provides an average-time complexity that is linear with respect to the number of elements in the array. It's also a simple and elegant solution that is easy to implement.

Alternative approaches

An alternative approach is to sort the array first and then look for consecutive equal elements. This removes the need for extra space (apart from the space needed to sort the array), complying with the follow-up requirement. However, sorting the array will increase the time complexity to $O(n \log n)$, making it less efficient than the hashing approach in terms of time.

Similar problems to this one

Similar problems often involve finding duplicates or checking for the existence of certain elements within a dataset. Problems such as "Find the Duplicate Number," "Find All Duplicates in an Array," and "Contains Duplicate II" are variations that pose additional constraints or slight modifications to the underlying problem of detecting duplicates.

Things to keep in mind and tricks

In problems like these, hashing is a common trick that leverages the high-speed access time of a hash table to quickly check for the existence of an element. It's

important to consider the trade-off between time and space complexity when choosing the approach to implement.

Corner and special cases to test when writing the code

When implementing solutions, it is crucial to handle corner cases, such as empty arrays or arrays with a single element where no duplicates can exist. Additionally, for the hashing approach, one must consider the hash set's potential impact on the runtime if the dataset contains many elements, causing hash collisions and degrading performance to $O(n^2)$ in the worst case.

Valid Anagram

Problem Statement

Given two strings 's' and 't', write a function to determine if 't' is an anagram of 's'. An Anagram is a word or phrase formed by rearranging the letters of another word or phrase, typically using all the original letters exactly once.

For example, the word "anagram" can be rearranged into "nag a ram".

Note that two empty strings are considered to be anagrams of each other.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

Example 2:

Input: s = "rat", t = "car"

Output: false

LeetCode link: [Valid Anagram](#)

Algorithmic Approach

To determine if one string is an anagram of another, we can count the occurrences of each letter in both strings and then compare these counts. This can be accomplished with a hash table or an array of fixed size if the strings only consist of lowercase alphabetic characters.

Complexities

- **Time Complexity:** The time complexity of the algorithm is $O(n)$, where n is the length of the input strings, because we have to count each letter in both strings.
- **Space Complexity:** The space complexity is $O(1)$ because the additional space used by the algorithm depends only on the size of the alphabet used in the strings and not on the length of the strings themselves.

Python Implementation

Below is the complete Python code for the 'isAnagram' function to check if one string is an anagram of another:

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False

        count = [0] * 26 # Since the problem statement mentions
                        ↪ only lowercase alphabets

        for char in s:
            count[ord(char) - ord('a')] += 1
        for char in t:
            count[ord(char) - ord('a')] -= 1

        for c in count:
            if c != 0:
                return False

        return True
```

This implementation counts the frequency of each character in the string 's' and 't' by incrementing and decrementing the value at the respective indices in the 'count' array. After processing both strings, if all elements in the 'count' array are zeros, the strings are anagrams; otherwise, they aren't.

Why this approach

This approach was chosen due to its efficiency in both time and space complexity. By utilizing an array of fixed size to count character occurrences instead of a hash table, we ensure constant time operations for updating counts, which significantly reduces the overhead and maintains linear time complexity with respect to the size of the input.

Alternative approaches

An alternative approach involves sorting both strings and comparing them to check for equality. Although intuitive, this method has a higher time complexity of $O(n \log n)$, where n is the length of the strings due to the sorting operation.

Similar problems to this one

Similar problems may ask for permuting strings to match patterns, checking if a string can break another, or finding the minimum number of steps to make two strings anagrams. These variations usually apply a similar concept of character frequency counting but tweak the problem constraints or goals.

Things to keep in mind and tricks

Remember that direct array access is faster than using a hash table for counting when the set of possible characters is small and fixed, as is the case with lowercase English letters.

Corner and special cases to test when writing the code

Ensure that you consider and test for edge cases such as one or both strings being empty, strings of differing lengths (which should immediately return 'false'), and strings with the same characters but different frequencies.

Group Anagrams

The "Group Anagrams" problem is an interesting question that requires understanding of hash tables and string manipulation. It is a typical interview question that checks the candidate's ability to handle and classify data based on specific rules, in this case, anagrams.

Problem Statement

Given an array of strings `strs`, the task is to group the anagrams together. An **Anagram** is a word or phrase that is formed by rearranging the letters of a different word or phrase, using all the original letters exactly once.

LeetCode link: [Group Anagrams](#)

Algorithmic Approach

To solve this problem, we can categorize each string by its character count or sorted version. We will use a hash table to maintain a list of strings that, when sorted, yield the same string. Each key in the hash table will represent a sorted string, and its value will be a list of strings that are anagrams of each other.

Complexities

- **Time Complexity:** The time complexity is $O(NK \log K)$, where N is the length of `strs`, and K is the maximum length of a string in `strs`. This accounts for the sorting of each string.
- **Space Complexity:** The space complexity is $O(NK)$, to store the result in the hash table.

Python Implementation

Below is the complete Python code that implements the grouping of anagrams by using a hash table:

```
def groupAnagrams(strs):
    anagrams = {}
    for s in strs:
        sorted_s = ''.join(sorted(s))
        if sorted_s not in anagrams:
            anagrams[sorted_s] = [s]
        else:
            anagrams[sorted_s].append(s)
    return list(anagrams.values())
```

The function 'groupAnagrams' takes a list of strings and classifies them based on their sorted order, effectively grouping the anagrams together.

Why This Approach

This approach is chosen because it effectively utilizes the property that anagrams share the same sorted sequence of characters. Sorting the strings provides a consistent key to group the anagrams. Sorting each string takes $O(K \log K)$, and doing this for N strings results in the time complexity of $O(NK \log K)$.

Alternative Approaches

An alternative to sorting might be to use a character count as the key in the hash table, which could potentially reduce the time complexity to $O(NK)$ since calculating the character count of a string takes linear time with respect to the string length.

Similar Problems to This One

There are various problems that require categorization or grouping based on certain characteristics, such as "Valid Sudoku", which asks to categorize numbers based on their positions in a grid, or "Find Duplicate File in System", where files need to be grouped by their content.

Things to Keep in Mind and Tricks

- Utilizing sorting or counting as the way to identify anagrams.
- Being careful with how the hash table handles collisions and understanding how the lookups work.

Corner and Special Cases to Test When Writing the Code

- Input list containing empty strings. - Strings with varying lengths. - Ensuring that strings with the same characters but in different orders are grouped correctly.

Longest Repeating Character Replacement

The "Longest Repeating Character Replacement" problem is a classic example of how a combination of string manipulation and the sliding window technique can be employed to solve complex problems efficiently. In this problem, you are given a string 's' and an integer 'k' representing the maximum number of character replacements you are allowed to perform in order to create the longest possible substring containing the same letters.

Problem Statement

The aim is to find the length of the longest substring of 's' which can be obtained by replacing at most 'k' characters within the substring. The replacement can be done with any uppercase English letter.

Example:

Input: $s = "AABABBA", k = 1$

Output: 4

Explanation:

When given the string $s = "AABABBA"$ with $k = 1$, it means you can replace one character at most. One optimal approach is to substitute one occurrence of 'B' in "AABA" to 'A', resulting in the substring "AAAA". This substring has a length of 4, thus being the longest substring possible with at most one character replacement.

LeetCode link: <https://leetcode.com/problems/longest-repeating-character-replacement/>

Algorithmic Approach

To tackle this issue, we employ a sliding window approach. We maintain a count of the most commonly occurring character in the current window and try to substitute the other characters. If the number of characters we need to replace is more than k , the window is too wide, so we reduce it by moving the start of the window to the right. If not, we can expand the window by shifting the end further to the right. With this method, the window length always equals a valid substring where characters can be replaced k times or fewer to match each other.

Complexities

- **Time Complexity:** The overall time complexity is $O(n)$, where n is the length of the string s . This is because each character in s is processed at most twice - once when extending the window and once when shrinking it.

Python Implementation

The following is the complete Python implementation for solving the "Longest Repeating Character Replacement" issue:

```
from collections import Counter

def characterReplacement(s, k):
    count = Counter()
    max_count = 0
    max_length = 0
    left = 0

    for right in range(len(s)):
        count[s[right]] += 1
        max_count = max(max_count, count[s[right]])

        if right - left + 1 - max_count > k:
            count[s[left]] -= 1
            left += 1

        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
s = "AABABBA"
k = 1
print(characterReplacement(s, k)) # Output: 4
```

This code first creates a ‘Counter’ to keep track of character frequencies. As we slide the window across the string, we update the counter to reflect the characters within the window and calculate the longest valid substring at each step.

Why this approach

The sliding window approach is chosen due to its efficiency in solving problems related to strings and substrings, especially when the goal is to optimize for lengths of certain conditions within the string. It significantly decreases the time complexity compared to other brute force methods that would otherwise have higher computational costs.

Alternative approaches

An alternative approach might involve dynamic programming or brute force checking of all possible substrings, but these methods would not be as efficient. The sliding window technique is specifically well-suited for this problem due to its linear time complexity.

Similar problems to this one

There are various other problems where a sliding window technique is applicable, such as "Minimum Window Substring", "Longest Substring Without Repeating Characters", and "Subarrays with K Different Integers". Each of these problems involves optimizing a particular property of the substring (e.g., length, number of unique characters) while maintaining a window that adheres to problem-specific constraints.

Things to keep in mind and tricks

Remember to always check that the window size minus the frequency of the most common element does not exceed k . It's a good practice to keep track of the frequency of the most common element as it helps to quickly evaluate whether the current window can potentially lead to an optimal solution.

Corner and special cases to test when writing the code

Make sure to test the code with strings where all characters are the same, all characters are unique, and cases where no character replacements are required. It's also important to consider edge cases where k is zero or equal to the string's length, testing the algorithm's handling of the entire range of possible values for k .

Maximum Product Subarray

The "Maximum Product Subarray" problem requires finding the subarray within a given array of integers that has the largest product, and returning that product. Data structures and algorithms such as dynamic programming and arrays are utilized in solving this problem.

Problem Statement

Given an integer array `nums`, the objective is to find a contiguous non-empty subarray within the array that has the largest product, and to return the product. A subarray is defined as a contiguous part of an array.

Example:

- **Input:** `nums = [2,3,-2,4]`
- **Output:** 6
- **Explanation:** The subarray `[2,3]` has the largest product 6.
- **Input:** `nums = [-2,0,-1]`
- **Output:** 0
- **Explanation:** The result cannot be 2, because `[-2,-1]` is not a subarray.

The problem can be more complex as the input array `nums` may contain positive, negative, and zero values. The solution thus requires careful consideration of these cases and how they affect the product of a subarray.

LeetCode link: [Maximum Product Subarray](#)

Algorithmic Approach

The efficient solution to this problem involves dynamic programming, where we keep track of both the maximum and minimum products up to each element of the array. This is important because a large negative product can turn into a large positive product when multiplied by another negative number. We iterate over the array once, maintaining the maximum and minimum product that ends at the current element, and update our global maximum product accordingly.

Complexities

- **Time Complexity:** The time complexity of this algorithm is $O(n)$ as it makes a single pass through the array, where n is the number of elements in the array.
- **Space Complexity:** The space complexity is $O(1)$ as only a constant amount of extra space is used, apart from the input array.

Python Implementation

Below is the complete Python code for the 'Solution' class, which implements the 'maxProduct' method to find the maximum product subarray:

```
class Solution:
    def maxProduct(self, nums):
        # If the array is empty, return 0
        if not nums:
            return 0

        # Initialize max_product, min_product, and result with
        # ↪ the first element
        max_product = min_product = result = nums[0]

        # Iterate through nums starting from the second element
        for i in range(1, len(nums)):
            # If the current element is negative, swap
            # ↪ max_product and min_product
            if nums[i] < 0:
                max_product, min_product = min_product,
                # ↪ max_product

            # Update max_product and min_product
            max_product = max(nums[i], max_product * nums[i])
            min_product = min(nums[i], min_product * nums[i])

            # Update result
            result = max(result, max_product)

        return result
```

This implementation utilizes a single pass solution while maintaining a current maximum and minimum which are updated at each step. If a negative number is encountered, the current maximum and minimum are swapped before updating, since multiplying by a negative number reverses their roles. The `max_product` is updated by taking the maximum value between the current number and the product of `max_product` and the current number. Similarly, `min_product` is updated with the minimum value. With each iteration, the global result is updated with the maximum of itself and the current `max_product`.

Why This Approach

The approach taken efficiently calculates the maximum product of any subarray with $O(n)$ time and $O(1)$ space complexities. The key insight is to keep track of both maximum and minimum values due to the presence of negative numbers in the array, which can change the sign of the product when multiplied. This algorithmic decision avoids the need to compute all possible subarrays, thus reducing the time complexity from $O(n^2)$ to $O(n)$.

Alternative Approaches

A naive approach might involve calculating the product of all possible subarrays, leading to an $O(n^2)$ time complexity. Another approach could use divide and conquer to solve the problem in $O(n \log n)$ time, but this is still not as efficient as the dynamic programming solution provided.

Similar Problems to This One

Similar problems include other array dynamic programming challenges such as "Best Time to Buy and Sell Stock" or "Maximum Subarray Sum". The solutions typically involve keeping track of local and global optimal values while iterating through the array.

Things to Keep in Mind and Tricks

Remember that the presence of zeros and negative numbers significantly affects the approach to solving this problem. Special attention must be given to the treatment of negative numbers and zeros when maintaining the running product. Also, resetting the running product to 1 when encountering a zero is important because the product of any subarray containing zero is zero.

Corner and Special Cases to Test When Writing the Code

When testing the implementation, consider corner cases such as arrays with a single element, arrays with all positive or all negative numbers, arrays with one or more zeros, and arrays where the maximum product is the product of the entire array. It is crucial to test arrays with alternating positive and negative numbers and arrays with large subarrays with a product of zero.

Non-overlapping Intervals

The "Non-overlapping Intervals" problem is a common interview question that falls under the category of interval scheduling optimization problems. The main goal of the problem is to find the minimum number of intervals you can remove from a set of intervals such that none of the remaining intervals overlap with each other.

Problem Statement

Given a collection of intervals, find the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Input:

An array of intervals where `intervals[i] = [start_i, end_i]`.

Output:

Return the minimum number of intervals that need to be removed to make the rest of the intervals non-overlapping.

Examples

Example 1:

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: By removing the interval `[1,3]`, the remaining intervals `[1,2]`, `[2,3]`, and `[3,4]` are non-overlapping.

LeetCode link: [Non-overlapping Intervals](#)

Algorithmic Approach

To solve the problem, the steps are:

1. Sort the intervals based on their ending point.
2. Iterate through the sorted intervals, keeping track of the end of the last interval in the non-overlapping set.
3. If an interval starts after the end of the last interval added, it can be added to the set.
4. If an interval does overlap, remove the interval with the later end.
5. Count the number of intervals removed during this process to obtain the solution.

Complexities

- **Time Complexity:** Sorting the intervals takes $O(n \log n)$ time. The subsequent iteration takes $O(n)$, leading to a total time complexity of $O(n \log n)$.
- **Space Complexity:** The space complexity is $O(1)$ if the sorting is done in place. Otherwise, it might be $O(n)$ depending on the implementation of the sorting algorithm.

Python Implementation

Below is the complete Python code for solving the "Non-overlapping Intervals" problem:

```
class Solution:
    def eraseOverlapIntervals(self, intervals):
        if not intervals:
            return 0

        # Sort intervals based on their ending value
        intervals.sort(key=lambda x: x[1])

        # Initialize the end of the first interval
        end = intervals[0][1]
        # Count the intervals that are kept
        cnt = 1

        for x in intervals[1:]:
            if x[0] >= end:
                # If the interval does not overlap, keep it
                cnt += 1
                end = x[1]
            # If it does overlap, skip it (since we have
            # already sorted by end, no need to remove it
            # → explicitly)

        # The result is the total count minus the number of
        # intervals we have kept
        return len(intervals) - cnt

# Example usage:
intervals = [[1,2],[2,3],[3,4],[1,3]]
sol = Solution()
print(sol.eraseOverlapIntervals(intervals)) # Output: 1
```

The code begins by handling the edge case where the given list of intervals is empty. Afterward, it sorts the intervals based on their end times to choose the intervals that finish earliest. It then iterates through the sorted intervals, checking to see if the current interval overlaps with the last kept interval. If it doesn't overlap, we increment the count of kept intervals and update the end variable. The number of intervals erased is the total number of intervals minus the count of kept intervals.

Why this approach

The greedy approach was chosen for its efficiency and because it guarantees an optimal solution for interval scheduling problems. By always selecting the interval with the earliest end time, we maximize the number of intervals that can be included without overlap.

Alternative approaches

An alternative, less optimal, approach is to use dynamic programming to find the longest sequence of non-overlapping intervals and subtract its length from the total number of intervals.

Similar problems to this one

Similar problems include "Meeting Rooms II" where we need to find the minimum number of conference rooms needed to hold all the meetings, and "Interval List Intersections" where we need to find the intersections of two lists of intervals.

Things to keep in mind and tricks

Always remember to sort the intervals, as it is a necessary step for the greedy algorithm. Also, when intervals are sorted by their end times, if an overlap occurs, it is always optimal to remove the current interval, not the previous one.

Corner and special cases to test when writing the code

Testing should include edge cases such as an empty list of intervals and cases where all intervals are non-overlapping or overlap entirely. It is also essential to test with intervals that share endpoints, e.g., $[1,2]$ and $[2,3]$.

Merge Intervals

The "Merge Intervals" problem is a common algorithmic challenge involving array manipulation and sorting. The task is to merge all overlapping intervals in a given array to produce a list of non-overlapping intervals that cover all the intervals in the input.

Problem Statement

Given an array of intervals where intervals

i

=

$start_i, end_i$

, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example:

Input: intervals = [[1,3],[2,6],[8,10],[15,18]]

Output: [[1,6],[8,10],[15,18]]

Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].

LeetCode link: [Merge Intervals](#)

Algorithmic Approach

The solution to this problem typically involves sorting the intervals by their start times, iterating through the sorted intervals, merging overlapping intervals, and adding non-overlapping intervals directly to the result list.

Python Implementation

Below is the complete Python code for merging overlapping intervals:

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        if not intervals:
            return []

        # Sort the intervals by the start of each interval
        intervals.sort(key=lambda x: x[0])

        merged = []
        for interval in intervals:
            # if the list of merged intervals is empty or if the
            #     ↪ current
            # interval does not overlap with the previous, simply
            #     ↪ append it.
            if not merged or merged[-1][1] < interval[0]:
                merged.append(interval)
            else:
                # otherwise, there is overlap, so we merge the
                #     ↪ current and previous
                # intervals by updating the end of the previous
                #     ↪ interval
                merged[-1][1] = max(merged[-1][1], interval[1])

        return merged
```

This implementation begins by checking if the intervals list is empty. It then sorts the intervals based on the start time to ensure that overlapping intervals are adjacent. As it iterates through the sorted list, it either merges overlapping intervals or appends non-overlapping intervals to the ‘merged’ list.

Why this approach

Sorting the intervals helps in identifying overlaps easily, which is vital because merging is only necessary when intervals overlap. Comparing the end of the last interval in the merged list with the start of the current interval allows us to determine whether an overlap exists.

Alternative approaches

An alternative approach could be using a tree data structure, like an interval tree, to manage overlapping intervals. However, this comes with more complex implementation and does not improve the worst-case time complexity.

Similar problems to this one

Other interval-related problems like "Insert Interval" or "Non-overlapping Intervals" use similar techniques for managing and comparing intervals, often also requiring the intervals to be sorted as a first step.

Things to keep in mind and tricks

Always consider edge cases where intervals might just touch each other, e.g.,

$$1, 2$$

and

$$2, 3$$

. The definition of overlap in the problem description determines how to handle these cases.

Corner and special cases to test when writing the code

It's important to test cases with a single interval, intervals that do not overlap at all, and intervals that overlap completely. Additionally, consider testing intervals with identical start and end times, which should be properly merged into a single interval.

Insert Interval

The "Insert Interval" problem is a common challenge in algorithmic interviews that assesses a candidate's capability to manipulate intervals and understand complex edge cases. In this task, we're presented with a list of non-overlapping, sorted intervals and we're asked to insert a new interval into this list.

Problem Statement

Given a list of non-overlapping intervals sorted by their start times, where each interval is represented as `intervals[i] = [start_i, end_i]`, and a new interval `newInterval = [start, end]`, the objective is to insert `newInterval` into the intervals list. After the insertion, the intervals must remain sorted by start times, and all overlapping intervals must be merged as needed. The merged intervals should represent the union of the intervals after the insertion.

Example 1:

Input: `intervals = [[1, 3], [6, 9]]`, `newInterval = [2, 5]`

Output: `[[1, 5], [6, 9]]`

Explanation: The new interval `[2, 5]` overlaps with `[1, 3]`. Merging these yields the list `[[1, 5], [6, 9]]`.

Example 2:

Input: `intervals = [[1, 2], [3, 5], [6, 7], [8, 10], [12, 16]]`, `newInterval = [4, 8]`

Output: `[[1, 2], [3, 10], [12, 16]]`

Explanation: The new interval `[4, 8]` overlaps with the intervals `[3, 5]` and `[6, 7]`, and touches `[8, 10]`. Merging yields the list `[[1, 2], [3, 10], [12, 16]]`.

LeetCode link: [LeetCode](#)

Algorithmic Approach

One effective way to approach this challenge is to:

1. Iterate through the list while the new interval does not overlap with the current interval, adding the original intervals to the output list.
2. Merge the new interval with overlapping intervals by updating the start and end to encompass the overlapping range.
3. Continue the iteration for further possible overlaps, merging as necessary.
4. Once past all overlaps, add the new interval to the output.
5. Append the rest of the original non-overlapping intervals to the output.

This approach is systematic, ensuring that all intervals, overlapping or not, are eventually part of the merged list of intervals.

Complexities

- **Time Complexity:** The process involves iterating through all the intervals, leading to an $O(n)$ time complexity, where n is the number of intervals.
- **Space Complexity:** Since the result list has to accommodate all intervals, in the worst case all intervals might be non-overlapping, giving a space complexity of $O(n)$.

Python Implementation

Below is the complete Python code for the 'insert' function that merges a new interval into the sorted, non-overlapping list of intervals:

```
def insert(intervals, newInterval):
    output = []
    for i in range(len(intervals)):
        if newInterval[1] < intervals[i][0]:
            output.append(newInterval)
            return output + intervals[i:]
        elif newInterval[0] > intervals[i][1]:
            output.append(intervals[i])
        else:
            newInterval = [min(newInterval[0], intervals[i][0]),
                           max(newInterval[1], intervals[i][1])]
    output.append(newInterval)
    return output
```

This implementation first checks for non-overlap, and once it finds the spot for the new interval, merges it if necessary, and then appends the rest of the intervals.

Why this approach

The chosen approach effectively handles all edge cases by segregating the process into clear steps: adding non-overlapping intervals, merging when necessary, and then finishing off with the rest of the list. This makes the code robust and easy to understand.

Alternative approaches

An alternative method is to construct a min-heap with the intervals for efficient overlapping checks, but given the sorted nature of the input, iterating over the list is more optimal in terms of both time and space complexity.

Similar problems to this one

Other interval problems such as "Merge Intervals" or "Non-overlapping Intervals" also require careful merging or interval selection strategies, often requiring similar thought processes but with varied constraints and objectives.

Things to keep in mind and tricks

A common trick used in interval problems is to start with the key operations (insertion, merging) and then handle special cases. Always run through edge cases like empty intervals or complete non-overlapping to test the robustness of your code.

Corner and special cases to test when writing the code

Pay extra attention to testing cases where intervals are touching boundaries, only partially overlapping, or fully engulfed by the new interval, as well as cases where the new interval is either fully before or after existing intervals.

Find Minimum in Rotated Sorted Array

The "Find Minimum in Rotated Sorted Array" problem entails locating the smallest element within a sorted array that has been rotated between 1 and n times. Given an increasingly sorted array that has undergone this transformation, the objective is to efficiently determine the minimal value considering that the array's sorted nature provides a strategic advantage for a binary search algorithm, as opposed to a linear search which would exceed the desired $O(\log n)$ time complexity.

Problem Statement

Given a rotated sorted array, the task is to identify the minimum element. A sorted array is considered rotated if it is shifted in such a way that some of the elements that were initially at the beginning of the array are moved to the end while preserving the overall order. It is important to note that the algorithm should run in $O(\log n)$ time complexity.

LeetCode link: [Find Minimum in Rotated Sorted Array](#)

Algorithmic Approach

To solve this problem, we will employ a binary search technique. Binary search enables us to divide the search space by half at each step. By comparing the middle element to the end element of the search space, we can determine whether the minimum element lies on the left or right side and continue searching in the correct half.

Complexities

- **Time Complexity:** Our binary search approach has a time complexity of $O(\log n)$ since at each step, we are halving the search space.
- **Space Complexity:** The space complexity is $O(1)$, as no additional space proportional to the input size is used.

Python Implementation

The Python code below illustrates the implementation of the binary search algorithm to find the minimum element in the rotated sorted array:

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > nums[right]:
                left = mid + 1
            else:
                right = mid
        return nums[left]
```

In this code, we maintain two pointers, 'left' and 'right', which indicate the current search space. We calculate the 'mid' point and use it to decide which half contains the minimum element. The loop continues until 'left' is the same as 'right', indicating that we have narrowed down to the minimum element.

Why this approach

The decision to apply binary search in this case is based on the fact that despite the array's rotation, it is still partially sorted. During each iteration, the binary search is able to establish which segment of the array (divided at the midpoint) is sorted and thus eliminate the opposite half from consideration, effectively narrowing the potential locations of the minimum element.

Alternative approaches

An alternate, less efficient method would be a linear search through the entire array to find the minimum element. However, since this would have a time complexity of $O(n)$, it does not meet the problem's requirement for $O(\log n)$ time complexity.

Similar problems to this one

This problem shares similarities with other problems that involve searching in a rotated sorted array, such as "Search in Rotated Sorted Array" where you are tasked with finding the position of a specific target value.

Things to keep in mind and tricks

When implementing binary search algorithms, it's important to be careful with the calculation of the middle index and the update steps for the search boundaries to avoid infinite loops or off-by-one errors. It's also necessary to ensure that the

comparison logic is well thought out to determine the correct half of the search space.

Corner and special cases to test when writing the code

Make sure to test cases where the minimum element is at the beginning or the end of the array, as well as when the array is not rotated at all. These edge cases can help ensure that the binary search conditions are correctly handling all possible scenarios.

Search in Rotated Sorted Array

The "Search in Rotated Sorted Array" problem involves finding the index of a target value within an array that has been both sorted and rotated. A challenge here is to maintain a logarithmic $O(\log n)$ search time, indicating the need for a modified binary search approach, despite the array's rotation.

Problem Statement

Given an array *nums* that is originally sorted in ascending order and then possibly rotated, we need to find and return the index of a target value *target* in *nums*. If the target value is not found within the array, we should return -1.

The problem exemplifies a scenario where a sorted array like $[0, 1, 2, 4, 5, 6, 7]$ is rotated at a pivot unknown beforehand, resulting in an array like $[4, 5, 6, 7, 0, 1, 2]$. Despite the rotation, we need to ensure the search algorithm is efficient with a runtime complexity of $O(\log n)$.

LeetCode Link: [Search in Rotated Sorted Array](#)

Algorithmic Approach

To solve the problem, we can start by examining the array and determining how it might have been rotated. By comparing the middle element to the end of the array, we can decide which half is in proper ascending order and whether the target lies in that half. If not, we then search the other half accordingly. This approach is a subtle variation of the traditional binary search method, which is only performant on sorted arrays.

Complexities

- **Time Complexity:** Since the algorithm still fundamentally follows a binary search pattern, the time complexity is $O(\log n)$, where n is the number of elements in the input array.
- **Space Complexity:** The algorithm operates in constant space, hence space complexity is $O(1)$.

Python Implementation

The following Python code implements the search function for finding the target in a rotated sorted array:

```
class Solution(object):
    def search(self, nums, target):
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid

            # Left sorted portion
            if nums[left] <= nums[mid]:
                if nums[left] <= target <= nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            # Right sorted portion
            else:
                if nums[mid] <= target <= nums[right]:
                    left = mid + 1
                else:
                    right = mid - 1

        return -1
```

By maintaining two pointers, 'left' and 'right', that define the search space, the algorithm effectively halves this space at every step, discarding the half that can be confidently asserted to not contain the target.

Why this approach

This binary search approach was chosen due to its logarithmic time complexity, which aligns with the requirements of achieving $O(\log n)$ runtime. The problem's constraint that the array is originally sorted, even though it might be rotated, allows us to perform a modified binary search and makes this approach both elegant and efficient.

Alternative approaches

An alternative brute force approach would involve a linear search through the array to find the target, but this would not meet the logarithmic time complexity requirement. Another alternative could involve first finding the pivot point where the array is rotated and then performing a regular binary search in the appropriate subarray, but this is more complex and unnecessary.

Similar problems to this one

Similar problems that use binary search with a twist include "Find Minimum in Rotated Sorted Array", "Search in Rotated Sorted Array II" (where elements may be duplicated), and "Find Peak Element".

Things to keep in mind and tricks

When implementing the binary search for this problem, it's crucial to carefully handle the conditionals within the while loop to avoid off-by-one errors. Deciding when to move the 'left' or 'right' pointer requires a clear understanding of the sorted portions of the array after its rotation.

Corner and special cases to test when writing the code

Special cases that are essential to test include arrays with just one element, arrays where the target is the pivot point, and arrays where the target does not exist. Additionally, one should test cases where the target is at the beginning or end of the array.

Meeting Rooms

Meeting Rooms problems are a staple in coding interviews, testing one's ability to analyze interval data and apply sorting or heap-based algorithms. There are two classic variations of this problem: Meeting Rooms I (checking if a person can attend all meetings) and Meeting Rooms II (determining the minimum number of conference rooms needed).

Problem Statement

Meeting Rooms (Variant I)

Given an array of meeting time intervals where each interval consists of a start and end time $[s_i, e_i]$ (with $s_i < e_i$), determine if a person could attend all meetings without any overlaps.

Examples:

- **Input:** $[[0,30],[5,10],[15,20]]$
Output: false
- **Input:** $[[7,10],[2,4]]$
Output: true

Meeting Rooms II (Variant II)

Given an array of meeting time intervals where each interval consists of start and end times $[s_i, e_i]$, find the minimum number of conference rooms required to accommodate all meetings.

Examples:

- **Input:** $[[0,30],[5,10],[15,20]]$
Output: 2
- **Input:** $[[7,10],[2,4]]$
Output: 1

For both variations, a typical solution involves sorting the intervals by their start times to efficiently process the interval overlaps. Variant I may only need a

single pass after sorting, while Variant II often requires a priority queue (min-heap) to actively manage room end times to calculate the minimum rooms required.

Algorithmic Approach

Variant I (Can Attend All Meetings)

The most common approach is to sort the intervals by their start times. After sorting, we iterate over the intervals and check if the current start time is less than the previous end time, indicating an overlap.

Variant II (Minimum Number of Conference Rooms)

For this variant, after sorting by start times, we use a min-heap to keep track of the end times of meetings currently using a room. We iterate over the sorted intervals, and whenever we find that a room is freed up (the current meeting's start time is greater than or equal to the minimum end time in the heap), we reuse that room (by updating the heap). Otherwise, if no room is available, we allocate a new room.

Complexities

- **Time Complexity:**
 - **Variant I:** $O(n \log n)$ due to the sorting of intervals.
 - **Variant II:** $O(n \log n)$ for sorting the intervals, plus $O(n \log n)$ for managing the heap, resulting in $O(n \log n)$ overall.
- **Space Complexity:**
 - **Variant I:** $O(1)$ if we sort the intervals in place.
 - **Variant II:** $O(n)$ for storing the min-heap.

Python Implementation

The Python implementations for Variant I and II of the Meeting Rooms problem are as follows:

```
# Variant I: Can Attend All Meetings
def canAttendMeetings(intervals):
    intervals.sort(key=lambda x: x[0])
    for i in range(1, len(intervals)):
        if intervals[i][0] < intervals[i-1][1]:
            return False
    return True

# Variant II: Minimum Number of Conference Rooms
def minMeetingRooms(intervals):
    if not intervals:
        return 0
    free_rooms = []
    intervals.sort(key=lambda x: x[0])
    heapq.heappush(free_rooms, intervals[0][1])

    for i in intervals[1:]:
        if free_rooms[0] <= i[0]:
            heapq.heappop(free_rooms)
            heapq.heappush(free_rooms, i[1])

    return len(free_rooms)
```

Why this approach

The approaches above leverage the efficient time complexity of sorting algorithms and the convenient properties of min-heaps for managing simultaneous events. Sorting helps in dealing with the chronology of events, which is essential for both problems. The heap is an excellent structure for dynamically keeping track of the meeting rooms' end times, which is crucial for determining room availability in real-time.

Alternative approaches

For Variant I, an alternative approach could involve a sweep line algorithm with events for starting and ending of meetings. For Variant II, one could potentially use a chronologically ordered list or another data structure that allows efficient insertion and extraction of the smallest element, but a heap is typically the most suitable due to its $O(\log n)$ operations.

Similar problems to this one

Problems involving intervals or scheduling, such as "Car Pooling", "Merge Intervals", and "Insert Interval", share similarities with the Meeting Rooms problems, as they also require thoughtful handling of overlapping intervals or managing start and end events.

Things to keep in mind and tricks

When working with intervals, it's vital to consider ordering by start times and being aware of the endpoints' handling. A common mistake is not considering edge cases where intervals end and start at the same time. For heap operations, make sure to understand how Python's `heapq` module functions, particularly the fact that it only provides a min-heap implementation.

Corner and special cases to test when writing the code

Some special cases to consider are intervals that are contained within each other, intervals that have the same start or end times, and also when there are no intervals or only one interval. Thorough testing should include these edge cases to ensure robustness.

Meeting Rooms II

The "Meeting Rooms II" problem is typically asked in technical interviews and coding challenge platforms like LeetCode. The problem focuses on finding the minimum number of conference rooms required to accommodate all meetings based on their start and end times.

Problem Statement

Given an array of meeting time intervals consisting of start and end times $[[s1, e1], [s2, e2], \dots]$ ($s_i < e_i$), determine the minimum number of conference rooms required to accommodate all meetings.

Example:

Input: `[[0, 30], [5, 10], [15, 20]]`

Output: 2

Explanation:

- Meeting one can be held in a conference room from time 0 to 30.
- Meeting two can be held in another room from time 5 to 10 (as it
 ↪ conflicts with meeting one).
- Meeting three can be held in the conference room used for meeting
 ↪ two after it ends at time 15 to 20.

Therefore, two conference rooms are the minimum number of rooms
 ↪ required to hold all the meetings without overlapping.

LeetCode Link: [Meeting Rooms II](#)

Algorithmic Approach

The solution to this problem can be approached by using a min-heap to track the end times of meetings currently occupying rooms. By sorting the meetings by their start times and iterating through them, we can assign a room if one is free (i.e., if the current meeting's start time is later than the end time of the earliest meeting) or allocate a new room if no rooms are free.

Complexities

- **Time Complexity:** The total time complexity is $O(N \log N)$ mainly due to the sort operation, where N is the number of meetings.
- **Space Complexity:** The space complexity is $O(N)$, as we may need to keep track of end times for all meetings in the worst-case scenario.

Python Implementation

Below is the complete Python code for solving the "Meeting Rooms II" problem:

```
import heapq

class Solution(object):
    def minMeetingRooms(self, intervals):
        if not intervals:
            return 0

        # Initialize a heap.
        free_rooms = []

        # Sort the meetings in increasing order of their start time
        ↪ .
        intervals.sort(key=lambda x: x[0])

        # Add the first meeting. We have to give a new room to the
        ↪ first meeting.
        heapq.heappush(free_rooms, intervals[0][1])

        # For all the remaining meeting rooms
        for i in intervals[1:]:

            # If the room due to free up the earliest is free,
            ↪ assign that room to this meeting.
            if free_rooms[0] <= i[0]:
                heapq.heappop(free_rooms)

            # If a new room is to be assigned, then also we add to
            ↪ the heap.
            # If an old room is allocated, then also we have to add
            ↪ to the heap with updated end time.
            heapq.heappush(free_rooms, i[1])

        # The size of the heap tells us the minimum rooms required
        ↪ for all the meetings.
        return len(free_rooms)
```

This implementation starts by checking if there are any meetings at all. It then sorts the meetings by their start times and initializes a min-heap to keep track of the end times of meetings that are currently happening. By checking against the smallest end time in the heap, we either free up a room or allocate a new room for each incoming meeting.

Why this approach

The heap-based approach is efficient because it keeps the end times in a sorted manner, allowing us to quickly check for room availability. Since the insertion and

removal operations in a min-heap take $O(\log N)$ time where N is the number of meetings, it ensures that we can manage the rooms with minimal overhead on top of the sorting step.

Alternative approaches

An alternative approach is using chronological ordering where we separate the start and end times into separate arrays and sort them. We then iterate through them to determine room allocation, incrementing a room count when starting a new meeting and decrementing when a meeting ends.

Similar problems to this one

Similar problems in the domain of interval scheduling and room allocation include car pooling, job scheduling problems, and determining if a person could attend all meetings (the non-overlapping variant of this problem).

Things to keep in mind and tricks

The tricky part of this problem is handling the condition where multiple meetings end and start at the same time. Remember to free up rooms before checking for room availability for new meetings. Also, comparing only end times is sufficient, as the start times are already sorted.

Corner and special cases to test when writing the code

Test cases should include situations where all meetings are non-overlapping, all meetings overlap, and where some meetings start exactly when others end. Edge cases with empty meeting schedules or a single meeting should also be considered.

Rotate Image

The "Rotate Image" problem is a classic example of a matrix manipulation challenge, commonly faced during technical interviews. The goal is to rotate a square matrix by 90 degrees clockwise, in place.

Problem Statement

Given an $n \times n$ two-dimensional matrix representing an image, the task is to rotate the image by 90 degrees (clockwise). The rotation must be performed in place, which means that the original matrix must be modified directly without using an additional matrix.

LeetCode link: [Rotate Image](#)

Algorithmic Approach

To solve this problem, we can break down the rotation into two steps:

1. **Transpose the matrix:** Swap the element at `matrix[i][j]` with the element at `matrix[j][i]`, for all i and j such that $i < j$.
2. **Flip the matrix horizontally:** Swap the element at `matrix[i][j]` with the element at `matrix[i][n-1-j]` for all i and j , to achieve the clockwise rotation effect.

Complexities

- **Time Complexity:** Since the matrix is traversed twice, once for transposing and once for flipping horizontally, the overall time complexity is $O(n^2)$, where n is the number of rows or columns of the matrix.
- **Space Complexity:** As the rotation is performed in place, the space complexity is $O(1)$, since no additional storage proportional to the input size is used.

Python Implementation

Below is the complete Python code for rotating an $n \times n$ matrix by 90 degrees clockwise in place:

```
def rotate(matrix):
    n = len(matrix)
    # Transpose the matrix
    for i in range(n):
        for j in range(i, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

    # Flip the matrix horizontally
    for i in range(n):
        for j in range(n // 2):
            matrix[i][j], matrix[i][n - 1 - j] = matrix[i][n - 1 - j]
            ↪ j], matrix[i][j]

# Example usage:
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

rotate(matrix)
print(matrix) # Output: [[7, 4, 1], [8, 5, 2], [9, 6, 3]]
```

This implementation first transposes the matrix by swapping elements across the diagonal, and then it flips the matrix horizontally to emulate the 90 degrees clockwise rotation.

Why this approach

This approach is chosen because of its simplicity and efficiency. By working in place, it minimizes space usage, and the operations of transposition and horizontal flipping are straightforward transformations that achieve the desired rotation.

Alternative approaches

Alternative approaches could involve using extra space, such as creating a new matrix to hold the rotated values. However, these approaches would not satisfy the in-place requirement.

Similar problems to this one

Similar problems include rotating a matrix by different angles, performing matrix transpositions, reflections, or other in-place operations, as well as handling non-square matrices.

Things to keep in mind and tricks

- When addressing matrix problems, it's important to clearly visualize the index transformations occurring pre and post-rotation. - Understand the difference between rotating clockwise and counterclockwise, as the latter may require a horizontal flip before transposing.

Corner and special cases to test when writing the code

Consider test cases where the matrix is: - Very small (1x1 or 2x2), as base cases might behave differently. - Contains negative numbers or zeroes, to ensure the operations handle all value ranges. - Very large, to ensure the algorithm performs well under stress and handles edge cases in indices properly.

Rotate Image: Spin That Matrix!

Problem Statement

The challenge here is to give an $n \times n$ matrix a 90-degree spin in a clockwise direction, all without the luxury of extra space for another matrix. In other words, we need to perform this matrix magic in place.

LeetCode link: [Rotate Image](#)

Algorithmic Approach

The rotation can be done in layers, starting from the outermost layer and moving towards the inner layers. Rotating a matrix by 90 degrees clockwise can be done by a two-step process:

1. Transposing the matrix.
2. Reversing each row of the transposed matrix.

Complexities

- Time Complexity: $O(n^2)$, where n is the number of rows (or columns) of the matrix. We process each element once.
- Space Complexity: $O(1)$, as we perform the rotation in place without using any extra space.

Python Implementation

```

def rotate(matrix):
    n = len(matrix)

    # Transpose the matrix
    for i in range(n):
        for j in range(i, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

    # Reverse each row
    for i in range(n):
        matrix[i].reverse()

# Example usage:
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
rotate(matrix)
print(matrix) # Output: [[7, 4, 1], [8, 5, 2], [9, 6, 3]]

```

Explanation

The given code first transposes the matrix, which is essentially flipping the matrix over its diagonal. In this state, the columns become rows but in incorrect order (i.e., the first column becomes the last row). The second step is to reverse each row to get the columns in the correct order for the 90-degree rotation.

Why this approach

This approach is chosen because it manipulates the matrix in place, satisfying the constraint of not using additional memory. It's an efficient and straightforward method to perform the rotation.

Alternative approaches

An alternative approach is to rotate each layer of the matrix individually. This involves moving the elements four at a time, corresponding to the four sides of a layer. However, this approach is more complex and error-prone.

Similar problems to this one

Similar matrix problems include "Transpose Matrix," "Spiral Matrix," and "Diagonal Traverse." Each of these problems requires understanding of matrix traversal and transformation techniques.

Things to keep in mind and tricks

When rotating a matrix, remember you can perform rotation using a combination of transposing and reversing operations. Halving the operational limits in loops to not over-process elements is crucial. Handling odd and even dimensions needs attention to detail, as the center element in an odd-dimension matrix does not move.

Corner and special cases to test when writing the code

Test with both even and odd n , ensuring the algorithm handles the central element correctly for odd n . Edge cases should include the smallest matrix possible (a 2×2 matrix) as well as larger matrices to gauge performance. Also, testing matrices with duplicate values can help ensure that the algorithm correctly handles identical elements.

Container With Most Water

The "Container With Most Water" problem, often referred to as "Max Area," is a classic problem that challenges us to find the two lines that together with the x-axis forms a container that could hold the maximum amount of water.

Problem Statement

You are given an array `height` which represents the height of n vertical lines drawn on the x-axis at positions $[0, 1, 2, \dots, n - 1]$. Each line is drawn between the x-axis and the point $(x, \text{height}[x])$.

The goal is to select two lines that form a container along with the x-axis, such that it contains the most water. The container cannot be slanted, and n must be at least 2.

Input: An array of non-negative integers `height`.

Output: The maximum volume of water a container can store.

Example:

Input: `height = [1, 8, 6, 2, 5, 4, 8, 3, 7]`

Output: 49

Explanation: The max area of water (blue section) the container can contain is 49.

Input: `height = [1, 1]`

Output: 1

[LeetCode Link](#)

Algorithmic Approach

We can solve this problem with a two-pointer approach. The two pointers are initialized at both ends of the array, and we move the pointer pointing to the shorter line towards the other pointer. This is because if we moved the pointer at the taller line, the height of the container would remain the same or decrease, and the width would also decrease, necessarily resulting in a smaller area.

Complexities

- **Time Complexity:** The two-pointer approach effectively scans the array once, giving a time complexity of $O(n)$.
- **Space Complexity:** Since the approach uses only a constant amount of extra space, the space complexity is $O(1)$.

Python Implementation

Below is the complete Python code that employs the two-pointer technique to solve the problem:

```
class Solution:
    def maxArea(self, height):
        max_water = 0
        left, right = 0, len(height) - 1

        while left < right:
            width = right - left
            max_water = max(max_water, min(height[left], height[
                ↪ right]) * width)

            if height[left] < height[right]:
                left += 1
            else:
                right -= 1

        return max_water
```

This implementation finds the maximum area by iterating through the array only once. The pointers move towards each other, and after each step, the maximum area is updated if a larger area is found.

Why this approach

The two-pointer approach was chosen for its efficiency, both in terms of time and space complexity. By constraining the container's vertical boundaries to the heights in the array, and then moving inward, we ensure that we only consider potentially larger areas without having to check each possible pair of lines.

Alternative approaches

A brute force solution could be used, where one would check every possible pair of lines. However, this approach would have a time complexity of $O(n^2)$, making it inefficient for larger input sizes.

Similar problems to this one

There are other problems involving arrays and two-pointer techniques such as "Two Sum", "3Sum", "Trapping Rain Water", and others that deal with similar ideas of using pointers to navigate and compute values based on the array elements.

Things to keep in mind and tricks

- Remember that moving the pointer at the taller line inward will never yield a larger area. - The width between the lines decreases with every move, so only consider moves that might increase the height to possibly increase the area.

Corner and special cases to test when writing the code

- Arrays with all elements being the same value (e.g., [1, 1, 1, 1]). - Arrays with only two elements. - Arrays with large differences in height values. - Cases where the maximum area is formed by lines that are not the furthest apart.

Product of Array Except Self

The "Product of Array Except Self" problem is a classic problem in the realm of algorithm and data structure problems often encountered on platforms like LeetCode.

Problem Statement

Given an array `nums` of n integers where $n > 1$, return an array `output` such that `output[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

Given the constraints, it's guaranteed that the product of the array elements is within the range of a 32-bit signed integer.

LeetCode link: [Product of Array Except Self](#)

Examples

Example 1:

Input: `[1, 2, 3, 4]`

Output: `[24, 12, 8, 6]`

Explanation: By calculating the product of all elements except for the one at the current index, you would get:

- For index 0: $(2 \times 3 \times 4) = 24$
- For index 1: $(1 \times 3 \times 4) = 12$
- For index 2: $(1 \times 2 \times 4) = 8$
- For index 3: $(1 \times 2 \times 3) = 6$

Note:

- You cannot use division in solving this problem.
- You should try to solve the problem in $O(n)$ time complexity.

Algorithmic Approach

To solve this problem, we utilize two arrays to store the prefix and suffix products of every element. We then multiply these prefix and suffix products to obtain the final output.

Complexities

- **Time Complexity:** The total time complexity of the solution is $O(n)$ as we are iterating through the array a constant number of times.
- **Space Complexity:** The space complexity is $O(n)$ due to the extra space taken up by the prefix and suffix arrays.

Python Implementation

Below is the complete Python code for solving the "Product of Array Except Self" problem without using division and with linear time complexity:

```
class Solution:
    def productExceptSelf(self, nums):
        length = len(nums)

        # Initialize arrays for left and right products
        left_products = [0]*length
        right_products = [0]*length
        output = [0]*length

        # left_products[i] contains the product of all elements to
        #   ↪ the left of i
        left_products[0] = 1
        for i in range(1, length):
            left_products[i] = nums[i - 1] * left_products[i - 1]

        # right_products[i] contains the product of all elements to
        #   ↪ the right of i
        right_products[length - 1] = 1
        for i in reversed(range(length - 1)):
            right_products[i] = nums[i + 1] * right_products[i + 1]

        # Construct the output array
        for i in range(length):
            output[i] = left_products[i] * right_products[i]

        return output
```

By precalculating the product of elements to the left and right of every element, we can construct the output array without the use of division.

Why this approach

This approach is chosen because it adheres to the constraints of not using division and maintains a linear time complexity. By keeping track of the products to the left and right separately, we can multiply these values together to get the desired result for each index.

Alternative approaches

An alternative approach could have been directly calculating the total product and then dividing it by the current element, but the problem explicitly forbids the use of division. Additionally, using extra passes and division might have exceeded the desired linear time complexity.

Similar problems to this one

Similar problems may include other array transformation challenges that involve prefix sums or products, or problems that require the efficient computation of cumulative properties without using direct division, such as "Maximum Product Subarray" or "Trapping Rain Water."

Things to keep in mind and tricks

- Precomputation of cumulative quantities can lead to efficient algorithms. -
- When division is not allowed, consider the use of prefix and suffix arrays to maintain state. - Always consider the constraints and desired time complexity when designing a solution. - Remember to check for edge cases, such as an array containing zeros or negative numbers.

Corner and special cases to test when writing the code

While testing, consider arrays with:

- A single zero, multiple zeros, and no zeros.
- Negative numbers, as they can affect the product sign. - Large lengths to ensure that the time complexity is indeed linear. - Edge cases, such as an empty array or an array with a single element.

Maximum Subarray

The Maximum Subarray problem is a classic in computer science and is often used to introduce dynamic programming and divide-and-conquer techniques.

Problem Statement

Given an integer array `nums`, find the contiguous subarray (containing at least one number) which has the largest sum and return its sum.

Example:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: `[4,-1,2,1]` has the largest sum = 6.

Additional examples may be provided to clarify:

- A **subarray** is a contiguous part of an array.
- An array of one element is also considered a subarray.

LeetCode link: [Maximum Subarray](#)

Algorithmic Approach

Various methods can be utilized to tackle the problem:

1. **Brute Force:** Examine every subarray, calculating the sum to maintain the track of the maximum sum. This naïve approach has a time complexity of $O(n^2)$ to $O(n^3)$.
2. **Divide and Conquer:** This technique involves splitting the array into two, recursively find the maximum subarray sum in each half, and also the maximum subarray sum that crosses the midpoint. This has a time complexity of $O(n \log n)$.
3. **Dynamic Programming (Kadane's Algorithm):** Iterate through the array, compute the maximum subarray sum at each position by taking the greater of either the current element or the sum of the current element and the previous maximum subarray sum. This has a time complexity of $O(n)$ and is the most efficient approach.

Python Implementation

The following is the complete Python code that implements Kadane's Algorithm for this problem:

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        max_current = max_global = nums[0]

        for x in nums[1:]:
            max_current = max(x, max_current + x)

            if max_current > max_global:
                max_global = max_current

        return max_global

```

This implementation initiates two variables, `max_current` and `max_global`, to the first element of the array. For each succeeding element, it calculates `max_current` and updates `max_global` if a higher sum is found.

Why this approach

Kadane's algorithm is chosen for its efficient time complexity and straightforward implementation. It leverages the insight that the maximum subarray up to a certain point can be either the current element itself (which would start a new subarray) or the current element added to the maximum subarray ending just before that point.

Alternative approaches

The divide-and-conquer method could be used as an alternative, providing valuable insights into transforming and conquering strategy but is less efficient than Kadane's algorithm.

Similar problems to this one

Other array computation problems like "Maximum Product Subarray" or "Best Time to Buy and Sell Stock" utilize similar dynamic programming techniques and are related in terms of finding optimal contiguous segments within an array.

Things to keep in mind and tricks

Make sure to handle the case where all the elements in the array could be negative. In such a case, the maximum sum would be the largest negative number. Avoid unnecessary recalculation of sums which is a common inefficiency in brute-force solutions.

Corner and special cases to test when writing the code

Pay attention to testing edge cases such as an empty array, a single-element array, an array of all negative numbers, and an array with both large positive and negative numbers. These cases ensure the robustness and accuracy of the implementation.

Number of Islands

The "Number of Islands" problem is a classical algorithmic challenge that requires counting distinct clusters of connected components on a two-dimensional grid. This problem is a good exercise for understanding how to perform depth-first search (DFS) or breadth-first search (BFS) on a grid.

Problem Statement

Given a 2D grid map of '1's (land) and '0's (water), we are tasked with counting the number of islands. An island is defined as a group of adjacent lands connected horizontally or vertically. It is assumed that the four edges of the grid are surrounded by water.

Examples

Example 1:

Input:

```
11110
11010
11000
00000
```

Output: 1

Example 2:

Input:

```
11000
11000
00100
00011
```

Output: 3

LeetCode link: [Number of Islands](#)

Algorithmic Approach

To count the number of islands, we can iterate over each cell in the grid. When we encounter a '1', we trigger a DFS or BFS to mark all adjacent land cells (also '1's).

This process should recursively continue until it encounters water ('o') or the edge of the grid. Each distinct DFS or BFS traversal corresponds to one island, and thus we increment our count of islands accordingly.

Complexities

- **Time Complexity:** The overall time complexity is $O(M \times N)$, where M and N are the number of rows and columns in the grid, respectively. Each cell is visited once during the traversal.
- **Space Complexity:** The space complexity is $O(M \times N)$ in the worst-case scenario for the DFS recursion stack or the BFS queue when the grid is filled with land.

Python Implementation

Below is the complete Python code that uses depth-first search for the `number_of_islands` function:

```
def number_of_islands(grid):
    if not grid:
        return 0

    def dfs(grid, i, j):
        if i < 0 or i >= len(grid) or j < 0 or j >= len(grid[0]) or
            ↪ grid[i][j] == '0':
            return
        grid[i][j] = '0' # Mark as visited
        dfs(grid, i - 1, j) # Up
        dfs(grid, i + 1, j) # Down
        dfs(grid, i, j - 1) # Left
        dfs(grid, i, j + 1) # Right

    count = 0
    for i in range(len(grid)):
        for j in range(len(grid[0])):
            if grid[i][j] == '1':
                count += 1
                dfs(grid, i, j)

    return count

# Example usage:
grid = [
    ['1', '1', '0', '0', '0'],
    ['1', '1', '0', '0', '0'],
    ['0', '0', '1', '0', '0'],
    ['0', '0', '0', '1', '1']
]
print(number_of_islands(grid)) # Output: 3
```

```
class Solution(object):
    def numIslands(self, grid):
        """
        :type grid: List[List[str]]
        :rtype: int
        """
        def dfs(i, j):
            if i < 0 or j < 0 or i >= len(grid) or j >= len(grid)
                ↪ [0] or grid[i][j] == '0':
                return
            grid[i][j] = '0' # mark as visited
            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
```



```

        dfs(i, j - 1)

    if not grid:
        return 0

    count = 0
    for i in range(len(grid)):
        for j in range(len(grid[0])):
            if grid[i][j] == '1':
                dfs(i, j)
                count += 1

    return count

```

This Python function performs DFS to explore all connecting lands for each unvisited island. Marking land as visited by replacing '1's with '0's prevents counting the same land twice.

Why this approach

The DFS approach was chosen because it is a straightforward way to explore and mark connected components in a grid. It efficiently traverses nodes and their neighbors, using recursion to handle the navigation through adjacent lands, which makes the implementation concise and elegant.

Alternative approaches

An alternative approach to solving this problem could be using BFS. Instead of using recursion, a queue is used to store and visit the cells in level order. This approach might be easier to understand for people less comfortable with recursion and has the same time and space complexity as DFS.

Similar problems to this one

Similar problems to "Number of Islands" include "Max Area of Island," "Surrounded Regions," and "Walls and Gates." These problems also involve traversing a grid to identify and process connected regions of certain values.

Things to keep in mind and tricks

It is crucial to remember to mark visited cells to prevent infinite loops. Additionally, always check for boundary conditions when writing grid-based traversal algorithms to avoid index errors.

Corner and special cases to test when writing the code

When writing the code, edge cases to test include empty grids, grids with no lands ('r's), grids with all lands, and grids with one cell. Being aware of these cases helps ensure robust and accurate code.

Word Break

The "Word Break" problem is a classic example of a dynamic programming challenge and is often used to test one's understanding of this concept. The goal is to figure out if a given string can be segmented into a sequence of one or more words that are present in a given dictionary of words. This is not only a fundamental problem in computer science for understanding string manipulation and dynamic programming but also has practical applications in areas such as natural language processing.

Problem Statement

Given a non-empty string 's' and a dictionary 'wordDict' containing a list of non-empty words, determine if 's' can be segmented into a space-separated sequence of one or more dictionary words.

- The same word in the dictionary may be reused multiple times in the segmentation.
- You are to return true if the string can be segmented, and false otherwise.

LeetCode link: [Word Break](#)

Algorithmic Approach

The Word Break problem can be solved efficiently using dynamic programming. The key idea is to keep track of whether each substring of the string 's' can be segmented into words from the 'wordDict'. We use an array 'dp' where 'dp[i]' indicates whether the substring 's[0...i]' can be segmented. The dynamic programming formula involves checking for a 'j' where 'dp[j]' is true and 's[j...i]' is a word in the dictionary, then setting 'dp[i]' to true.

Complexities

- **Time Complexity:** The time complexity of this approach is $O(n^2)$, where 'n' is the length of the string 's'. This is because we are iterating over the substring lengths and checking each substring against the dictionary.

- **Space Complexity:** The space complexity is $O(n)$ due to the extra space used to store the dynamic programming array 'dp'.

Python Implementation

Below is the complete Python code for solving the Word Break problem using dynamic programming:

```
def wordBreak(s, wordDict):
    word_set = set(wordDict)
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break
    return dp[len(s)]

# Examples
s1 = "leetcode"
wordDict1 = ["leet", "code"]
print(wordBreak(s1, wordDict1)) # Output: true

s2 = "applepenapple"
wordDict2 = ["apple", "pen"]
print(wordBreak(s2, wordDict2)) # Output: true

s3 = "catsandog"
wordDict3 = ["cats", "dog", "sand", "and", "cat"]
print(wordBreak(s3, wordDict3)) # Output: false
```

The ‘wordBreak’ function checks whether the string ‘s’ can be segmented into words found in the ‘wordDict’. We use a dynamic programming approach with the array ‘dp’, where ‘dp[i]’ represents whether ‘s[:i]’ can form a word sequence.

Why this approach

The dynamic programming approach for this problem is chosen because of its efficiency in solving problems that involve making decisions at each step based on previous results. This problem exhibits optimal substructure, where the solution can build upon the solutions to subproblems. Thus, dynamic programming is a natural fit that also avoids redundant computations seen in other methods such as backtracking or naive recursion.

Alternative approaches

Alternative approaches to solve this problem include using recursion with memoization, which is top-down dynamic programming, or Breadth-First-Search where each node of the BFS tree represents a prefix of the string and we explore the next possible words that can be appended.

Similar problems to this one

Similar problems that also involve dynamic programming and string manipulation include:

- LeetCode Problem 139: "Word Break II", which asks for all the possible sentence segmentations.
- LeetCode Problem 91: "Decode Ways", which involves determining the number of ways to decode a message.

Things to keep in mind and tricks

When implementing the dynamic programming solution for the Word Break problem, it's important to efficiently check if a substring is in the word dictionary. Using a set data structure for the dictionary can reduce the substring lookup time to constant time. Additionally, initializing `dp[0]` to true allows handling the base case where no characters have been processed.

Corner and special cases to test when writing the code

Some edge cases to consider when writing the code for this problem include:

- The string contains repeated segments (like "aaaa...") and the dictionary contains a word that could match those segments ("a"). This could result in higher time complexity if not handled appropriately.
- The string being empty or consisting of only one character.
- The dictionary being empty or containing one very long word.

Word Search

The "Word Search" problem involves determining if a given word can be found in a grid of letters. The word must be constructed by connecting letters sequentially in adjacent cells, with the constraint that any cell can only be used once in the construction of the word. Adjacent cells are directly above, below, to the left, or to the right of a cell, but not diagonally connected.

Problem Statement

Given an $m \times n$ grid of characters *board* and a string *word*, return true if *word* exists in the grid. The *word* can be constructed from letters of sequentially adjacent cells, with the condition that the same letter cell may not be used more than once.

Example 1:

```
Input: board = [ ["A","B","C","E"],
                  ["S","F","C","S"],
                  ["A","D","E","E"] ],
        word = "ABCCED"
```

Output: true

Example 2:

```
Input: board = [ ["A","B","C","E"],
                  ["S","F","C","S"],
                  ["A","D","E","E"] ],
        word = "SEE"
```

Output: true

Example 3:

```
Input: board = [ ["A","B","C","E"],
                  ["S","F","C","S"],
                  ["A","D","E","E"] ],
        word = "ABCB"
```

Output: false

LeetCode link: [Word Search](#)

Algorithmic Approach

To solve this problem, a typical approach is to use Depth-First Search (DFS) combined with backtracking. The algorithm starts by iterating over each cell in the grid. If the cell contains the first letter of the word, it recursively explores all adjacent cells that could contribute to the word. Through backtracking, the algorithm marks the current cell as visited and proceeds to the next letter in the word. If the path does not lead to the word, the algorithm backtracks by unmarking the visited cell and tries a different path.

Python Implementation

Below is the complete Python code for solving the "Word Search" problem:

```
class Solution:
    def exist(self, board, word):
        def dfs(board, word, i, j, k):
            if not (0 <= i < len(board)) or not (0 <= j < len(board)
                ↳ [0])) or board[i][j] != word[k]:
                return False
            if k == len(word) - 1:
                return True
            board[i][j] = ''
            res = (dfs(board, word, i+1, j, k+1) or
                dfs(board, word, i-1, j, k+1) or
                dfs(board, word, i, j+1, k+1) or
                dfs(board, word, i, j-1, k+1))
            board[i][j] = word[k]
            return res

        for i in range(len(board)):
            for j in range(len(board[0])):
                if dfs(board, word, i, j, 0):
                    return True
        return False
```

This implementation uses a nested function 'dfs' that checks whether the current path in the grid leads to the given word. It searches through all possible directions from the current cell and returns true if the word is found. The 'exist' function iterates over the entire board, invoking 'dfs' when it finds the starting letter of the word.

Why this approach

Depth-First Search is a suitable algorithm for this problem because it allows exploration of all possible paths in the grid that may form the word. Backtracking ensures that once a path is found to be invalid, it reverses changes and explores alternative paths without duplicating the search in previously visited cells.

Alternative approaches

An alternative solution could implement Breadth-First Search (BFS), but BFS is less efficient in this case as it does not take full advantage of the problem's constraints and may require additional space to store all possible paths at each level.

Similar problems to this one

Similar problems that involve grid traversal and backtracking include "N-Queens", "Sudoku Solver", and "Word Search II", which may require modifications to the search

Unique Paths

The "Unique Paths" problem involves finding the total number of unique paths from the top-left corner to the bottom-right corner of a grid, with the restriction that the robot can only move either down or right.

Problem Statement

A robot is located at the top-left corner of a $m \times n$ grid (marked 'Start' in the diagram below). The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid (marked 'Finish' in the diagram below). How many possible unique paths are there? The task is to calculate the total number of distinct paths from the start to the finish.

LeetCode link: [Unique Paths](#)

Algorithmic Approach

To solve this problem, we can use a dynamic programming approach where we build up the number of paths to each cell from the top-left to the bottom-right. For each cell, the number of paths to it is the sum of the number of paths to the cell directly above it and the number of paths to the cell directly to the left of it. The cells in the top row and the leftmost column have only one path leading to them, so they are initialized to '1'.

Complexities

- **Time Complexity:** The total time complexity is $O(mn)$, where m is the number of rows and n is the number of columns in the grid, because we need to compute the number of paths for each cell.
- **Space Complexity:** The space complexity is also $O(mn)$, as we need to store the number of paths for all cells in a 2D array. The space can be optimized to $O(n)$ if we only keep track of the current and previous rows or columns at a time.

Python Implementation

Below is the complete Python code for the function ‘uniquePaths’, which calculates the total number of unique paths from the top-left corner to the bottom-right corner of a $m \times n$ grid:

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        dp = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i-1][j] + dp[i][j-1]
        return dp[-1][-1]
```

This implementation initializes a 2D list ‘dp’ with ‘1’s, then starting from the second row and second column, it accumulates the total paths by adding the number from the top and left cells. The result is given by the value in the last cell ‘dp[-1][-1]’.

Why this approach

The dynamic programming approach is chosen for this problem because it capitalizes on the optimal substructure and overlapping subproblems inherent to the question. Instead of recomputing the number of paths to reach the same cell multiple times through different routes, we consolidate the result in a DP table which significantly cuts down on redundant computations.

Alternative approaches

An alternative to dynamic programming is using combinatorics. Since the robot can only move right or down, for some path from the start to the finish, there are exactly $m - 1$ right moves and $n - 1$ down moves that the robot will make. Therefore, we can frame the problem as selecting $m - 1$ moves out of $m + n - 2$ total moves, which becomes a combination calculation $C(m + n - 2, m - 1)$.

Similar problems to this one

Similar problems include "Unique Paths II" where obstacles are placed on the grid, and "Minimum Path Sum" where, instead of finding the number of unique paths, the goal is to find the path with the minimum sum of numbers along its length.

Things to keep in mind and tricks

One of the tricks to optimize space complexity is to realize that the value of a cell depends only on the cells directly above and to its left. Thus, we can reduce the 2D DP array to a 1D array that is updated on the fly.

Corner and special cases to test when writing the code

Corner cases include grids with just one row or one column, where the number of unique paths is evidently '1'. Special cases might also include large grid values for m and n , to ensure that the function runs efficiently even for maximum input sizes.

Climbing Stairs

The "Climbing Stairs" problem is a great example to introduce the concept of dynamic programming due to its optimal substructure property where the total number of ways to reach a particular step is dependent only on the number of ways to reach the previous two steps.

Problem Statement

Given a non-negative integer n , the task is to find the number of distinct ways you can climb to the top of a staircase with n steps, where you can take either one step or two steps at a time.

LeetCode link: [Climbing Stairs](#)

Algorithmic Approach

The problem can be approached with dynamic programming by breaking it down into smaller subproblems. The total number of ways to reach the i -th step is the sum of ways to reach the $(i - 1)$ -th step and the $(i - 2)$ -th step.

Complexities

- **Time Complexity:** The iterative dynamic programming solution has a time complexity of $O(n)$ as we calculate the number of ways for each step once, up to the n -th step.
- **Space Complexity:** The space complexity is $O(n)$ if we store the number of ways for all steps up to n . This can be optimized to $O(1)$ by only keeping track of the last two steps at any time.

Python Implementation

Below is the complete Python code for the 'climbStairs' function that calculates the number of distinct ways to climb to the top of a staircase with n steps:

```
class Solution:
    def climbStairs(self, n: int) -> int:
        if n <= 2:
            return n
        one_step_before, two_steps_before = 2, 1
        all_ways = 0
        for step in range(3, n + 1):
            all_ways = one_step_before + two_steps_before
            two_steps_before = one_step_before
            one_step_before = all_ways
        return all_ways

# Example Usage
# Create an instance of the Solution class
solution = Solution()

# Input: n = 2
print(solution.climbStairs(2)) # Output: 2

# Input: n = 3
print(solution.climbStairs(3)) # Output: 3
```

This implementation uses an iterative approach to compute the number of ways in a bottom-up fashion. The space optimization is achieved by not using an array to store intermediate results for each step.

Why this approach

The dynamic programming approach is chosen because it provides an efficient way to solve this problem, avoiding redundant calculations. It systematically builds up the solution by solving each subproblem only once.

Alternative approaches

An alternative approach could use recursion with memoization, but this might be less efficient due to a larger stack depth and potentially higher memory usage.

Similar problems to this one

Other dynamic programming problems with optimal substructure and overlapping subproblems, such as the Fibonacci sequence, 'House Robber', and 'Minimum Path Sum' problems, share similarities with the Climbing Stairs problem.

Things to keep in mind and tricks

One trick is to realize that this problem is essentially a Fibonacci sequence where the number of ways to climb n steps is the n -th Fibonacci number. The iterative approach with two variables is akin to maintaining the last two numbers of the Fibonacci sequence to generate the next number.

Corner and special cases to test when writing the code

Some corner cases to consider while writing the code could be when $n = 1$, where there's only one way to reach the top, and when n is large, nearing the upper bound of the constraints. These cases help ensure that the implementation handles small and large inputs efficiently.

Min Stack

The Min Stack problem is about designing a data structure that supports the following operations with $O(1)$ complexity:

- **push(val):** Pushes an element val onto the stack.
- **pop():** Removes the element on the top of the stack.
- **top():** Gets the top element of the stack.
- **getMin():** Retrieves the minimum element in the stack.

Problem Statement

LeetCode link: [Min Stack](#)

Algorithmic Approach

The key to solve this problem is to keep track of the minimum element for each state of the stack. We do this by maintaining a secondary stack that holds the minimums. Each time an element is pushed, we compare it with the current minimum (which is the top element in the secondary stack) and push the smaller one onto the secondary stack. When an element is popped, we simply pop from both stacks, so the minimum stack correctly represents the minimum of the remaining elements.

Complexities

- Time Complexity: $O(1)$ for push, pop, top, and getMin operations.
- Space Complexity: $O(n)$ where n is the number of elements in the stack. This is due to maintaining a secondary stack for tracking the minimums.

Python Implementation

```

class MinStack:

    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self) -> None:
        val = self.stack.pop()
        if val == self.min_stack[-1]:
            self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]

```

Explanation

The MinStack class is designed to keep a secondary stack (`min_stack`) to track the minimums. When a new element is pushed, it's compared with the current minimum and, if it is smaller or equal, it's also pushed onto the `min_stack`. Pop operations are mirrored on both stacks to ensure the `min_stack` correctly reflects the current state of the main stack.

Why this approach

This approach is chosen because it satisfies the constraint of $O(1)$ time complexity for all operations without needing complex data structures or algorithms. It uses the stack data structure efficiently and the logic is straightforward and easy to understand.

Alternative approaches

Without the restriction of $O(1)$ time for all operations, we could have used a single stack and when a `getMin` operation is called, we could iterate through the stack to find the minimum. However, this would not meet the $O(1)$ time complexity requirement.

Similar problems to this one

Similar problems include "155. Min Stack" on LeetCode and variations where you might need to also retrieve the maximum element, or maintain a sorted version of the stack.

Things to keep in mind and tricks

Remember that each operation must run in $O(1)$ time complexity. This means that for the getMin operation, we cannot search through the stack – we must have the minimum readily available. The use of a secondary stack helps satisfy the constant time requirement.

Corner and special cases to test when writing the code

It's important to test edge cases such as:

- Pushing elements in increasing or decreasing order.
- Pushing duplicate elements and ensuring the minimum is maintained accurately after pops.
- Popping until the stack is empty and then trying to pop or get the top element/min.

All the above cases should be handled without any errors and should maintain the $O(1)$ time complexity.

647. Palindromic Substrings

Problem Statement

Given a string s , the objective is to find the number of palindromic substrings within that string. A palindrome is defined as a sequence of characters that reads the same forwards as it does backwards. A substrings is a contiguous block of characters within the original string.

LeetCode link: [Problem 647](#)

Algorithmic Approach

The algorithm to solve this problem involves two main steps:

1. Iterate through each character in the string, considering it as the middle of a possible palindrome.
2. Expand around the center for both odd and even length palindromes, counting all valid palindromes.

To implement this, a helper function can check for palindromes by expanding from the center. This function will be called twice for each character: once for the odd length palindrome (with the same start and end) and once for the even length (with start at the current character and end at the next character).

Complexities

- Time Complexity: The time complexity for this approach is $O(n^2)$, where n is the length of the string. This is because for each character, we potentially expand in both directions until we reach the ends of the string.
- Space Complexity: The space complexity is $O(1)$ as we are not using any additional space that scales with the input size. The only extra space used is for a few variables to keep track of counts and indices.

Python Implementation

```

class Solution:
    def countSubstrings(self, s: str) -> int:
        def expand_around_center(left: int, right: int) -> int:
            count = 0
            while left >= 0 and right < len(s) and s[left] == s[
                ↪ right]:
                count += 1
                left -= 1
                right += 1
            return count

        total_palindromes = 0
        for i in range(len(s)):
            total_palindromes += expand_around_center(i, i)      #
                ↪ For odd lengths
            total_palindromes += expand_around_center(i, i + 1) #
                ↪ For even lengths

        return total_palindromes

```

Explanation

The provided Python implementation defines a function `countSubstrings` which takes a string `s` as its parameter. Within this function, a nested helper function `expand_around_center` is defined, which takes two indices `left` and `right`. The helper function expands outwards from these indices while the characters at `left` and `right` are equal, incrementing the count of palindromic substrings for each iteration of the `while` loop. This helper function is then called for each character in the string `s`, counting all odd and even palindromic substrings centered at that character.

Why this approach

This approach is chosen because it efficiently examines potential palindromes by expanding around a center, allowing us to count all palindromes in $O(n^2)$ without the need for additional space. It's a direct and intuitive method to solve the problem, given the definition and properties of palindromes.

Alternative approaches

An alternative approach could use dynamic programming to build up a table that keeps track of which substrings are palindromes, but this would increase the space complexity to $O(n^2)$. The Manacher's algorithm can also be used to solve this problem in linear time, but it is significantly more complex to implement.

Similar problems to this one

Similar problems would include those related to substrings, such as the longest palindromic substring or problems involving dynamic programming for string manipulation. Problems involving counting particular types of subsequences or patterns within a string may also apply similar methodologies.

Things to keep in mind and tricks

When solving problems involving palindromes, always consider the possibility of expanding from the center. This approach often simplifies the algorithm. Moreover, be aware of the differences in handling even and odd length palindromes.

Corner and special cases to test when writing the code

To thoroughly test the implementation, consider corner cases such as: - A single character string - A string with all identical characters - A string with no palindromic substrings longer than 1 - Very long strings to ensure the performance is within acceptable bounds

Jump Game I

The "Jump Game I" problem is a well-known challenge in LeetCode's array and dynamic programming category. The goal is to determine if it is possible to reach the final index of an array, where each element represents the maximum jump length from that position.

Problem Statement

Given an array of non-negative integers 'nums', where 'nums[i]' is the maximum jump length at position 'i', the task is to decide whether it is possible to reach the last index starting from the first index. The output should be a boolean, 'true' if you can reach the last index, otherwise 'false'.

Leet code link: [Jump Game I](#)

Algorithmic Approach

The solution to this problem can be approached by using a greedy algorithm. It involves iterating over each element of the array and updating the furthest reachable index to see if one can reach the end.

Complexities

- **Time Complexity:** The total time complexity of the algorithm is $O(N)$, where N is the number of elements in the input array.
- **Space Complexity:** The space complexity is $O(1)$ since we are not using any additional space that scales with the input size.

Python Implementation

Below is the complete Python code for the ‘Solution’ class, which implements the ‘canJump’ method to check if it is possible to jump to the last index:

```
class Solution(object):
    def canJump(self, nums):
        """
        :type nums: List[int]
        :rtype: bool
        """
        max_reachable = 0
        last_index = len(nums) - 1

        for i in range(len(nums)):
            if i > max_reachable:
                return False
            max_reachable = max(max_reachable, i + nums[i])
            if max_reachable >= last_index:
                return True

        return False
```

This implementation iterates through each index of the array and checks if the current index is within the reach of the previously computed maximum reachable index (`max_reachable`). If it is, it updates the `max_reachable` with the maximum value between itself and the sum of the current index `i` plus the jump length `nums[i]`.

Why this approach

The greedy algorithm is chosen for its efficiency and simplicity. By always tracking the furthest we can reach at each step, we can solve the problem by making a single pass through the array, thus achieving a linear time complexity.

Alternative approaches

An alternative approach would be to use dynamic programming to solve the problem. This would involve building an array where each element represents whether you can reach the current position from the start. However, this approach has a higher time complexity of $O(N^2)$.

Similar problems to this one

Similar problems include Jump Game II, which asks for the minimum number of jumps to reach the end of the array, and Jump Game III, which includes additional constraints such as negative jump lengths.

Things to keep in mind and tricks

One common trick in solving this type of problem is to work backwards from the final index or be greedy about your choices, only concerning yourself with the most extended jump you can make at each step. Also, keep track of the maximum reachable index at every position rather than the number of steps taken.

Corner and special cases to test when writing the code

When writing the code, be sure to handle edge cases, such as when the array contains only one element, or when the maximum jump length at an early index is zero (potentially creating an unreachable position). Test the code with the maximum length array and jumps to ensure it handles large inputs efficiently.

Skyline Problem

Problem Statement

Given a list of buildings represented by tuples $(left, right, height)$, the task is to compute the skyline formed by these buildings when viewed from a distance. Each tuple represents a building where $left$ and $right$ are the x-coordinates of the left and right sides of the building, respectively, and $height$ denotes the height of the building.

The skyline should be represented as a list of tuples $(x, height)$, where each tuple represents a point on the skyline with its x-coordinate and height. These points must be sorted by their x-coordinate in ascending order. If multiple points share the same x-coordinate, they should be sorted by their height in descending order.

Algorithmic Approach

To solve this problem, we can use a priority queue or a sorted data structure to keep track of the active buildings at any point in time while we sweep over the x-coordinates. We consider both the start and end points of the buildings. An event-based approach is commonly used where we process two types of events: building starts and building ends.

Complexities

The time complexity of this problem primarily depends on the sorting of the events and how efficiently we can add, find, and remove buildings from our active set. Typically, with an efficient priority queue or self-balancing binary search tree, the overall time complexity would be $O(N \log N)$, where N is the number of events (start and end of buildings). The space complexity would be $O(N)$ for storing the events and the active buildings.

Python Implementation

```

class Solution:
    def getSkyline(self, buildings):
        # Events sorted by x-coordinate (left edge before right
        #   ↪ edge), and by height descending
        events = [(L, -H, R) for L, R, H in buildings]
        events += list({(R, 0, 0) for _, R, _ in buildings})
        events.sort()

        # Result list, with an initial dummy building
        res = [[0, 0]]

        # Active buildings sorted by height
        active_buildings = [(0, float('inf'))] # (height, right)

        for x, neg_h, R in events:
            while x >= active_buildings[0][1]:
                # Remove buildings that are already passed
                heapq.heappop(active_buildings)

            if neg_h:
                # Push current building into the heap
                heapq.heappush(active_buildings, (neg_h, R))

            # If the height has changed from the last building, add
            #   ↪ to result
            if res[-1][1] != -active_buildings[0][0]:
                res.append([x, -active_buildings[0][0]])

        return res[1:] # Remove initial dummy building

# Example usage:
# buildings = [(2,9,10), (3,7,15), (5,12,12), (15,20,10), (19,24,8)
#   ↪ ]
# sol = Solution()
# print(sol.getSkyline(buildings))

```

Explanation

The provided Python code maintains a list of 'events', each event either signifies the start or the end of a building. It ensures that for any x-coordinate, the start event is processed before the end event.

Why this approach

This approach is chosen because it efficiently processes the buildings in order, and uses a min-heap to keep track of the highest building at the current position of the x-coordinate sweep line.

Alternative approaches

An alternative approach could use a balanced binary search tree (BST) to maintain the active buildings' heights instead of a heap, which would still result in $O(N \log N)$ time complexity.

Similar problems to this one

A similar problem would be merging intervals, where the intervals represent time slots or ranges, and the goal is to merge overlapping intervals.

Things to keep in mind and tricks

One of the main things to keep in mind is handling the removal of buildings from the priority queue efficiently. It's also important to handle buildings with the same x-coordinate correctly.

Corner and special cases to test when writing the code

Special cases include buildings with zero width, buildings with the same height, and overlapping buildings. It's also crucial to handle the transition between consecutive buildings correctly.

Decode Ways

The "Decode Ways" problem is a classic dynamic programming challenge that requires determining the total number of ways a given digit string can be decoded into letters, where each letter from A to Z is represented by numbers from 1 to 26.

Problem Statement

The task is to find the number of ways to decode a string s that consists only of digits with a mapping from 'A' to 'Z' where 'A' maps to "1", 'B' maps to "2", ..., and 'Z' maps to "26". A valid decoding can be any combination of single-digit numbers or two-digit numbers that fall within the range from "1" to "26".

LeetCode link: [Decode Ways](#)

Algorithmic Approach

To solve this problem, we can use dynamic programming. We define a cache array where $cache[i]$ represents the number of ways to decode the substring $s[:i]$. We iterate through the given string and update the cache based on whether the current character or the combination of the current and previous characters can represent a valid decoding.

Complexities

- **Time Complexity:** The total time complexity is $O(n)$ where n is the length of the string, since we iterate through the string once and perform $O(1)$ operations for each character.
- **Space Complexity:** The space complexity is $O(n)$ for caching the number of ways to decode each substring. However, we can optimize this to $O(1)$ by only keeping track of the number of ways to decode the last two substrings.

Python Implementation

Below is the complete Python code to the ‘numDecodings’ function which returns the total number of ways to decode the input string *s*:

```
class Solution:
    def numDecodings(self, s: str) -> int:
        if not s or s[0] == '0':
            return 0

        # Initialization
        prev = 1 # Number of ways to decode an empty string
        curr = 1 # Number of ways to decode a string with one
            ↪ character

        for i in range(1, len(s)):
            temp = curr
            if s[i] == '0':
                if s[i-1] == '1' or s[i-1] == '2': # Check for
                    ↪ valid "10" or "20"
                    curr = prev
            else:
                return 0
            elif 10 < int(s[i-1:i+1]) <= 26: # Check if the two-
                ↪ digit number is valid
                curr += prev
            prev = temp

        return curr
```

This implementation first checks if the input string is empty or starts with ‘0’. If either is true, there are no ways to decode the message, and it returns ‘0’. It initializes two variables, ‘prev’ and ‘curr’, which keep track of the number of ways to decode substrings of length ‘i-1’ and ‘i’, respectively, then iterates through the string while updating these variables based on each character’s validity both individually and potentially as part of a two-digit number.

Why this approach

The dynamic programming approach is chosen for its efficiency in both time and space. It allows us to build up from smaller subproblems, reducing the overall number of redundant calculations as compared to a naive recursive solution.

Alternative approaches

An alternative approach is to use a recursive function with memoization to reduce the repetition of subproblem calculations. However, this may still not be as efficient as the dynamic programming approach presented above, particularly in terms of space complexity.

Similar problems to this one

Similar problems include "Climbing Stairs," "House Robber," and other sequence-based dynamic programming problems where the current state depends on a combination of one or more previous states.

Things to keep in mind and tricks

When solving dynamic programming problems, especially those involving strings, it's important to consider edge cases such as empty strings and strings with leading zeros. For this specific problem, the decoding is invalid if there are any zeros not preceded by '1' or '2'.

Corner and special cases to test when writing the code

Some corner cases include: - A single '0' or strings with leading '0's. - Strings only composed of '1's and '2's, which may result in a larger number of valid decodings. - Strings with numbers greater than '26' which should not increase the count of valid decodings.

Longest Increasing Subsequence

The Longest Increasing Subsequence (LIS) problem is a classic problem in the field of computer science that comes under the dynamic programming category.

Problem Statement

The task is to find the length of the longest strictly increasing subsequence in an unsorted array of integers. A subsequence is a sequence that can be derived from the array by deleting some or no elements without changing the order of the remaining elements. A strictly increasing subsequence is one where each element is greater than the one before it.

For example:

Input: `nums = [10, 9, 2, 5, 3, 7, 101, 18]`

Output: 4

Explanation: The longest increasing subsequence is [2, 3, 7, 101], therefore the length is 4.

LeetCode link: [Longest Increasing Subsequence](#)

Algorithmic Approach

The solution to this problem can be approached by using dynamic programming to construct a table where the i -th element stores the length of the longest increasing subsequence ending with the i -th element. Alternatively, one can use patience sorting technique to optimize the time complexity further.

Complexities

- **Time Complexity:** The total time complexity of the dynamic programming approach is $O(n^2)$, where n is the length of the input array. Using a binary search with patience sorting, the time complexity can be improved to $O(n \log n)$.
- **Space Complexity:** The space complexity is $O(n)$ due to the use of an additional array for storing the lengths of the subsequences.

Python Implementation

Below is the complete Python code for the 'lengthOfLIS' function to determine the length of the longest increasing subsequence:

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        if not nums:
            return 0

        LIS = [1] * len(nums)
        for i in range(1, len(nums)):
            for j in range(i):
                if nums[i] > nums[j]:
                    LIS[i] = max(LIS[i], LIS[j] + 1)
        return max(LIS)
```

This implementation constructs an array 'LIS', initialized with ones. It then iteratively updates the 'LIS' array by comparing each element with all the previous elements. If a valid increasing subsequence is found, the 'LIS' value is updated. Finally, the maximum value in the 'LIS' array is returned, which represents the length of the longest increasing subsequence.

Why this approach

The dynamic programming approach was chosen for its relatively straightforward implementation and reduced time complexity compared to a brute-force solution. It efficiently builds upon solutions to smaller problems, storing intermediate results to avoid redundant calculations.

Alternative approaches

An alternative approach involves the patience sorting technique coupled with binary search, reducing the overall time complexity. This approach takes advantage of the properties of the problem and demonstrates a more complex understanding of the underlying structures.

Similar problems to this one

Variants and extensions of the LIS problem include finding the actual longest increasing subsequence, determining the number of such subsequences, and solving the problem with modifications in the subsequence conditions (e.g., non-decreasing or with a fixed step size).

Things to keep in mind and tricks

- Always consider edge cases such as empty arrays or arrays with all identical elements.
- Binary search is a significant optimization when the problem constraints

allow. - It is crucial to thoroughly understand the subproblem overlap in dynamic programming to devise an optimal solution.

Corner and special cases to test when writing the code

- Arrays with only one element (should return 1). - Arrays with elements in strictly decreasing order (should return 1). - Arrays with all elements identical (should return 1). - Long arrays to evaluate performance and ensure that the implementation does not exceed time limits.

Longest Consecutive Sequence

The "Longest Consecutive Sequence" problem requires finding the length of the longest sequence of consecutive numbers in an unsorted array. This must be achieved with a time complexity of $O(n)$, which rules out the possibility of sorting the array as a pre-processing step.

Problem Statement

Given an unsorted array of integers 'nums', return the length of the longest consecutive elements sequence. It is essential that the algorithm operates in $O(n)$ time.

Examples:

- Example 1:

Input: `nums = [100,4,200,1,3,2]`

Output: 4

Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

- Example 2:

Input: `nums = [0,3,7,2,5,8,4,6,0,1]`

Output: 9

LeetCode link: [Longest Consecutive Sequence](#)

Algorithmic Approach

A hashtable or set can be used for this problem to store the integers and check for the presence of consecutive numbers. The algorithm iterates over the array and for each number, checks if it is the start of a sequence (i.e., the number just before it is not in the array). If it is the start, it continues to check consecutive numbers using the set for existence. This process is repeated for each number that is identified as a potential start of a sequence. The maximum length found during the iteration is the answer.

Complexities

- **Time Complexity:** $O(n)$, since each number in the array is processed once to insert into the set and at most once when finding a sequence.
- **Space Complexity:** $O(n)$, as a set is used to store the elements of the array.

Python Implementation

The Python code below implements the solution for finding the longest consecutive sequence in an array:

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        num_set = set(nums)
        longest_streak = 0

        for num in num_set:
            if num - 1 not in num_set:
                current_num = num
                current_streak = 1

                while current_num + 1 in num_set:
                    current_num += 1
                    current_streak += 1

                longest_streak = max(longest_streak, current_streak
                    ↪ )

        return longest_streak
```

The provided code uses a set to achieve constant time look-up when checking for consecutive numbers. It starts a new count whenever a number is the beginning of a new sequence (i.e., the number just smaller is not in the set). The code efficiently avoids counting consecutive sequences multiple times by counting only from the beginning of such sequences.

Why this approach

This approach is chosen for its efficiency in terms of time complexity. Instead of sorting which would take $O(n \log n)$ time, the algorithm leverages a set's $O(1)$ average time complexity for element lookup, enabling it to find the longest consecutive sequence in linear time.

Alternative approaches

An alternative method might involve sorting the array first; however, as mentioned, this would violate the $O(n)$ time complexity requirement. Another idea could be to use a Union-Find data structure, but that is generally more complex and not necessary when a simple set suffices.

Similar problems to this one

Related problems include "Missing Ranges" and "Summary Ranges", where the focus is on identifying missing or continuous ranges in sorted arrays. The core

idea of identifying sequences is similar; however, the constraints and output formats differ.

Things to keep in mind and tricks

The key to solving this problem efficiently lies in realizing that a set can be used to avoid re-examining numbers already identified as part of a sequence. It is also important to understand that counting should only commence from the sequence start points, which conserves operations.

Corner and special cases to test when writing the code

Be wary of duplicate numbers in the input, which is why a set is used to store the numbers. The edge cases where the array is empty or contains a single element should also be handled cleanly by the algorithm.

Minimum Window Substring

The Minimum Window Substring problem is a classic problem in the realm of string manipulation and searching algorithms.

Problem Statement

Given two strings 's' and 't', return the minimum window substring of 's' such that every character in 't' (including duplicates) is included in the window. If there is no such substring, return the empty string "".

A substring is a contiguous sequence of characters within the string.

Examples:

- Input: s = "ADOBECODEBANC", t = "ABC"
- Output: "BANC"
- Input: s = "a", t = "a"
- Output: "a"

Algorithmic Approach

To solve the problem using a sliding window approach, start with two pointers both set at the beginning of 's'. Expand the right pointer to find a valid window that contains all the characters from 't'. Once you have a valid window, increment the left pointer to try and minimize the size of the window without compromising its validity (i.e., it should still contain all the characters from 't'). Repeat this process until all potential windows have been checked. Return the smallest valid window observed during this process.

Complexities

- **Time Complexity:** The overall time complexity is $O(n + m)$, where n is the length of string 's' and m is the length of string 't'.
- **Space Complexity:** The space complexity is $O(m)$ for storing the characters of string 't' in a hash table.

LeetCode link: [Minimum Window Substring](#)

Python Implementation

Below is the complete Python code for solving the Minimum Window Substring problem:

```
from collections import Counter

def minWindow(s: str, t: str) -> str:
    if not t or not s:
        return ""

    # Dictionary which keeps a count of all the unique characters
    #   ↪ in t.
    dict_t = Counter(t)

    # Number of unique characters in t, which need to be present in
    #   ↪ the desired window.
    required = len(dict_t)

    # Left and Right pointer
    l, r = 0, 0

    # Formed is used to keep track of how many unique characters in
    #   ↪ t are present in the current window in its desired
    #   ↪ frequency.
    # e.g. if t is "AABC" then the window must have two A's, one B
    #   ↪ and one C. Thus formed would be = 3 when all these
    #   ↪ conditions are met.
    formed = 0

    # Dictionary which keeps a count of all the unique characters
    #   ↪ in the current window.
    window_counts = {}

    # ans tuple of the form (window length, left, right)
    ans = float("inf"), None, None

    while r < len(s):
        # Add one character from the right to the window
        character = s[r]
        window_counts[character] = window_counts.get(character, 0)
        #   ↪ + 1

        # If the frequency of the current character added equals to
        #   ↪ the desired count in t then increment the formed
        #   ↪ count by 1.
        if character in dict_t and window_counts[character] ==
            #   ↪ dict_t[character]:
            formed += 1

        # Try and contract the window till the point where it
```



```

    ↪ ceases to be 'desirable'.
while l <= r and formed == required:
    character = s[l]

    # Save the smallest window until now.
    if r - l + 1 < ans[0]:
        ans = (r - l + 1, l, r)

    # The character at the position pointed by the 'left'
    ↪ pointer is no longer a part of the window.
    window_counts[character] -= 1
    if character in dict_t and window_counts[character] <
        ↪ dict_t[character]:
        formed -= 1

    # Move the left pointer ahead, this would help to look
    ↪ for a new window.
    l += 1

    # Keep expanding the window once we are done contracting.
    r += 1
return "" if ans[0] == float("inf") else s[ans[1]: ans[2] + 1]

# Example usage:
print(minWindow("ADOBECODEBANC", "ABC")) # Output: "BANC"
print(minWindow("a", "a"))                # Output: "a"

```

This implementation uses a hash table to store the frequency of each character in 't', then uses a sliding window to find the smallest substring of 's' that contains all the characters of 't'. The hash table helps in quickly checking if the current window contains all the required characters.

Why this approach

This approach is chosen because it efficiently scales with the length of the strings 's' and 't'. Although the worst-case time complexity appears to be $O(n \cdot m)$, the use of a hash table ensures that the check for the current window can be done in constant time, thus effectively bringing the overall time complexity down to $O(n + m)$.

Alternative approaches

An alternative approach would be to use a fixed-size character array instead of a hash table to store the frequencies. However, this is only feasible if the character set of 's' and 't' is limited (e.g., ASCII characters only).

Similar problems to this one

Problems like "Longest Substring Without Repeating Characters", "Substring with Concatenation of All Words", and "Permutation in String" involve similar techniques of sliding window and hashing.

Things to keep in mind and tricks

One must take care to handle the counting accurately, especially when a character leaves the window when the left pointer moves. Also, initializing the 'ans' tuple with 'float("inf")' allows easy checking if no window was found.

Corner and special cases to test when writing the code

It's important to consider cases where 's' is much larger than 't', cases where 't' has repeated characters, and cases where no valid window is possible. These help in verifying that the window resizing logic is accurate and that the counting logic does not have off-by-one errors.

Set Matrix Zeroes

The "Set Matrix Zeroes" problem is a common interview question and falls under the category of array manipulation. It challenges the problem-solver to efficiently work with a two-dimensional array with the constraint of modifying the array in place.

Problem Statement

Given an $m \times n$ matrix, if an element is 0, set its entire row and column to 0. The key requirement is to perform this operation in place, without utilizing additional memory for another matrix structure.

Examples:

Example 1:

Input:

```
[
  [1,1,1],
  [1,0,1],
  [1,1,1]
]
```

Output:

```
[
  [1,0,1],
  [0,0,0],
  [1,0,1]
]
```

Example 2:

Input:

```
[
  [0,1,2,0],
  [3,4,5,2],
  [1,3,1,5]
]
```

Output:

```
[  
  [0,0,0,0],  
  [0,4,5,0],  
  [0,3,1,0]  
]
```

LeetCode link: [Set Matrix Zeroes](#)

Algorithmic Approach

The solution to this problem can be approached efficiently by using the matrix itself to store state information. By iterating over the matrix, we can use the first element of each row and the first element of each column to mark whether the rest of the rows or columns should be set to zero.

Complexities

- **Time Complexity:** Overall, the time complexity is $O(m \times n)$ since we need to traverse the entire matrix at least once.
- **Space Complexity:** The auxiliary space complexity is $O(1)$ as we are working in-place.

Python Implementation

Below is the complete Python code for the ‘setZeroes’ function which implements the in-place algorithm for setting matrix zeroes:

```
def setZeroes(matrix):
    m, n = len(matrix), len(matrix[0])
    rowZero = False

    # Determine which rows/cols need to be zeroed
    for i in range(m):
        for j in range(n):
            if matrix[i][j] == 0:
                matrix[0][j] = 0
                if i == 0:
                    rowZero = True
            else:
                matrix[i][0] = 0

    # Zero out cells based on the first row and column
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][0] == 0 or matrix[0][j] == 0:
                matrix[i][j] = 0

    # Zero out the first column if necessary
    if matrix[0][0] == 0:
        for i in range(m):
            matrix[i][0] = 0

    # Zero out the first row if necessary
    if rowZero:
        for j in range(n):
            matrix[0][j] = 0
```

The first loop determines which rows and columns should be set to zero, marking them using their first cell. The nested loops zero out matrix elements with the guidance of these markers. Special care is required for the first row, which we handle separately using the ‘rowZero’ flag.

Why this approach

This approach was chosen due to its efficiency in both time and space complexity. It uses the input matrix itself to store information about which rows and columns are to be set to zero, avoiding the use of additional storage.

Alternative approaches

An alternative could involve the use of extra space where we would create additional arrays to keep track of rows and columns to be zeroed.

Similar problems to this one

Similar problems that also deal with in-place array manipulation include rotating a matrix, and inverting a matrix without using additional space.

Things to keep in mind and tricks

When dealing with in-place modifications, one should be cautious of any changes that might affect future decisions in your algorithm. Make sure that the modifications made do not propagate in an unintended manner causing incorrect alterations.

Corner and special cases to test when writing the code

It's critical to consider the special cases, especially when the zero occurs in the first row or the first column since these entries are used to mark other rows and columns for modification. Test cases with empty matrices or matrices with a single row or column should also be considered.

“latex

Merge k Sorted Lists

The "Merge k Sorted Lists" problem is a classic algorithm problem that is often asked in coding interviews. The problem challenges the solver to efficiently combine multiple sorted data streams into a single sorted data stream, which has practical applications in various domains such as merging time-series data from multiple sources or combining sorted log files.

Problem Statement

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the k sorted linked-lists into one sorted linked-list and return it.

Example:

Consider the following k sorted linked lists:

List 1: $1 \rightarrow 4 \rightarrow 5$

List 2: $1 \rightarrow 3 \rightarrow 4$

List 3: $2 \rightarrow 6$

The merged list should be:

$1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4 \rightarrow 5 \rightarrow 6$

Input: The input consists of an array of k pointers to the head nodes of each of the k sorted linked lists.

Output: The output should be the head of the single sorted linked list that is the result of merging the k sorted lists.

LeetCode link: <https://leetcode.com/problems/merge-k-sorted-lists/>

Algorithmic Approach

The solution to this problem can be approached by using a min-heap or a priority queue to efficiently manage the current smallest nodes from each linked list.

This method takes advantage of the fact that the heads of each linked list are the smallest elements remaining in each list, so we can perform a similar operation to a merge in merge sort by always selecting the smallest head node.

Complexities

- **Time Complexity:** The total time complexity is $O(N \log k)$, where N is the total number of nodes and k is the number of linked lists.
- **Space Complexity:** The space complexity is $O(k)$ for storing the pointers in the heap at any given time.

Python Implementation

Below is the complete Python code for the ‘Solution’ class, which implements the ‘mergeKLists’ method to merge k sorted linked lists using a min-heap for efficient retrieval of the smallest node at any step:

```
from Queue import PriorityQueue

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def mergeKLists(self, lists):
        head = point = ListNode(0)
        pq = PriorityQueue()
        for l in lists:
            if l:
                pq.put((l.val, l))
        while not pq.empty():
            val, node = pq.get()
            point.next = ListNode(val)
            point = point.next
            node = node.next
            if node:
                pq.put((node.val, node))
        return head.next
```

This implementation initially sets up a dummy head and a point reference to track the merged list. The ‘PriorityQueue’ is used to store the tuple ‘(val, node)’ for each head of the linked lists. It ensures the heap property such that the smallest value is always at the top. By continuously extracting the smallest and inserting the next element from the same list, the algorithm merges all lists into one sorted linked list.

Why this approach

The min-heap approach was chosen for its optimal time complexity considering the need to frequently find and remove the smallest element from a collection of sorted arrays. This method is efficient because it maintains a heap of only k elements representing the heads of each list, and thus the extract-min and insert operations are $O(\log k)$.

Alternative approaches

Alternative approaches include the brute force method, which involves collecting all nodes into an array, sorting it, and then creating a new sorted list. Another

approach is to compare nodes one by one or use divide and conquer to merge lists in pairs.

Similar problems to this one

Similar problems include "Merge Two Sorted Lists" and "Sort List," where the principles of merging or handling sorted data structures are central to the solution.

Things to keep in mind and tricks

Keep

Coin Change

The "Coin Change" problem is a well-known dynamic programming challenge that requires determining the minimum number of coins needed to make a certain amount of money given coin denominations.

Problem Statement

Given an array 'coins' of different denominations and an integer 'amount' representing a total amount of money, the aim is to compute the fewest number of coins required to make up that amount. If it is not possible to make the amount with any combination of the coins, the function should return -1.

Example 1:

Input: coins = [1, 2, 5], amount = 11

Output: 3

Explanation: $11 = 5 + 5 + 1$

Example 2:

Input: coins = [2], amount = 3

Output: -1

Example 3:

Input: coins = [1], amount = 0

Output: 0

LeetCode link: [Coin Change](#)

Algorithmic Approach

The solution utilizes dynamic programming to build a table that records the minimal number of coins necessary for each sub-amount up to the target amount. We use an array 'dp' with 'amount + 1' entries, where 'dp[i]' holds the minimum number of coins required for the value 'i'. Initially, 'dp[0]' is set to 0 (since zero coins are needed for amount zero), and the other entries are set to some maximum value to symbolize an unreachable state.

Complexities

- **Time Complexity:** The overall time complexity is $O(\text{amount} \times \text{number of coins})$ because we traverse through all sub-amounts for each coin.
- **Space Complexity:** The space complexity is $O(\text{amount})$ since we maintain a 1-dimensional DP array to store minimum coins for each amount up to the target amount.

Python Implementation

Below is the complete Python code for the 'coinChange' function, which finds the least number of coins needed given denominations to make up a certain amount:

```
def coinChange(coins, amount):
    # Initialize DP array with amount+1, which is an invalid value
    #   ↳ as it's
    #   ↳ bigger than the amount itself (since the maximum number of
    #   ↳ coins to
    # make any amount is amount (using all 1s)).
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0 # Base case: no coins needed for amount 0

    # Solve for all amounts from 1 to amount
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:
                # If coin is not greater than the amount
                dp[a] = min(dp[a], 1 + dp[a - coin])

    # If dp[amount] is still amount+1, then it was not possible to
    #   ↳ make change
    return dp[amount] if dp[amount] != amount + 1 else -1
```

This implementation first initializes the 'dp' array with a default value that's not viable, signifying that for that amount, the minimum number of coins is still unknown. For every sub-amount up to the total amount, it tries each coin and determines if including that coin would result in a smaller number of coins to make the exact change.

Why this approach

The dynamic programming approach was chosen for its ability to break down the problem into subproblems and build up the answer efficiently. This characteristic of DP ensures that we do not recompute the minimum coins for the amounts we have already solved.

Alternative approaches

There are alternative approaches, such as a recursive solution or using a BFS approach, but these can be less efficient in terms of time complexity unless memoization is used with recursion. The greedy approach doesn't work for all sets of coin denominations.

Similar problems to this one

This problem is similar to other classic DP problems like "Minimum Path Sum," "0/1 Knapsack," and "Longest Increasing Subsequence," where subproblems are

solved to contribute to the overall solution.

Things to keep in mind and tricks

One of the key things

Sum of Two Integers Without Using Plus or Minus Operators

The problem's objective is to calculate the sum of two integers without using the '+' or '-' arithmetic operators. To address this challenge, we utilize bitwise operations that are fundamental to how computers perform addition at a low level.

Problem Statement

Given two integers a and b , we are tasked with implementing the function `getSum(a, b)` that returns the sum of a and b without using '+' or '-' operators.

LeetCode link: [Sum of Two Integers](#)

Algorithmic Approach

To achieve our goal, we use the following bitwise operations:

- **AND** (&), which can be used to determine carries in addition.
- **XOR** (^), which can be used to perform addition without considering carries.
- **Left Shift** (<<), which can be used to propagate carries.

The algorithm can be described in the following steps:

1. Compute the carry by performing an AND operation between a and b , and shift the result to the left by one bit.
2. Compute the partial sum by performing an XOR operation between a and b .
3. Continue this process, treating the carry as the new b and the partial sum as the new a , until there is no carry.

Complexities

- The **time complexity** is $O(1)$ because the number of operations is constant and does not depend on the size of the input numbers, assuming a constant bit-size (such as the 32-bit integers mentioned in the problem statement).

- The **space complexity** is also $O(1)$ since we are using a fixed amount of extra space.

Python Implementation

```

def getSum(a, b):
    # 32 bits integer max
    MAX = 0x7FFFFFFF
    # 32 bits integer min
    MIN = 0x80000000
    # mask to get last 32 bits
    mask = 0xFFFFFFFF

    while b != 0:
        # ^ get different bits and & gets double 1s, << moves carry
        a, b = (a ^ b) & mask, ((a & b) << 1) & mask

    # if a is negative, do a mask to get a 32 bits complement
    # ↪ positive number
    return a if a <= MAX else ~(a ^ mask)

```

Explanation

The function `getSum` repeatedly uses XOR to calculate the sum without carries and AND followed by left shift to calculate the carry until the carry is zero. When the carry is zero, the XOR result is the final sum.

Why This Approach

Bitwise operations are a natural fit for this problem because they are essentially the same processes a computer uses internally to perform arithmetic. They are fast and efficient.

Alternative Approaches

An alternative approach involves simulating the full adder logic used in digital circuits, which constitutes the XOR, AND, and OR gates to produce the sum and carry out. However, bitwise operation remains more concise for this problem.

Similar Problems to This One

An analogous problem is to subtract two integers without using '+' or '-'. This involves complementary operations using bit manipulation.

Things to Keep in Mind and Tricks

It is essential to apply a mask to ensure we only consider the proper number of bits, especially to handle negative numbers in languages like Python that do not have fixed integer sizes.

Corner and Special Cases to Test When Writing the Code

Since the sum may not exceed the range of a 32-bit signed integer, it is critical to test the function with values close to the top and bottom bounds, such as $2^{31} - 1$ (the maximum positive 32-bit signed integer) and -2^{31} (the minimum negative 32-bit signed integer). Additionally, it is also crucial to test the algorithm with zero and with one of the operands being zero to ensure that these cases are correctly handled.

Missing Number

The "Missing Number" problem is a search problem that involves finding the number that is missing from a given sequence.

Problem Statement

Given an array `nums` containing n distinct numbers in the range 0 to n , write a function to find the missing number in the sequence. Here, n represents the largest number in the sequence, which also indicates the length of the sequence since one number is missing.

For example, consider the sequence $[3, 0, 1]$, the missing number is 2 as it does not appear in the sequence. Similarly, in the sequence $[9, 6, 4, 2, 3, 5, 7, 0, 1]$, the missing number is 8.

LeetCode link: [Missing Number](#)

Algorithmic Approach

The key to solving this problem efficiently is to utilize either mathematical properties or bit manipulation. A mathematical approach involves summing the numbers and comparing with the expected sum. Alternatively, the XOR approach exploits the property that XOR-ing a number with itself results in zero and XOR-ing a number with zero results in the number itself.

Complexities

- **Time Complexity:** The total time complexity is $O(n)$ as we have to traverse the entire array once.
- **Space Complexity:** This can be achieved in $O(1)$ extra space by using mathematical summation or XOR which does not require additional data structures.

Python Implementation

Below is the complete Python code for the ‘Solution’ class, which includes one method to find the missing number using XOR bit manipulation:

```
class Solution:
    def missingNumber(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        missing = len(nums)
        for i, num in enumerate(nums):
            missing ^= i ^ num
        return missing
```

This implementation leverages the property of XOR that $a \oplus a = 0$ and $a \oplus 0 = a$. It iterates through the list of numbers and XORs each element with its index. In addition, it XORs with the length of the array to cover the full range from 0 to n . The result is the missing number in the sequence.

Why this approach

The XOR approach was chosen for its simplicity and efficiency. It runs in linear time and requires constant space, satisfying the problem constraints. By encoding both the element and its index in the XOR, we effectively cancel out each pair of numbers, leaving only the missing number.

Alternative approaches

An alternative method is to use mathematical summation of an arithmetic series. The sum of numbers from 0 to n can be calculated using the formula $n \times (n + 1) / 2$. Subtracting the sum of array elements from this total should give the missing number.

Similar problems to this one

Similar problems include finding duplicate numbers in a sequence or finding missing numbers when multiple elements are missing. Another variant is to solve such problems with additional space constraints or under different computational models.

Things to keep in mind and tricks

In computational problems involving sequences and ranges, leveraging mathematical formulas and properties of operations like XOR can often lead to efficient solutions. It is also helpful to be aware that XOR is its own inverse, which is a useful property that can simplify such problems.

Corner and special cases to test when writing the code

When implementing the solution, remember to test edge cases such as an empty sequence, a sequence with just one number, and cases where the missing number is either the first or the last in the range.

N-Queens Problem

The n-Queens problem is a well-known puzzle that requires placing n queens on an $n \times n$ chessboard such that no two queens threaten each other. This means that no two queens can be placed in the same row, column, or diagonal. The challenge is to find all possible arrangements of n queens on the board.



Figure 3: A cartoon illustrating the n-Queens problem.

Problem Statement

Algorithmic Approach

Explanation

The n-Queens problem is a well-known puzzle that requires placing n queens on an $n \times n$ chessboard such that no two queens threaten each other. This means that no two queens can be placed in the same row, column, or diagonal. The challenge is to find all possible arrangements of n queens on the board.

To solve the n-Queens problem, a common algorithmic approach is to use backtracking. Backtracking is a form of recursion that tries to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point in time (thus, "backtracking" and trying another set of solutions). This approach explores all possible placements for the queens, ensuring that no two queens threaten each other at any stage of the placement.

Complexities

The time complexity of the algorithm is $O(n!)$, which is the number of possible ways to place n queens on an $n \times n$ board without considering the constraints. However, the backtracking algorithm significantly prunes the search space, yet the worst-case complexity remains exponential because the possibility of placing a queen reduces only linearly with each placement.

The space complexity is $O(n)$, which is used to store the state of the board (columns, diagonals) and the current solution of the recursion stack.

Python Implementation

```

def solveNQueens(n):
    def create_board(state):
        board = []
        for row in state:
            board.append(''.join('Q' if i == row else '.' for i in
                ↪ range(n)))
        return board

    def backtrack(row=0):
        if row == n:
            solutions.append(create_board(state))
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col)
                ↪ in diag2:
                continue
            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)
            state.append(col)

            backtrack(row + 1)

            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)
            state.pop()

    solutions = []
    state = []
    cols = set()
    diag1 = set()
    diag2 = set()
    backtrack()
    return solutions

```

Explanation

In the provided Python code, the 'solveNQueens' function searches for all valid solutions of the n-Queens problem using a backtracking algorithm. The 'backtrack' function is a recursive function that tries to place a queen in every column of the current row and recursively tries to fill in the next rows. It uses three sets, 'cols', 'diag1', and 'diag2', to keep track of the columns and diagonals that are already under attack by other queens, thus ensuring no two queens can attack each other.

Why this approach

The backtracking approach is chosen because it is a classic solution for constraint satisfaction problems like the n-Queens problem. It is straightforward to implement and handles the constraints effectively by pruning the search space.

Alternative approaches

An alternative approach could be using a heuristic search algorithm like the genetic algorithm or simulating annealing algorithm for larger boards, but these are more complex and don't guarantee finding all solutions.

Similar problems to this one

Similar problems include solving sudoku, crosswords, filling a grid with colors or numbers under certain constraints, and other combinatorial puzzles that can be solved with backtracking.

Things to keep in mind and tricks

When implementing backtracking algorithms: - It's crucial to carefully consider how to place and remove elements from the state (e.g., sets for columns and diagonals) to avoid illegal states. - Remember to create a deep copy of the solution before adding it to the solutions list if the state is mutable.

Corner and special cases to test when writing the code

Make sure to test: - The smallest case with $n = 1$, which has a single solution. - Cases where no solution exists (e.g., $n = 2$ or $n = 3$). - Large values of n , which can help identify inefficiencies in the algorithm.

Encode and Decode Strings

The "Encode and Decode Strings" problem is an exercise in serialization and deserialization, specifically geared towards string data. The challenge is to create two functions, one for encoding a list of strings into a single string and another for decoding this single string back into the original list of strings.

Problem Statement

The task is to design an algorithm that can take any list of strings and encode it into a single, flat string, and then reverse that process, ensuring that the original list of strings and the end result are identical.

The 'encode(strs)' function will take in a list of strings, 'strs', and output a single string that represents the encoded list in a retrievable manner.

The 'decode(s)' function will take in the single encoded string, 's', and reconstruct the original list of strings that was encoded.

Leet code link: [Encode and Decode Strings](#)

Algorithmic Approach

To meet the requirements of no predefined delimiters and handling all ASCII characters, a length-prefixed encoding scheme can be used. Each string is encoded as its length, followed by a special character (commonly, a non-digit character), and then the string itself. During decoding, the length is parsed first to determine where the actual string starts and ends.

Complexities

- **Time Complexity:** The time complexity of encoding is $O(N)$, where N is the total number of characters across all strings. The time complexity of decoding is also $O(N)$ since each character is processed exactly twice.
- **Space Complexity:** The space complexity is $O(N)$, which is necessary to store the encoded string or the list of decoded strings.

Python Implementation

Below is the complete Python code for ‘encode’ and ‘decode’ functions designed to handle a list of strings:

```
class Codec:

    def encode(self, strs):
        """Encodes a list of strings to a single string.

        :type strs: List[str]
        :rtype: str
        """

        # The encoded format is len(string)/string
        # This handles any string, including strings with
            ↪ problematic characters
        return ''.join(f'{len(s)}/{s}' for s in strs)

    def decode(self, s):
        """Decodes a single string to a list of strings.

        :type s: str
        :rtype: List[str]
        """

        i, n = 0, len(s)
        decoded = []

        while i < n:
            j = i
            # find the position of the separator '/'
            while s[j] != '/':
                j += 1
            # extract the length of the next string
            length = int(s[i:j])
            i = j + 1
            # extract the string using the length
            decoded.append(s[i:i + length])
            # move the index to the next part of the encoded
                ↪ string
            i += length

        return decoded

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))
```

This implementation encodes each string by first representing its length, followed by a separator character ('/'), and then the string itself. During decoding, the encoded string is processed by identifying the length and then the string, consecutively for all encoded parts.

Why this approach

This approach was chosen because it is simple, efficient, and works for all ASCII characters without assuming any specific character cannot be present in the strings. It requires no additional data structures and processes each character in a linear pass.

Alternative approaches

Alternative approaches could include using escape characters to handle special characters or another form of delimiter-based encoding. However, those methods would add complexity and potentially fail if delimiter characters appeared within the strings.

Similar problems to this one

Problems related to serialization and deserialization of more complex data structures, such as binary trees or graphs, share similarities with this problem. They require devising an encoding scheme that can be reliably reversed.

Things to keep in mind and tricks

Choosing an appropriate encoding scheme that guarantees no collisions with characters in the strings is crucial. This implies you should consider problematic characters during design and stress-test your solution with edge cases.

Corner and special cases to test when writing the code

Special cases to consider include empty lists

Combination Sum

The "Combination Sum" problem is a classic backtracking algorithm problem that requires finding all unique combinations in a set of candidate numbers that add up to a specific target number.

Problem Statement

Given an array of distinct integers `candidates` and a target integer `target`, return a list of all unique combinations of `candidates` where the chosen numbers sum to `target`. You may return the combinations in any order. You can choose the same number from `candidates` an unlimited number of times. The combinations should not include the same combination twice, and the order of numbers in each combination does not matter.

LeetCode link: [Combination Sum](#)

Algorithmic Approach

Approaching this problem involves using a backtracking algorithm, which is a form of recursion that explores all potential combinations of the candidates that can add up to the target. The algorithm keeps track of the current combination being formed, and once the sum of its elements equals the target, that combination is added to the result list. If at any point the sum exceeds the target, or if no more elements can be added, the algorithm backtracks to explore other potential combinations.

Complexities

- **Time Complexity:** The overall time complexity of the algorithm is $O(2^N)$ where N is the number of candidates. In the worst case, we might have to explore all possible combinations.
- **Space Complexity:** The space complexity is $O(N)$, which accounts for the space used by the recursion stack during the backtracking process. Note that this does not include the space used to store the result.

Python Implementation

The complete Python code to solve the "Combination Sum" problem using backtracking is as follows:

```
from typing import List

class Solution:
    def combinationSum(self, candidates: List[int], target: int) ->
        ↪ List[List[int]]:
        def backtrack(start, curr_sum, path):
            if curr_sum == target:
                result.append(list(path))
                return
            elif curr_sum > target:
                return

            for i in range(start, len(candidates)):
                path.append(candidates[i])
                backtrack(i, curr_sum + candidates[i], path)
                path.pop()

        result = []
        backtrack(0, 0, [])
        return result
```

Why this approach

This backtracking approach is chosen because it provides a systematic way to explore all potential combinations of candidates, ensuring that all unique combinations are considered. It is a depth-first search algorithm that can effectively handle the constraint that each number can be chosen multiple times and can avoid duplicates by taking care of the order.

Alternative approaches

An alternative approach could involve using dynamic programming to build up a table of all possible combinations up to the target, although this could be less efficient in terms of space complexity. Another possibility is to use an iterative approach with a queue or stack to simulate the backtracking process; however, this method can be more complex to implement.

Similar problems to this one

Similar problems that involve combination or subset generation with a specific constraint include "Subset Sum", "Permutations", and other problems that employ backtracking techniques, such as "N-Queens" or "Sudoku Solver".

Things to keep in mind and tricks

It's important to remember that with backtracking problems like this, you will need to make a copy of the current path when you add it to the result list, as the path list will be modified as you continue the exploration. Also, since the problem statement mentions that combinations should not contain duplicate sets, always consider using a starting index to avoid generating the same combination in different orders.

Corner and special cases to test when writing the code

Special cases to consider include when the target is zero (expect an empty set as output), when there is only one candidate, and when the candidate values are larger than the target. It's also crucial to test cases where candidates contain duplicates to ensure the solution set does not include redundant combinations.

Pacific Atlantic Water Flow

The "Pacific Atlantic Water Flow" problem is a graph theory challenge that involves finding matrix coordinates from where water can flow to both the Pacific and Atlantic oceans under specific conditions.

Problem Statement

Imagine a continent as an $m \times n$ matrix of non-negative integers where each cell represents the height of a unit of terrain. The Pacific Ocean borders the continent to the west (left edge) and north (top edge), while the Atlantic Ocean borders it to the east (right edge) and south (bottom edge). Water flows from a cell to any adjacent cell with equal or lower elevation.

The task is to identify all the cells from where water can reach both the Pacific and Atlantic oceans. These cells are at crossroads where water, if allowed to flow naturally according to the rules, would be able to reach both oceans.

Algorithmic Approach

To solve this, we can perform a search from the oceans inwards using either Depth-First Search (DFS) or Breadth-First Search (BFS), marking the cells that can be reached by water coming from each ocean. Finally, we can find the intersection of the cells reachable from both oceans.

Complexities

- **Time Complexity:** Let m be the number of rows and n be the number of columns in the matrix, the time complexity is $O(m \cdot n)$ for both DFS and BFS approaches, because each cell is processed once.
- **Space Complexity:** The space complexity is also $O(m \cdot n)$ as we need to store two separate matrices (or equivalent data structures) to keep track of reachable cells from both oceans.

Python Implementation

Below is the complete Python code for the water flow problem:

```
class Solution(object):
    def pacificAtlantic(self, matrix):
        if not matrix or not matrix[0]: return []

        def dfs(x, y, visited, prevHeight):
            if (x, y) in visited or \
                x < 0 or x >= len(matrix) or y < 0 or y >= len(
                    ↪ matrix[0]) or \
                matrix[x][y] < prevHeight:
                return
            visited.add((x, y))
            for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                dfs(x + dx, y + dy, visited, matrix[x][y])

        pacific = set()
        atlantic = set()

        for i in range(len(matrix)):
            dfs(i, 0, pacific, matrix[i][0])
            dfs(i, len(matrix[0]) - 1, atlantic, matrix[i][-1])
        for j in range(len(matrix[0])):
            dfs(0, j, pacific, matrix[0][j])
            dfs(len(matrix) - 1, j, atlantic, matrix[-1][j])

        return list(pacific & atlantic)

# You can call the function with any matrix input to test.
# For example:
# solution = Solution()
# matrix = [[1, 2, 2, 3, 5],
#           [3, 2, 3, 4, 4],
#           [2, 4, 5, 3, 1],
#           [6, 7, 1, 4, 5],
#           [5, 1, 1, 2, 4]]
# print(solution.pacificAtlantic(matrix))
```

This implementation uses Depth-First Search (DFS) from the edge cells that are adjacent to either the Pacific or the Atlantic Ocean and recursively marks cells that are reachable, tracking the visited cells in a set to avoid duplicate processing.

Why this approach

DFS is a suitable approach for this problem because it allows us to traverse the matrix from the edges inward efficiently, marking cells reachable from each ocean. Since we need to find cells that can reach both oceans, using DFS from the border cells (with an ocean boundary) and marking those cells that are reached ensures

that we only consider valid paths where water can flow.

Alternative approaches

Alternatively, a Breadth-First Search (BFS) could also be used to achieve similar results. One could also optimize the space by using a single visited matrix if a careful update of state is performed to distinguish between cells visited by either ocean, both oceans, or none.

Similar problems to this one

Similar problems that involve traversal or pathfinding in a grid or matrix environment include "Number of Islands", "Walls and Gates", and "Longest Increasing Path in a Matrix". Each of these problems has a unique set of rules for traversal or conditions for path validity.

Things to keep in mind and tricks

Understanding how to modify and apply DFS and BFS to various types of problems is critical. Additionally, effectively using data structures such as sets and matrices to keep track of visited elements is vital.

Corner and special cases to test when writing the code

Consider boundary conditions such as when the matrix is empty or when there is no elevation difference between the cells, in which cases the program should function correctly. Testing the code with diverse and extreme scenarios is key to uncover failures and ensure robustness.

Linked List Problems

Remove Nth Node From End of List

The "Remove Nth Node From End of List" problem requires modifying a linked list by removing a specific node located a certain distance from the end.

Problem Statement

Given the head of a linked list, the task is to remove the n -th node from the end of the list and return its head.

Examples:

Example 1:

Input: head = [1,2,3,4,5], n = 2

Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1

Output: []

Example 3:

Input: head = [1,2], n = 1

Output: [1]

LeetCode link: [Remove Nth Node From End of List](#)

Algorithmic Approach

The general approach for solving this problem is to use the two-pointer technique. Here is the step-by-step process:

1. Initialize two pointers: 'fast' and 'slow'. Both start at the head of the list.
2. Advance the 'fast' pointer by 'n' nodes in the list.
3. Move the 'fast' and the 'slow' pointers simultaneously until 'fast' reaches the last node. 'slow' will then point to the node before the target node.
4. Modify the 'next' pointer of the node referenced by 'slow' to skip the target node.

5. Deal with the edge case where the node to be removed is the head itself, by checking if the 'fast' pointer directly reached the end after initialization.
6. Return the altered head of the linked list.

Complexities

- **Time Complexity:** The time complexity of this algorithm is $O(L)$ where L is the length of the linked list, as we traverse the list at most twice.
- **Space Complexity:** The space complexity is $O(1)$ because we only use two extra pointers, so it does not scale with the size of the input list.

Python Implementation

Here is the Python code that implements the aforementioned approach:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        # Initialize two pointers
        fast = slow = head
        # Advance fast by n nodes
        for _ in range(n):
            fast = fast.next

        # Edge case: removing the first node
        if not fast:
            return head.next

        # Advance both pointers until fast reaches the last node
        while fast.next:
            fast = fast.next
            slow = slow.next

        # Skip the target node
        slow.next = slow.next.next
        return head
```

This implementation first moves the ‘fast’ pointer ‘n’ nodes ahead...

Why this approach

The two-pointer technique is chosen for this problem as it allows removing the n -th node from the end in one pass. Specifically, the approach eliminates the need to compute the length of the list which would require an extra pass. This technique is efficient, with optimal time and space complexities.

Alternative approaches

Alternative approaches might include calculating the length of the linked list first and then removing the $L - n + 1$ -th node from the start. However, this requires two passes over the list, which is less efficient.

Similar problems to this one

Similar problems involving linked list manipulations with various constraints, such as reversing a linked list, detecting cycles, or finding the middle of a linked list, also use the two-pointer technique.

Things to keep in mind and tricks

Key points to remember when solving similar linked list problems include careful manipulation of pointers to avoid breaking the list, and consideration of edge cases, such as lists with a single node or when the head of the list needs to be removed.

Corner and special cases to test when writing the code

When implementing this algorithm, it is crucial to test edge cases such as small lists (e.g., with 1 or 2 elements), and scenarios where the first or the last element needs to be removed. Additionally, tests should ensure that the linked list remains correctly linked after the operation with no memory leaks.

Linked List Cycle

The "Linked List Cycle" problem is a common challenge that involves determining if a singly linked list contains a cycle—a situation where a node's 'next' pointer points to an earlier node, leading to an infinite loop when traversing the list. This problem is important in the field of computer science because cycles in linked lists can lead to bugs and inefficiencies in software applications. To solve this problem, an effective algorithm needs to be formulated that can detect the presence of a cycle without traversing the entire list repeatedly, which could cause an infinite loop if a cycle is present.

Problem Statement

To determine if a cycle is present in a given linked list, the typical strategy is to use indicators or "pointers" to traverse the list. One widely used method to detect cycles is Floyd's Cycle Detection Algorithm, also known as the Tortoise and Hare algorithm. In this approach, two pointers are initialized at the head of the linked list: a slow pointer that moves to the next node in each step, and a fast pointer that moves two steps at a time. If there is a cycle in the list, the fast pointer will loop around and eventually meet the slow pointer within the cycle. If there is no cycle, the fast pointer will reach the end of the list without meeting the slow pointer.

The function signature for this problem, in Python, is as follows:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        # Implementation goes here
```

The follow-up challenge is to solve this problem with $O(1)$ memory usage, which precludes storing node references in a hash table.

LeetCode link: <https://leetcode.com/problems/linked-list-cycle/>

Algorithmic Approach

Floyd's Cycle Detection Algorithm is the optimal solution to this problem because it requires only two pointers, satisfying the $O(1)$ memory constraint of the follow-up challenge. The implementation involves iterating through the list with the slow and fast pointers as described, checking for the condition where the fast pointer equals the slow pointer, which indicates a cycle. If the fast pointer reaches the end of the list (i.e., encounters a 'None' reference), the list does not contain a cycle and the function returns 'false'.

Complexities

- **Time Complexity:** The time complexity of Floyd's algorithm is $O(n)$, where n is the number of nodes in the linked list. In the worst case, this is the time it takes for the fast pointer to meet the slow pointer.
- **Space Complexity:** The algorithm achieves $O(1)$ space complexity since it only uses two pointers regardless of the size of the linked list.

Python Implementation

Below is the complete Python code for the 'Solution' class, which implements the 'hasCycle' function using Floyd's Cycle Detection Algorithm to determine whether a linked list has a cycle:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        if not head:
            return False

        slow = head
        fast = head.next

        while fast and fast.next:
            if slow == fast:
                return True
            slow = slow.next
            fast = fast.next.next

        return False
```

The code first checks for an empty list, which cannot have a cycle. It then initializes the 'slow' and 'fast' pointers and begins the iteration. If at any point, 'slow' is equal to 'fast', a cycle is detected, and the function returns 'true'. If 'fast' or 'fast.next' becomes 'None', meaning the end of the list is reached, the function returns 'false', indicating no cycle.

Why this approach

Floyd's Cycle Detection Algorithm is chosen for its efficiency in time and memory usage. It does not require additional data structures and thus adheres to the constant space complexity constraint. This algorithm is also intuitively straightforward and reliable for detecting cycles in linked lists.

Alternative approaches

An alternative approach would be to use a hash set to keep track of visited nodes. Each node visited by the traversal would be added to the

Reverse Linked List

The "Reverse Linked List" problem is a fundamental challenge often used to assess one's understanding of linked list manipulation and pointer operations. Given the head of a singly linked list, the objective is to reverse the list and return the head of the newly reversed list.

Problem Statement

The task at hand involves modifying the direction of the 'next' pointers in each node of a singly linked list such that the list's order is reversed. The input is the head of a singly linked list, and the expected output is the head of the list after it has undergone reversal.

LeetCode link: [Reverse Linked List](#)

Algorithmic Approach

A common strategy to reverse a linked list is to iterate through the list while adjusting the next pointers of each node to point to the previous node instead of the next one. This can be done using an iterative method with a temporary node to help in swapping the pointers, or using a recursive approach that reverses the list from the tail back to the head.

Complexities

- **Time Complexity:** The iterative and recursive solutions both traverse the list once, resulting in a time complexity of $O(n)$, where n is the number of nodes in the list.
- **Space Complexity:** The iterative approach uses constant extra space, giving it a space complexity of $O(1)$. The recursive approach, however, has a space complexity of $O(n)$ due to the call stack memory used in the recursion.

Python Implementation

Here is a Python implementation of the iterative method for reversing a linked list:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseList(self, head):
        prev = None
        current = head
        while current:
            next_temp = current.next
            current.next = prev
            prev = current
            current = next_temp
        return prev
```

This code defines a 'ListNode' class used to construct the linked list, as well as a 'Solution' class with the 'reverseList' method. The method takes the list's head node as an argument and reverses the list by reassigning the 'next' pointers of each node.

Why this approach

The iterative method is selected because it is straightforward, elegant, and requires only a few lines of code. It does not use additional space proportional to the size of the list, which makes it space efficient.

Alternative approaches

An alternative is the recursive approach, which is somewhat less intuitive but useful in understanding deeper recursive algorithmic thinking. The primary downside is the extra space required for the recursion call stack.

Similar problems to this one

Similar linked list manipulation problems include "Reverse Linked List II", where only a part of the list is reversed, and "Reverse Nodes in k-Group", where the list is reversed in groups of k nodes.

Things to keep in mind and tricks

When reversing a linked list, it's important to keep track of the next pointer of the current node before changing it, so that you do not lose access to the rest of

the list. Using multiple pointers (like `prev`, `current`, and `next_temp` in the implementation) is a common trick to simplify the process.

Corner and special cases to test when writing the code

Be sure to test cases where the list is empty or contains only one node, as these are edge cases that may cause incorrect implementations to fail. Additionally, one must ensure that the last node in the reversed list points to 'NULL' (or 'None' in Python), indicating the end of the list.

Reorder List

The "Reorder List" problem is a classic example of linked list manipulation. Given a singly linked list $L : L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$, the goal is to reorder it to follow a specific pattern $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$. This reordering must be done in-place without altering the nodes' values.

Problem Statement

The task is to write a function that reorders a singly linked list in such a way that the first element is followed by the last element, followed by the second element, and so on. The reordering should be done in place with no modification to the node values.

Examples:

Example 1:

Input: 1 → 2 → 3 → 4

Output: 1 → 4 → 2 → 3

Example 2:

Input: 1 → 2 → 3 → 4 → 5

Output: 1 → 5 → 2 → 4 → 3

The direct link to the problem: [Reorder List on LeetCode](#).

Algorithmic Approach

To solve the "Reorder List" problem, we follow these general steps:

1. Find the middle point of the linked list by using the fast and slow pointer method.
2. Reverse the second half of the list starting from the node right after the middle node found in step 1.
3. Reorder the list by alternating the nodes from the first part and the reversed second half.

Complexities

- **Time Complexity:** The total time complexity of the solution is $O(n)$, as we sequentially traverse the linked list a constant number of times.
- **Space Complexity:** The space complexity is $O(1)$, as we reorder the list in-place without using additional space proportional to the input size.

Python Implementation

Below is the complete Python code for the `Solution` class, which implements the `reorderList` method to reorder a singly linked list as specified:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reorderList(self, head):
        if not head:
            return

        # Step 1: Find the middle of the linked list
        slow, fast = head, head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # Step 2: Reverse the second half
        prev, curr = None, slow
        while curr:
            curr.next, prev, curr = prev, curr, curr.next

        # Step 3: Merge the two halves
        first, second = head, prev
        while second.next:
            first.next, first = second, first.next
            second.next, second = first, second.next
```

This implementation follows a three-step approach to reorder the singly linked list.

Why this approach

The chosen approach is efficient as it only requires linear time complexity and constant space complexity. It does the reordering in-place, saving space, and the simplicity of the algorithm makes it an elegant and practical solution.

Alternative Approaches

An alternative approach might be to use additional data structures like arrays or stacks to rearrange the elements, but this would require extra space, making it less optimal than the in-place solution.

Similar Problems

Similar linked list manipulation problems can be found on LeetCode, such as "Palindrome Linked List," "Reverse Linked List II," and "Split Linked List in Parts," all of which involve breaking down the list, modifying portions, and re-assembling the list in some way.

Things to Keep in Mind and Tricks

When working with linked lists, careful handling of node pointers is crucial to avoid losing track of nodes, which can lead to memory leaks or incorrect results. In addition, testing with edge cases like a linked list with a single node or an empty linked list is important to ensure robustness.

Corner and Special Cases to Test

- A linked list with a single node.
- An empty linked list.
- A linked list with an even number of nodes.
- A linked list with an odd number of nodes.

Reorder List

Problem Statement

Given a singly linked list, the task is to reorder it in a specific pattern such that the first element is followed by the last, the second by the second last, and so on. The reordering should be done in-place with the goal of achieving the new sequence without altering the node values.

LeetCode link: [Reorder List](#)

Algorithmic Approach

To achieve the reordering, we can follow these steps:

1. Find the middle of the linked list using the slow and fast pointer approach (Tortoise and Hare algorithm).
2. Reverse the second part of the list after the middle.
3. Merge the two parts of the list by alternating nodes from each part.

Complexities

- **Time Complexity:** The algorithm consists of three main operations which all run in linear time. Finding the middle requires $O(n)$ time. Reversing the second half of the list also takes $O(n)$ and the final merge operation is $O(n)$. Therefore, the total time complexity is $O(n)$.
- **Space Complexity:** Since the algorithm uses a constant amount of extra space and works in-place, the space complexity is $O(1)$.

Python Implementation

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reorderList(head):
    if not head:
        return

    # Step 1: Find the middle of the list
    slow, fast = head, head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    # Step 2: Reverse the second half
    prev, curr = None, slow
    while curr:
        next_temp = curr.next
        curr.next = prev
        prev = curr
        curr = next_temp

    # Step 3: Merge the two halves
    first, second = head, prev
    while second.next:
        first.next, second.next = first.next, second.next
        first.next = second
        second.next = first.next
        first = first.next
        second = second.next

```

Explanation

The provided Python code demonstrates a function that reorders a linked list according to the given pattern. It defines a 'ListNode' class for the linked-list node and then implements the 'reorderList' function.

Why this approach

This approach offers an optimal solution in terms of time and space complexity. It cleverly utilizes the process of finding the middle, reversing the second half, and then merging to reorder the list without the need for additional data structures.

Alternative approaches

An alternative brute-force approach could involve using an array to store the nodes of the list and then reconstructing the list in the reordered pattern. However, this approach would not satisfy the space complexity requirement of using only constant extra space.

Similar problems to this one

Similar problems that involve manipulating linked lists are "Reverse Linked List", "Palindrome Linked List", and "Split Linked List in Parts". These problems require understanding of linked list operations like reversal, merge, and traversal.

Things to keep in mind and tricks

Key points to remember:

- Handling edge cases where the linked list is short.
- Careful manipulation of node pointers to avoid losing track of next nodes and causing a cycle in the list.
- Ensuring that the last node of the newly ordered list points to 'None' to mark the list's end.

Corner and special cases to test when writing the code

Test cases should include:

- A list with only one node (no reordering needed).
- A list with two nodes (simple swap).
- A list with an odd number of nodes (the middle node remains in its original position).
- Very long lists to ensure the solution handles large input efficiently.

Merge Two Sorted Lists

The "Merge Two Sorted Lists" problem requires combining two sorted linked lists into a single, sorted linked list. The new list should be constructed by splicing together the nodes of the first two lists in sorted order.

Problem Statement

Given the heads of two sorted linked lists 'list1' and 'list2', the goal is to return the head of the merged linked list that maintains the sorted order. The linked list should be assembled by sequentially connecting nodes from the input lists to maintain overall sort order.

LeetCode link: <https://leetcode.com/problems/merge-two-sorted-lists/>

Algorithmic Approach

A common technique for merging two sorted lists involves using pointers to traverse each input list. As we move through the lists, we compare the current nodes and attach the node with the smaller value to the end of the merged list. We continue this process until all nodes from both lists are merged. If one list is exhausted before the other, the remaining nodes of the unexhausted list are appended to the merged list.

Complexities

- **Time Complexity:** The time complexity of the algorithm is $O(n + m)$, where n and m are the lengths of 'list1' and 'list2', respectively, since each node in both lists is visited at most once.
- **Space Complexity:** The space complexity is $O(1)$ because the merge is performed in-place without allocating any additional data structures for the new list.

Python Implementation

Below is the complete Python code for merging two sorted linked lists:

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def mergeTwoLists(self, list1: ListNode, list2: ListNode) ->
        ↪ ListNode:
        # Create a dummy node to serve as the starting point of the
        ↪ merged list
        dummy = ListNode()
        current = dummy

        # Loop until either list1 or list2 runs out of elements
        while list1 and list2:
            if list1.val < list2.val:
                current.next = list1
                list1 = list1.next
            else:
                current.next = list2
                list2 = list2.next
            current = current.next

        # Attach the remaining elements from list1 or list2, if any
        current.next = list1 if list1 else list2

        # Return the head of the merged list, which is the next
        ↪ node after dummy
        return dummy.next
```

The above code uses a dummy node as a placeholder to simplify edge cases where either of the input lists might be empty. Since the dummy node acts as the pre-head of the merged list, we can consistently append the next node without checking if the merged list is empty.

Why this approach

This approach is chosen because it is intuitive, easy to implement, and has optimal time and space complexities. Merging the lists in place without extra space allocation aligns with the constraints of the problem, making it an efficient solution.

Alternative approaches

An alternative method could involve recursively merging the two lists. In the recursive approach, the smaller head node is connected to the result of the recursive

call, which merges the rest of the lists.

Similar problems to this one

Similar problems include merging k sorted lists, sorting linked lists, and handling variants of merged intervals.

Things to keep in mind and tricks

Keep in mind to handle the edge cases where one or both input lists may be empty. By using a dummy node, we avoid multiple null checks, simplifying the code.

Corner and special cases to test when writing the code

It is essential to test the cases when one list is significantly longer than the other, when one or both lists are empty, and when there are duplicate values in the lists that must be merged. Additionally, verify that the merged list maintains the sorted order and does not contain any unexpected null references.

Add Two Numbers

The "Add Two Numbers" problem is a classic example of manipulating linked lists in an algorithmic context. It combines the simplicity of arithmetic addition with the complexity of data structure traversal.

Problem Statement

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:

Input: $l1 = [2,4,3]$, $l2 = [5,6,4]$

Output: $[7,0,8]$

Explanation: $342 + 465 = 807$.

Example 2:

Input: $l1 = [0]$, $l2 = [0]$

Output: $[0]$

Example 3:

Input: $l1 = [9,9,9,9,9,9,9]$, $l2 = [9,9,9,9]$

Output: $[8,9,9,9,0,0,0,1]$

LeetCode link: [Add Two Numbers](#)

Algorithmic Approach

The solution to this problem is straight-forward and simulates the addition process you would perform by hand...

Complexities

- **Time Complexity:** The time complexity is $O(\max(n, m))$...

Python Implementation

Below is the complete Python code implementation for the ‘Solution’ class to add two numbers represented by linked lists:

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) ->
        ↪ ListNode:
        dummy = ListNode()
        current = dummy
        carry = 0

        while l1 or l2 or carry:
            val1 = (l1.val if l1 else 0)
            val2 = (l2.val if l2 else 0)
            carry, out = divmod(val1 + val2 + carry, 10)

            current.next = ListNode(out)
            current = current.next

            l1 = (l1.next if l1 else None)
            l2 = (l2.next if l2 else None)

        return dummy.next
```

This function starts by creating a dummy node, which simplifies the handling of edge cases...

Why this approach

The approach chosen here...

Alternative approaches

An alternative approach could involve...

Similar problems to this one

Similar problems often deal with...

Tricks in solving linked list problems

When dealing with linked list problems, it's often useful...

Corner and special cases to test when implementing

Special cases that you should keep in mind include scenarios such as...

Tree and Graph Problems

100. Same Tree

Problem Statement

This problem involves determining whether two binary trees are identical. Two binary trees are considered the same if they are structurally identical and all corresponding nodes have the same value.

The problem as described on LeetCode can be found at the following link: [Same Tree](#).

Algorithmic Approach

To solve this problem, a recursive approach is typically used. The algorithm proceeds as follows:

1. Check if both trees are empty (NULL pointers). If they are, return *true* since two empty trees are considered the same.
2. If one tree is empty and the other is not, return *false* because they cannot be the same.
3. Compare the values of the current nodes from both trees. If they do not match, return *false*.
4. Recursively check the left subtrees of both trees to see if they are the same.
5. Recursively check the right subtrees of both trees to see if they are the same.
6. If the values of the current nodes match and the recursive checks for both subtrees return *true*, then the current trees are the same; hence, return *true*.
7. Otherwise, return *false*.

Complexities

- Time Complexity: The time complexity is $O(n)$, where n is the number of nodes in the tree. This is because the algorithm must visit each node exactly once.

- Space Complexity: The space complexity is $O(h)$, where h is the height of the tree. This space is used in the call stack during recursive calls. In the worst case, the tree can be completely unbalanced, yielding a space complexity of $O(n)$.

Python Implementation

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def isSameTree(p: TreeNode, q: TreeNode) -> bool:
    # Both trees are empty
    if not p and not q:
        return True
    # One tree is empty, and the other is not
    if not p or not q:
        return False
    # Node values are different
    if p.val != q.val:
        return False
    # Recursively check the left and right subtrees
    return isSameTree(p.left, q.left) and isSameTree(p.right, q.
    ↪ right)

```

Explanation

The function `isSameTree` determines whether two binary trees, `p` and `q`, are the same by recursively comparing their structure and node values. It returns *true* only if both trees are structurally identical and all corresponding nodes have the same value.

Why this approach

The recursive approach is natural for tree problems because it mimics the structure of a tree, allowing us to perform the same operation on subtrees as we would on a full tree. It is elegant and leads to a simple yet effective solution.

Alternative approaches

An alternative method would be to use an iterative approach with a queue or stack to perform a level-order or depth-first traversal, respectively. At each step, you would compare the nodes from both trees. However, the recursive approach is more straightforward in this case.

Similar problems to this one

Similar tree comparison problems include checking if a tree is symmetric (*Symmetric Tree*), determining if a tree is a subtree of another tree (*Subtree of Another Tree*), and checking if two trees are mirror images of each other (*Invert Binary Tree*).

Things to keep in mind and tricks

- Always check for NULL pointers when dealing with trees to avoid dereferencing them. - If the tree nodes have unique values, a pre-order or post-order traversal would suffice to determine tree equality. If not, a combination of in-order and pre/post-order traversal can be used.

Corner and special cases to test when writing the code

Some corner and special cases to consider: - One or both trees are empty. - Trees have only one node. - Trees with a different structure but the same values in a different arrangement. - Trees where one is a complete binary tree and the other is not.

Graph Valid Tree

The "Graph Valid Tree" problem is a well-known question in computer science and competitive programming, focusing on determining whether a given graph constitutes a valid tree.

Problem Statement

Given n nodes labeled from 0 to $n - 1$ and a list of undirected edges (each edge is a pair of nodes), write a function to check whether these edges make up a valid tree.

Inputs:

- n : an integer representing the total number of nodes in the graph.
- $edges$: a list of pairs of integers that represent the undirected edges between nodes.

Output:

- Return *true* if the given $edges$ constitute a valid tree, and *false* otherwise.

Examples:

Example 1:

Input: $n = 5$, $edges = [[0,1], [0,2], [0,3], [1,4]]$

Output: *true*

Example 2:

Input: $n = 5$, $edges = [[0,1], [1,2], [2,3], [1,3], [1,4]]$

Output: *false*

LeetCode link: [Graph Valid Tree](#)

Algorithmic Approach

To solve this problem, one might use various graph algorithms like depth-first search (DFS), breadth-first search (BFS), or union-find to check if the given graph is connected and does not contain any cycles. These are the two key properties that define a tree.

Complexities

- **Time Complexity:** The time complexity of the solution will depend on the graph traversal algorithm chosen. For instance, using a union-find algorithm would provide near constant-time queries and $O(N \log N)$ complexity for union operations.
- **Space Complexity:** The space complexity for storing the graph is $O(N)$, where N is the number of nodes.

Python Implementation

Below is the complete Python code for checking if the given edges make up a valid tree:

```
class Solution:
    def validTree(self, n, edges):
        parent = list(range(n))
        rank = [0] * n

        def find(x):
            if parent[x] != x:
                parent[x] = find(parent[x])
            return parent[x]

        def union(x, y):
            xroot = find(x)
            yroot = find(y)
            if xroot == yroot:
                return False
            if rank[xroot] < rank[yroot]:
                parent[xroot] = yroot
            elif rank[xroot] > rank[yroot]:
                parent[yroot] = xroot
            else:
                parent[yroot] = xroot
                rank[xroot] += 1
            return True

        for edge in edges:
            if not union(edge[0], edge[1]):
                return False

        roots = set(find(x) for x in range(n))
        return len(edges) == n - 1 and len(roots) == 1
```

This implementation uses the union-find algorithm to detect cycles...

Why this approach

The union-find algorithm is an efficient method...

Alternative approaches

An alternative approach to solving the "Graph Valid Tree" ...

Similar problems to this one

Similar problems that involve checking...

Things to keep in mind and tricks

A trick to improve the efficiency...

Corner and special cases to test when writing the code

When writing the code for the "Graph Valid Tree" problem...

Number of Connected Components in an Undirected Graph

The "Number of Connected Components in an Undirected Graph" problem involves finding the number of connected components in a given undirected graph considering that each node in the graph is labeled from 0 to $n - 1$.

Problem Statement

Given n nodes and a list of undirected edges, we need to count how many connected components exist. Each edge is represented by a pair of nodes, defining the connections in the graph.

Algorithmic Approach

For this problem, one can utilize Depth-First Search (DFS), Breadth-First Search (BFS), or the Union-Find algorithm. These methods allow us to traverse the graph and group nodes into isolated connected components.

Complexities

Depending on the algorithm used, the time and space complexities may vary:

– **Time Complexity:**

- * For DFS or BFS: $O(V + E)$, where V is the number of vertices and E is the number of edges.
- * For Union-Find: $O(E \log V)$ in the average case with path compression.

– **Space Complexity:**

- * For DFS or BFS: $O(V)$, which is used for storing visited information and the recursion stack or queue.
- * For Union-Find: $O(V)$, which is needed for storing the parent and rank of each node.

Python Implementation

Below is the complete Python code using the Union-Find algorithm with path compression for finding the number of connected components in an undirected graph:

```
class Solution(object):
    def countComponents(self, n, edges):
        # Initialize parent and rank.
        parent = [i for i in range(n)]
        rank = [1] * n

        # Find function with path compression.
        def find(v):
            if parent[v] != v:
                parent[v] = find(parent[v])
            return parent[v]

        # Union function with union by rank.
        def union(v1, v2):
            root1, root2 = find(v1), find(v2)
            if root1 != root2:
                if rank[root1] > rank[root2]:
                    parent[root2] = root1
                    rank[root1] += rank[root2]
                else:
                    parent[root1] = root2
                    if rank[root1] == rank[root2]:
                        rank[root2] += rank[root1]

        # Process all edges and perform union operations.
        for u, v in edges:
            union(u, v)

        # Count distinct roots, which represent separate
        ↪ components.
        return sum(parent[i] == i for i in range(n))

# Example usage:
solution = Solution()
print(solution.countComponents(5, [[0, 1], [1, 2], [3, 4]])) #
    ↪ Output: 2
```

This implementation uses the Union-Find algorithm with two optimizations - path compression and union by rank. Path compression ensures that we flatten the tree structure as we perform find operations, reducing the time complexity. Union by rank guarantees that we always attach the tree with the smaller rank to the root of the tree with the larger rank, preventing skewed trees and thereby maintaining a balanced tree to minimize the time complexity of subsequent find operations.

Why this approach

The Union-Find algorithm is particularly suited for these kinds of connectivity problems because it provides a way to efficiently query and merge components. While DFS and BFS are equally valid, Union-Find tends to be more efficient in scenarios where there are many merge operations.

Alternative approaches

As mentioned, DFS or BFS can be used to solve this problem. Both these approaches involve visiting each vertex and all its edges, marking the visited vertices, and incrementing our count of connected components whenever starting a traversal at an unvisited vertex.

Similar problems to this one

This problem is similar to classic Union-Find problems like "Redundant Connection," "Number of Islands," and various other problems where the connectivity within a structure is evaluated.

Things to keep in mind and tricks

Always consider using path compression and union by rank when using the Union-Find algorithm to achieve the optimal time complexity.

Corner and special cases to test when writing the code

It's important to cover cases such as isolated nodes (no edges), the graph being a fully connected component (essentially one large component), and an empty graph (no nodes). Additionally, multiple edges connecting the same pair of nodes can occur and should not affect the component count.

Clone Graph

The "Clone Graph" problem is centered around the concept of creating an exact replica of a given undirected graph. A graph is comprised of nodes connected to each other by undirected edges. The objective is to produce a new graph where the structure—an arrangement of nodes and edges—of the original graph is identically maintained.

Problem Statement

The task involves taking a reference to a node in a connected undirected graph and returning a deep copy (clone) of the entire graph. Every node within the graph encloses a value ('val') along with a list ('neighbors') indicating the adjacent nodes.

```
class Node {  
    public int val;  
    public List<Node> neighbors;  
}
```

You are required to write a function that receives a node from a graph as input and returns a comprehensive copy of the graph. The duplication must preserve the structure of the original graph perfectly. You must ensure not to alter the original graph. Every node in the created graph must hold the same value as its counterpart in the original graph, and all connecting edges should be cloned to link the corresponding nodes in the copy.

Addressing this problem effectively requires one to manage potential cycles within the graph, preventing a straightforward recursive copy. Utilizing a hash table or a dictionary is a common strategy to monitor already cloned nodes during the copying process.

LeetCode link: [Clone Graph](#)

Algorithmic Approach

Creating a clone of a graph typically involves a depth-first search or breadth-first search traversal while keeping track of copied nodes to avoid infinite loops

caused by cycles. A commonly used approach is to maintain a hash map where each original node's reference is mapped to its corresponding cloned node.

Complexities

- **Time Complexity:** The time complexity of cloning a graph is $O(N)$, where N is the number of nodes in the graph.
- **Space Complexity:** The space complexity is also $O(N)$, considering the space required for the hash map to store references to cloned nodes.

Python Implementation

The following is a complete Python code for the ‘Solution’ class, implementing the ‘cloneGraph’ method to clone a given undirected graph:

```

from collections import deque

class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None
        ↪ else []

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return node

        # Using a dictionary to keep track of copied nodes
        cloned_nodes = {node: Node(node.val)}

        # Use a queue for BFS
        queue = deque([node])

        while queue:
            current = queue.popleft()

            for neighbor in current.neighbors:
                if neighbor not in cloned_nodes:
                    # Clone the neighbor and add it to the
                    ↪ queue
                    cloned_nodes[neighbor] = Node(neighbor.val)
                    queue.append(neighbor)

                # Add the clone of the neighbor to the
                ↪ neighbors of the clone node
                cloned_nodes[current].neighbors.append(
                    ↪ cloned_nodes[neighbor])

        return cloned_nodes[node]

```

This implementation uses a breadth-first search strategy starting from the given node. It maintains a dictionary `cloned_nodes` to map original nodes to their clones. For each node, it copies the node’s value upon the first encounter and then iterates over its neighbors. If a neighbor has not been cloned yet, it is cloned and added to the queue. Finally, the cloned neighbor is linked to the current node’s clone.

Why this approach

The breadth-first search (BFS) approach is chosen for its natural fit in graph traversal scenarios. It facilitates copying and linking nodes level by level, effectively managing the graph's breadth while ensuring that all nodes are visited. Using a hash map to track visited nodes solves the problem of possible cycles and prevents redundant copying.

Alternative approaches

An alternative to the BFS approach is using depth-first search (DFS), where a recursive strategy can be employed to traverse and clone the graph. DFS might lead to a more concise implementation, but attention must be paid to maintain the hash map for tracking cloned nodes.

Similar problems to this one

Similar problems might include traversing and modifying the graph, such as computing the shortest path, determining if a graph is bipartite, or detecting cycles in a graph.

Things to keep in mind and tricks

- Always keep track of the nodes that have already been cloned to avoid duplicating efforts and to prevent infinite recursion due to cycles. - It's crucial to decide between DFS and BFS, as the choice affects the implementation and may influence the performance based on the graph's structure.

Corner and special cases to test when writing the code

- A graph with only one node and no edges. - A graph where one node is connected to all other nodes, forming a star topology. - A graph with a long chain of nodes (linear topology). - A graph with complex cycles and multiple connections between nodes.

Binary Tree Maximum Path Sum

Given a binary tree, we are tasked with finding the maximum path sum within it. The path can start and end at any node in the tree, must follow the parent-child connections, and must contain at least one node.

Problem Statement

Leet code link: [Binary Tree Maximum Path Sum](#)

Algorithmic Approach

To solve this problem, we can use a recursive depth-first search (DFS) algorithm. The recursive function will compute two things for each node: the maximum path sum that includes the current node and extends to one side (either left or right), and the maximum path sum that can be formed using the current node as the highest point (turning point) in a path.

Complexities

- Time Complexity: The time complexity of the algorithm is $O(N)$, where N is the number of nodes in the tree. This is because we visit each node exactly once during our recursive traversal.
- Space Complexity: The space complexity is $O(H)$, where H is the height of the tree, accounting for the recursive call stack. In the worst case (a skewed tree), the height can be $O(N)$.

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxPathSum(self, root: TreeNode) -> int:
        def max_gain(node):
            nonlocal max_path_sum
            if not node:
                return 0

            # Recursive call on children
            left_gain = max(max_gain(node.left), 0)
            right_gain = max(max_gain(node.right), 0)

            # Path sum with the current node as the highest point
            price_newpath = node.val + left_gain + right_gain

            # Update the global maximum path sum if the new path
            #   ↳ is better
            max_path_sum = max(max_path_sum, price_newpath)

            # Return the maximum gain the current node can
            #   ↳ contribute to the path
            return node.val + max(left_gain, right_gain)

        max_path_sum = float('-inf')
        max_gain(root)
        return max_path_sum

```

Explanation

The code defines a recursive function `max_gain` which calculates two values:

1. The maximum gain from either the left or right subtrees when including the current node (`left_gain` and `right_gain`).
2. The value of a potential path using the current node as the turning point (`price_newpath`).

The global variable `max_path_sum` stores the highest path sum seen so far. At each node, it is updated if the sum of the node's value and the gain from its left and right children surpasses the current maximum.

Why this approach

This recursive DFS approach is chosen because it efficiently traverses through each node once while keeping track of the maximum path sum that can be achieved. It implicitly handles various cases such as negative node values by using a 'max' function with 0 to ignore paths with negative sums.

Alternative approaches

An alternative approach could involve dynamic programming where we calculate the maximum path sum for each subtree and use those results to compute the maximum for the whole tree. However, this typically is more complex and does not offer a better time complexity than the recursive DFS.

Similar problems to this one

Similar problems include finding the diameter of a binary tree or the maximum depth of a binary tree. These also typically use depth-first searches with some modification to the recurrence relation.

Things to keep in mind and tricks

A key trick in this problem is to consider that paths contributing negative sums to the overall path sum can be ignored (hence the 'max(gain, 0)' in the recursion). This simplifies the logic required for handling negative node values.

Corner and special cases to test when writing the code

It's crucial to consider cases where all or most node values are negative, as these serve as edge cases for the algorithm. Additionally, one should test the algorithm on single-node trees and skewed trees (which impact space complexity due to recursion depth).

Validate Binary Search Tree

Problem Statement

Given a binary tree, the task is to determine if it is a valid binary search tree (BST). A binary search tree is a rooted binary tree, where each node contains a key (value), and satisfies the following properties:

1. The left subtree of a node contains only nodes with keys less than the node's key.
2. The right subtree of a node contains only nodes with keys greater than the node's key.
3. Both the left and right subtrees must also be binary search trees.

LeetCode link: [Validate Binary Search Tree](#)

Algorithmic Approach

To determine if a binary tree is a valid BST, an in-depth traversal of the tree is performed. During this traversal, we maintain the range (min, max) for each node's value based on the properties of BST. For the root node, the range is initially infinite ($-\infty, \infty$). As we traverse to the left, the maximum of the range becomes the value of the current node, and as we traverse to the right, the minimum becomes the value of the current node. If at any node, the value does not fit within the range, the tree is not a valid BST.

Complexities

The time complexity of this algorithm is $O(n)$, where n is the number of nodes in the binary tree. This is because each node is visited exactly once. The space complexity is $O(h)$, where h is the height of the tree, due to the recursion stack during the depth-first traversal.

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def isValidBST(root):
    def validate(node, low=float('-inf'), high=float('inf')):
        # An empty tree is a valid BST
        if not node:
            return True
        # The current node's value must be between low and high.
        if node.val <= low or node.val >= high:
            return False
        # Recursively check the subtrees while narrowing the
        #   ↪ range for each node
        return (validate(node.left, low, node.val) and
                validate(node.right, node.val, high))

    return validate(root)

```

Explanation

This algorithm uses recursion to traverse the tree in a depth-first manner. The function ‘validate’ is a helper function that takes three parameters: the current ‘node’, a ‘low’ range, and a ‘high’ range. If the node is null, we return ‘True’ as an empty tree is a BST by definition. Then, we check if the current node’s value is between ‘low’ and ‘high’. If it is not, the tree cannot be a BST and we return ‘False’. If the current node’s value is within the range, we recursively call the validate function for the left and right subtrees, while updating the range to ‘low’ and ‘node.val’ for the left subtree, and ‘node.val’ and ‘high’ for the right subtree, respectively. Lastly, the ‘isValidBST’ function initiates the validation process from the root with the full possible range.

Why this approach

The approach of validating each node in the context of its allowable range ensures that we catch any violations of the BST properties as soon as they occur. This makes for a very efficient solution that doesn’t require any additional data structures, such as arrays.

Alternative approaches

An alternative approach would be to perform an in-order traversal and store the values in an array. The tree is a valid BST if the resulting array is sorted in ascending order. However, this requires $O(n)$ additional space and can be less efficient.

Similar problems to this one

Problems like *Find Mode in Binary Search Tree*, *Kth Smallest Element in a BST*, or *Binary Tree Inorder Traversal* all revolve around the traversal of a BST and can be solved using similar techniques.

Things to keep in mind and tricks

It is important to consider the upper and lower limits for the values in the nodes, especially with languages that have larger than 32-bit integer ranges. Avoid using hardcoded constants for comparison when possible.

Corner and special cases to test when writing the code

Include test cases with just one node, complete trees, and trees that are not balanced. Also, consider cases where the tree value might exceed a 32-bit signed integer range if relevant to the language of choice.

“`latex`

Alien Dictionary

The "Alien Dictionary" problem is an intriguing interview question that tests one's ability to infer the ordering of an unknown language's alphabet using its lexicon sorted according to alien linguistic rules. It is implied that the alien language utilizes the same alphabet as English but with a reorganized sequence.

Problem Statement

In the problem, you are given a list of sorted words from an alien language, with each word comprising a sequence of lowercase letters. Your task is to derive and return the ordering of the letters in the alien language as a string. If no valid order exists or if the input list is empty, the expected output is an empty string.

Example

Input: words = ["wrt", "wrf", "er", "ett", "rftt"]

Output: "wertf"

Explanation: Given the list of words from the alien language, we deduce the following order: - "t" precedes "r" (wrt -> wrf) - "w" precedes "e" (wrt -> er) - "r" precedes "t" (wrt -> ett) - "e" precedes "r" (er -> ett)

Thereby, the derived order is "wertf".

Algorithmic Approach

The solution to this problem is often tackled using graph theory concepts like directed acyclic graphs (DAGs). We can construct a graph where each vertex represents a character in the alien language, and directed edges define the lexicographic order between characters. Then we perform a topological sort on this graph to find the order of the characters. If we detect a cycle during this process, it would indicate that there is a conflict in the order, and no valid alphabet order can be determined, resulting in an empty string instead.

LeetCode link: [Alien Dictionary](#)

Python Implementation

Below is the complete Python code that implements a solution to the "Alien Dictionary" problem:

```
from collections import defaultdict, deque

def alienOrder(words):
    # Create a graph (adjacency list)
    adj_list = defaultdict(set)
    # Count of how many times a character appears as 'second' in
    # ↪ the alien dictionary order
    in_degree = {char: 0 for word in words for char in word}

    # Populate the graph and in_degree
    for first_word, second_word in zip(words, words[1:]):
        for char1, char2 in zip(first_word, second_word):
            if char1 != char2:
                if char2 not in adj_list[char1]:
                    adj_list[char1].add(char2)
                    in_degree[char2] += 1
                break
        else:
            # Check if second word is a prefix of the first word
            if len(second_word) < len(first_word): return ""

    # Initialize a queue with letters that have in_degree of 0
    queue = deque([char for char in in_degree if in_degree[char]
    ↪ == 0])
    alien_alphabet = []

    while queue:
        char = queue.popleft()
        alien_alphabet.append(char)
        # Reduce the in_degree for neighbors and add to queue if
        ↪ it becomes 0
        for neighbor in adj_list[char]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(alien_alphabet) == len(in_degree):
        return "".join(alien_alphabet)
    else:
        return "" # Cycle or not all letters are connected
        ↪ through edges

# Example usage:
words = ["wrt", "wrf", "er", "ett", "rftt"]
print(alienOrder(words)) # Output should be 'wertf'
```

The provided implementation begins by establishing a graph in the form of an adjacency list.

Why this approach

This approach is very fitting for the "Alien Dictionary" problem because it essentially translates to finding a possible topological ordering of characters given partial order constraints. Utilizing graph traversal and topological sort principles allows us to systematically derive the full order, if one exists, or recognize the impossibility of such an order when a cycle is present in the graph.

Alternative approaches

One could approach the problem with different variations of graph traversal strategies or by examining the pairwise comparisons of consecutive words in a different manner, but any valid solution will likely utilize some form of topological sorting given the nature of the problem.

Similar problems to this one

Problems that also involve deducing orders or hierarchies based on constraints, such as "Course Schedule" where prerequisites dictate a valid order for taking courses, or tasks scheduling problems where dependencies determine task execution order, share a resemblance to the "Alien Dictionary" problem.

Things to keep in mind and tricks

Remember that graph-based algorithms and data structures such as in-degree representation and adjacency lists play a crucial role in implementing the solution. Recognizing a topological sort as the primary technique helps guide the overall strategy.

Corner and special cases to test when writing the code

Edge cases may include scenarios where a word is a prefix of another, or the input list has only one word or characters that don't follow any others. It is essential to account for these to avoid incorrect inferences about the alien language's alphabet order. ““

Alien Dictionary

Problem Statement

Sorting a list of words in an alien language according to the given order requires understanding the lexicographical rules of that language. This task involves constructing a graph representation to determine the order of characters. The problem involves a topological sort on a directed graph where an edge from character 'u' to character 'v' means 'u' comes before 'v' in the alien language.

LeetCode link: [Alien Dictionary](#)

Algorithmic Approach

The following steps describe the algorithm to solve the problem:

1. Create a graph (adjacency list) representing the ordering of characters.
2. Populate the graph with the correct edges by comparing adjacent words.
3. Perform a topological sort to determine the order of characters.
4. Detect cycles during the topological sort (which would mean that the given dictionary order is invalid).
5. Return the lexicographically sorted order of the alphabet as per the alien language rules.

Complexities

- **Time Complexity:** The time complexity is $O(C)$, where C is the total length of all the words in the input list, because each adjacent pair of words is compared only once.
- **Space Complexity:** The space complexity is $O(1)$ or $O(U + \min(U^2, N))$, where U is the total number of unique characters and N is the total length of all the words. The space is bounded because the number of English characters is limited to 26.

Python Implementation

```

def alienOrder(words):
    # Create an adjacency list for the graph
    adj = {c: set() for word in words for c in word}

    # Populate the graph
    for i in range(len(words) - 1):
        w1, w2 = words[i], words[i + 1]
        min_len = min(len(w1), len(w2))
        if len(w1) > len(w2) and w1[:min_len] == w2[:min_len]:
            return "" # The first word is a prefix of the second
                       ↪ word, but longer
        for j in range(min_len):
            if w1[j] != w2[j]:
                adj[w1[j]].add(w2[j])
                break

    # Perform a topological sort
    visited = {} # False = visited, True = current path
    res = []

    def dfs(c):
        if c in visited:
            return visited[c]
        visited[c] = True
        for neigh in adj[c]:
            if dfs(neigh):
                return True
        visited[c] = False
        res.append(c)

    for c in adj:
        if dfs(c):
            return ""

    return "".join(res[::-1])

```

Explanation

The implementation uses a depth-first search (DFS) based topological sort. By iterating through each character of each word, we are able to build our graph with directed edges from earlier characters to later characters according to the alien language rules. If at any point during our DFS we encounter a node that is already in our current path, this indicates a cycle, and it is impossible to return a valid ordering.

Why this approach

The topological sorting approach is suitable because it naturally represents the relative ordering constraints between characters. Since the problem can be represented as a graph of dependencies, topological sort is the standard algorithm to extract linear ordering from such constraints.

Alternative approaches

An alternative approach could involve comparing all pairs of words using a modified comparison function based on the given alien alphabet order. While this approach would also give a valid sort, it has a higher time complexity compared to using a graph.

Similar problems to this one

Similar problems often involve determining an order from a set of constraints, such as "Course Schedule" which can also be solved using topological sorting.

Things to keep in mind and tricks

- Constructing the graph correctly is crucial, including handling the edge case where one word is a prefix of another.
- Detecting cycles is a key part of the topological sort process in this context.

Corner and special cases to test when writing the code

- Cases where one word is a prefix of another, which should return an empty string.
- Cases with cycles in the graph, which should also return an empty string.
- Inputs where every word is the same should return that word.
- Inputs with unrelated words, where the characters do not establish a particular order between them.

Trie (Prefix Tree)

The problem deals with implementing a Trie (also known as a prefix tree), which is a specialized tree used to handle a common set of strings. A trie is a variant of an n-ary tree in which characters are stored at each node and each path down the tree may represent a word. It has several methods, such as insert, search, and startsWith, which allow for efficient information retrieval and manipulation based on prefixes.

Problem Statement

LeetCode link: [Implement Trie \(Prefix Tree\)](#)

Algorithmic Approach

The trie structure is typically implemented using a class (*TrieNode*) representing each node in the trie. Each *TrieNode* contains a map or an array of child nodes, and a boolean flag indicating whether the node represents the end of a word.

The insert method traverses the trie based on the characters of the word being inserted, creating new nodes as necessary, and marks the final character's node as the end of a word.

The search method traverses the trie based on the characters of the word being searched, returning false if a character does not have a corresponding child node in the trie. If the final character's node is reached, the method returns whether the node is marked as the end of a word.

The startsWith method is similar to the search method, but it only checks the existence of the prefix and does not require the final node to be marked as the end of a word.

Complexities

- Time complexity:

- * For insertion: $O(m)$, where m is the length of the word to be inserted.
- * For search and startsWith: $O(m)$, where m is the length of the word or prefix to be searched.
- Space complexity: $O(n \times l)$, where n is the number of nodes in the trie and l is the size of the alphabet (26 for lowercase English letters in this problem).

Python Implementation

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, word: str) -> bool:
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end_of_word

    def startsWith(self, prefix: str) -> bool:
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True

```

Explanation

The ‘TrieNode’ class serves as the blueprint for each node. It includes a dictionary to store children nodes (edges in the trie), and a boolean to mark the end of a word.

The ‘Trie’ class itself has methods for insertion, searching a whole word, and searching by prefix. When inserting, we iterate over the word. If the character is not already a child of the current node, we create a new ‘TrieNode’. After adding all characters, we mark the last node as the end of a word.

The search method checks for the presence of each character in the word. If any character is missing from the children of the current node, the search

returns false. If we reach the end of the word, we return whether the last node is marked as an end of a word.

The `startsWith` method checks if the prefix is present in the trie. It is similar to `search`, except it does not require the final node to be marked as an end.

Why this approach

The trie data structure is chosen for its efficiency in searching and storing strings, especially when dealing with prefixes. It allows for quick insertions and prefix-based lookups, scaling well when dealing with a large collection of strings.

Alternative approaches

An alternative is to use a hash set. While this can be effective for word lookups, it does not handle prefix searches efficiently, requiring $O(n*m)$ time where n is the number of words and m is the length of the longest word.

Similar problems to this one

- Autocomplete feature implementation - Dictionary word lookup - IP routing table with prefix lookups

Things to Keep in Mind and Tricks

- Understanding the difference between searching for a whole word and a prefix.
- Avoiding duplicate nodes by checking if a child node already exists before creating a new one.
- Efficiently finding the end of a word and checking the `is_end_of_word` flag to return accurate search results.

Corner and special cases to test when writing the code

- Inserting and then searching for the same word - Inserting a word and then searching for a longer word with the same prefix - Searching a prefix that matches multiple words in the trie - Handling case sensitivity if required

Course Schedule

The "Course Schedule" problem is a challenge that focuses on determining the possibility of completing a series of courses given their prerequisite requirements. This problem analogy can be directly applied to understanding dependencies within systems, and it is commonly found as an algorithmic challenge that reinforces the understanding of graph theory concepts such as cycle detection and topological sorting.

Problem Statement

Given `numCourses` courses labeled from 0 to `numCourses - 1`, and a list of prerequisite pairs (where each pair $[a, b]$ implies that you must take course b before course a), the goal is to ascertain whether it is achievable to complete all the courses.

The crux of the problem involves formulating an algorithm to determine if an order exists to take these courses such that for each course, all its prerequisites are met.

LeetCode link: [Course Schedule](#)

Algorithmic Approach

To resolve this challenge, we can model the courses and prerequisites as a directed graph, with each course as a node and edges representing the dependency direction (from prerequisite course to the dependent course). The solution entails checking for cycles within this directed graph since the presence of a cycle indicates that there is no feasible way to complete all courses. This is because a cycle would mean a course's prerequisites could never be fully satisfied. Detecting a cycle can be performed via Depth-First Search (DFS) or by attempting to perform a Topological Sort on the graph. If we successfully perform a topological sort with all nodes visited, it implies there is no cycle, and completing all courses is feasible.

Complexities

- **Time Complexity:** In the scenario where we use Depth-First Search (DFS), the time complexity is $O(V + E)$ where V is the number of courses and E is the number of dependencies (prerequisites). This stems from the fact that every node and edge is visited in the worst case.
- **Space Complexity:** The space complexity is also $O(V + E)$ to store the graph data structure, as well as the recursion stack for the DFS procedure.

Python Implementation

Here is the complete Python solution utilizing DFS to check for the presence of cycles in the graph built from the course prerequisites:

```
class Solution(object):
    def canFinish(self, numCourses, prerequisites):
        def dfs(course):
            if visited[course] == -1:
                return False
            if visited[course] == 1:
                return True
            visited[course] = -1
            for next_course in graph[course]:
                if not dfs(next_course):
                    return False
            visited[course] = 1
            return True

        graph = {i: [] for i in range(numCourses)}
        for course, prereq in prerequisites:
            graph[course].append(prereq)

        visited = [0] * numCourses
        for course in range(numCourses):
            if not dfs(course):
                return False
        return True
```

This implementation begins by constructing a graph that represents the courses and their dependencies. It then utilizes a DFS approach to traverse this graph. For each course, it delves deeper into its prerequisites, using a `visited` list to track the state of each node (unvisited, visiting, visited). If a cycle is detected, indicated by finding a course in the 'visiting' state during the DFS, it returns `False`. If all courses can be traversed without encountering a cycle, the function returns `True`, indicating it is possible to finish all courses.

Why this approach

This graph-based approach is selected as it offers an intuitive method for representing and processing course prerequisites. DFS is a classic algorithm for detecting cycles in directed graphs, making it an apt choice for this scenario. Topological sorting is a closely related alternative that could also be used, especially since it provides the actual order in which the courses should be taken.

Alternative Approaches

Aside from the previously mentioned topological sort, an alternative approach could involve using the Disjoint Set Union (DSU) data structure to manage and detect cycles.

Similar Problems

Similar problems that students might encounter include "Course Schedule II", which extends this problem by asking for the actual order of courses to be completed, and "Redundant Connection", which requires cycle detection in an undirected graph.

Things to Keep in Mind and Tricks

When working with graph problems, particularly those involving cycle detection, it is essential to manage the states of nodes carefully to avoid false positives. Understanding and visualizing the graph can be very helpful in debugging and verifying the solution. Testing with edge cases such as an empty set of courses or prerequisites, as well as cases with multiple independent sub-graphs, is crucial for ensuring the robustness of the solution.

Dynamic Programming

Climbing Stairs

The "Climbing Stairs" problem is a great example to introduce the concept of dynamic programming due to its optimal substructure property where the total number of ways to reach a particular step is dependent only on the number of ways to reach the previous two steps.

Problem Statement

Given a non-negative integer n , the task is to find the number of distinct ways you can climb to the top of a staircase with n steps, where you can take either one step or two steps at a time.

LeetCode link: [Climbing Stairs](#)

Algorithmic Approach

The problem can be approached with dynamic programming by breaking it down into smaller subproblems. The total number of ways to reach the i -th step is the sum of ways to reach the $(i - 1)$ -th step and the $(i - 2)$ -th step.

Complexities

- **Time Complexity:** The iterative dynamic programming solution has a time complexity of $O(n)$ as we calculate the number of ways for each step once, up to the n -th step.
- **Space Complexity:** The space complexity is $O(n)$ if we store the number of ways for all steps up to n . This can be optimized to $O(1)$ by only keeping track of the last two steps at any time.

Python Implementation

Below is the complete Python code for the ‘climbStairs’ function that calculates the number of distinct ways to climb to the top of a staircase with n steps:

```
class Solution:
    def climbStairs(self, n: int) -> int:
        if n <= 2:
            return n
        one_step_before, two_steps_before = 2, 1
        all_ways = 0
        for step in range(3, n + 1):
            all_ways = one_step_before + two_steps_before
            two_steps_before = one_step_before
            one_step_before = all_ways
        return all_ways

# Example Usage
# Create an instance of the Solution class
solution = Solution()

# Input: n = 2
print(solution.climbStairs(2)) # Output: 2

# Input: n = 3
print(solution.climbStairs(3)) # Output: 3
```

This implementation uses an iterative approach to compute the number of ways in a bottom-up fashion. The space optimization is achieved by not using an array to store intermediate results for each step.

Why this approach

The dynamic programming approach is chosen because it provides an efficient way to solve this problem, avoiding redundant calculations. It systematically builds up the solution by solving each subproblem only once.

Alternative approaches

An alternative approach could use recursion with memoization, but this might be less efficient due to a larger stack depth and potentially higher memory usage.

Similar problems to this one

Other dynamic programming problems with optimal substructure and overlapping subproblems, such as the Fibonacci sequence, 'House Robber', and 'Minimum Path Sum' problems, share similarities with the Climbing Stairs problem.

Things to keep in mind and tricks

One trick is to realize that this problem is essentially a Fibonacci sequence where the number of ways to climb n steps is the n -th Fibonacci number. The iterative approach with two variables is akin to maintaining the last two numbers of the Fibonacci sequence to generate the next number.

Corner and special cases to test when writing the code

Some corner cases to consider while writing the code could be when $n = 1$, where there's only one way to reach the top, and when n is large, nearing the upper bound of the constraints. These cases help ensure that the implementation handles small and large inputs efficiently.

Decode Ways

The "Decode Ways" problem is a classic dynamic programming challenge that requires determining the total number of ways a given digit string can be decoded into letters, where each letter from A to Z is represented by numbers from 1 to 26.

Problem Statement

The task is to find the number of ways to decode a string s that consists only of digits with a mapping from 'A' to 'Z' where 'A' maps to "1", 'B' maps to "2", ..., and 'Z' maps to "26". A valid decoding can be any combination of single-digit numbers or two-digit numbers that fall within the range from "1" to "26".

LeetCode link: [Decode Ways](#)

Algorithmic Approach

To solve this problem, we can use dynamic programming. We define a cache array where $cache[i]$ represents the number of ways to decode the substring $s[:i]$. We iterate through the given string and update the cache based on whether the current character or the combination of the current and previous characters can represent a valid decoding.

Complexities

- **Time Complexity:** The total time complexity is $O(n)$ where n is the length of the string, since we iterate through the string once and perform $O(1)$ operations for each character.
- **Space Complexity:** The space complexity is $O(n)$ for caching the number of ways to decode each substring. However, we can optimize this to $O(1)$ by only keeping track of the number of ways to decode the last two substrings.

Python Implementation

Below is the complete Python code to the ‘numDecodings’ function which returns the total number of ways to decode the input string *s*:

```
class Solution:
    def numDecodings(self, s: str) -> int:
        if not s or s[0] == '0':
            return 0

        # Initialization
        prev = 1 # Number of ways to decode an empty string
        curr = 1 # Number of ways to decode a string with one
                  ↪ character

        for i in range(1, len(s)):
            temp = curr
            if s[i] == '0':
                if s[i-1] == '1' or s[i-1] == '2': # Check for
                  ↪ valid "10" or "20"
                    curr = prev
                else:
                    return 0
            elif 10 < int(s[i-1:i+1]) <= 26: # Check if the two-
              ↪ digit number is valid
                curr += prev
            prev = temp

        return curr
```

This implementation first checks if the input string is empty or starts with ‘0’. If either is true, there are no ways to decode the message, and it returns ‘0’. It initializes two variables, ‘prev’ and ‘curr’, which keep track of the number of ways to decode substrings of length ‘i-1’ and ‘i’, respectively, then iterates through the string while updating these variables based on each character’s validity both individually and potentially as part of a two-digit number.

Why this approach

The dynamic programming approach is chosen for its efficiency in both time and space. It allows us to build up from smaller subproblems, reducing the overall number of redundant calculations as compared to a naive recursive solution.

Alternative approaches

An alternative approach is to use a recursive function with memoization to reduce the repetition of subproblem calculations. However, this may still

not be as efficient as the dynamic programming approach presented above, particularly in terms of space complexity.

Similar problems to this one

Similar problems include "Climbing Stairs," "House Robber," and other sequence-based dynamic programming problems where the current state depends on a combination of one or more previous states.

Things to keep in mind and tricks

When solving dynamic programming problems, especially those involving strings, it's important to consider edge cases such as empty strings and strings with leading zeros. For this specific problem, the decoding is invalid if there are any zeros not preceded by '1' or '2'.

Corner and special cases to test when writing the code

Some corner cases include: - A single '0' or strings with leading '0's. - Strings only composed of '1's and '2's, which may result in a larger number of valid decodings. - Strings with numbers greater than '26' which should not increase the count of valid decodings.

Unique Paths

The "Unique Paths" problem involves finding the total number of unique paths from the top-left corner to the bottom-right corner of a grid, with the restriction that the robot can only move either down or right.

Problem Statement

A robot is located at the top-left corner of a $m \times n$ grid (marked 'Start' in the diagram below). The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid (marked 'Finish' in the diagram below). How many possible unique paths are there?

The task is to calculate the total number of distinct paths from the start to the finish.

LeetCode link: [Unique Paths](#)

Algorithmic Approach

To solve this problem, we can use a dynamic programming approach where we build up the number of paths to each cell from the top-left to the bottom-right. For each cell, the number of paths to it is the sum of the number of paths to the cell directly above it and the number of paths to the cell directly to the left of it. The cells in the top row and the leftmost column have only one path leading to them, so they are initialized to '1'.

Complexities

- **Time Complexity:** The total time complexity is $O(mn)$, where m is the number of rows and n is the number of columns in the grid, because we need to compute the number of paths for each cell.
- **Space Complexity:** The space complexity is also $O(mn)$, as we need to store the number of paths for all cells in a 2D array. The space can be optimized to $O(n)$ if we only keep track of the current and previous rows or columns at a time.

Python Implementation

Below is the complete Python code for the function 'uniquePaths', which calculates the total number of unique paths from the top-left corner to the bottom-right corner of a $m \times n$ grid:

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        dp = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i-1][j] + dp[i][j-1]
        return dp[-1][-1]
```

This implementation initializes a 2D list 'dp' with '1's, then starting from the second row and second column, it accumulates the total paths by adding the number from the top and left cells. The result is given by the value in the last cell 'dp[-1][-1]'.

Why this approach

The dynamic programming approach is chosen for this problem because it capitalizes on the optimal substructure and overlapping subproblems inherent to the question. Instead of recomputing the number of paths to reach the same cell multiple times through different routes, we consolidate the result in a DP table which significantly cuts down on redundant computations.

Alternative approaches

An alternative to dynamic programming is using combinatorics. Since the robot can only move right or down, for some path from the start to the finish, there are exactly $m - 1$ right moves and $n - 1$ down moves that the robot will make. Therefore, we can frame the problem as selecting $m - 1$ moves out of $m + n - 2$ total moves, which becomes a combination calculation $C(m + n - 2, m - 1)$.

Similar problems to this one

Similar problems include "Unique Paths II" where obstacles are placed on the grid, and "Minimum Path Sum" where, instead of finding the number of unique paths, the goal is to find the path with the minimum sum of numbers along its length.

Things to keep in mind and tricks

One of the tricks to optimize space complexity is to realize that the value of a cell depends only on the cells directly above and to its left. Thus, we can reduce the 2D DP array to a 1D array that is updated on the fly.

Corner and special cases to test when writing the code

Corner cases include grids with just one row or one column, where the number of unique paths is evidently '1'. Special cases might also include large grid values for m and n, to ensure that the function runs efficiently even for maximum input sizes.

House Robber

The "House Robber" problem is a typical example of dynamic programming. This problem is designed to help you think about optimizing decisions based on the given constraints—specifically, maximizing the total value of non-adjacent elements in a sequence.

Problem Statement

Given an array of non-negative integers 'nums', where 'nums[i]' represents the amount of money stashed in the i -th house, your task is to determine the maximum amount of money you can rob tonight without alerting the police. You cannot rob two adjacent houses because of connected security systems.

LeetCode link: [House Robber](#)

Algorithmic Approach

The key is to realize that at each step, you choose between two options: 1. Rob the current house and add its value to the total from the house before the immediate neighbor (since directly adjacent houses cannot be robbed). 2. Skip the current house and take the total from the immediate neighbor, which represents the best total without including this house's value.

To implement this, a dynamic programming approach is adopted, typically using a DP array to store the cumulative maximum value that can be robbed up to each house.

Complexities

- **Time Complexity:** The time complexity is $O(n)$, where n is the number of houses, because we need to iterate through the entire array once to build up our DP array.
- **Space Complexity:** By using two variables to keep track of the totals, we can reduce the space complexity to $O(1)$. A traditional DP array implementation would take $O(n)$ space.

Python Implementation

Here is the complete Python code for solving the House Robber problem using a simplified dynamic programming approach with constant space complexity:

```
class Solution(object):
    def rob(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        prev_max = 0
        current_max = 0
        for money in nums:
            temp = current_max
            current_max = max(prev_max + money, current_max)
            prev_max = temp
        return current_max
```

This implementation uses two variables, '*prev_max*' and '*current_max*', to keep track of the maximal robbed amount up till the previous house and the current house, respectively. As we iterate over the houses, these variables are updated to reflect the optimal robbery strategy up to that point.

Why this approach

This approach is chosen because it optimally solves the problem by making use of the concept of dynamic programming without the need for a full DP table, thus saving on space. It allows us to compute the solution with constant space while still maintaining the idea of building up to the final solution step by step.

Alternative approaches

An alternative would be using a DP array where '*dp[i]*' represents the maximum money that can be robbed up to the '*i*-th' house. This approach is more straightforward to understand but comes at the cost of higher spatial complexity.

Similar problems to this one

Similar problems include variants like "House Robber II" (where the houses are arranged in a circle), "House Robber III" (trees instead of arrays), and other dynamic programming problems that deal with optimizing selections such as "Maximum Subarray" or "Best Time to Buy and Sell Stock".

Things to keep in mind and tricks

When solving dynamic programming problems, always think about the state that needs to be carried forward and if the problem has overlapping subproblems and optimal substructure, both of which are fulfilled in this problem. A common trick is to reduce space complexity by using a fixed number of variables instead of an array when the DP state only depends on a limited set of previous states.

Corner and special cases to test when writing the code

Test cases should include: - The minimal case, with only one house. - Cases where the optimal solution involves skipping more than one house in between robberies. - Cases with varying amounts of money to ensure that the function is selecting houses optimally and not based on the number of houses robbed. - Test with large input size to verify time and space efficiency.

House Robber II

The "House Robber II" problem is a classical example of a dynamic programming challenge that deals with optimizing decisions over a circular arrangement of houses. Specifically, it is an extension of the "House Robber" problem with an added twist: the houses are arranged in a circle, which brings an additional constraint— one cannot rob the first and last house simultaneously.

Problem Statement

In the "House Robber II" problem, you are presented with an array of non-negative integers representing the amount of money stashed in each house. Since the houses are arranged in a circle, the first and last houses are considered adjacent. Your task is to determine the maximum amount of money you can rob without alerting the police, which will happen if two adjacent houses are robbed on the same night.

Example:

Input: `nums = [2,3,2]`

Output: `3`

Explanation: Robbing house 1 (money = 2) and then robbing house 3 (money = 2) would alert the police.

LeetCode link: <https://leetcode.com/problems/house-robber-ii/>

Algorithmic Approach

To solve this problem, we adopt a dynamic programming approach that breaks down the problem into subproblems. Since the first and last houses are adjacent, we can consider two separate cases: one in which we include the first house and exclude the last, and the other in which we exclude the first house and include the last. We then solve each case using the same dynamic programming approach as in the "House Robber" problem and take the maximum of the two cases as our final answer.

Complexities

- **Time Complexity:** The time complexity of this approach is $O(n)$, where n is the number of houses. This is because we solve the dynamic programming subproblem for the houses twice—once excluding the first house and once excluding the last house.
- **Space Complexity:** The space complexity is $O(1)$, assuming we are using an iterative approach and only storing a constant number of variables to keep track of the running maximum amounts that can be robbed.

Python Implementation

Below is the complete Python code for solving the "House Robber II" problem. The `rob` function handles the circular arrangement by considering two separate cases and calling the `rob_linear` helper function on each.

```
def rob(nums):
    def rob_linear(nums):
        rob1, rob2 = 0, 0
        for n in nums:
            new_rob = max(rob1 + n, rob2)
            rob1 = rob2
            rob2 = new_rob
        return rob2

    if len(nums) == 1:
        return nums[0]

    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))

# Example usage:
nums = [2, 3, 2]
print(rob(nums)) # Output: 3

nums = [1, 2, 3, 1]
print(rob(nums)) # Output: 4
```

The code begins by handling the edge case where there is only one house. For the general case, it calculates the maximum possible loot by considering two scenarios: excluding the first house or excluding the last house, treating the houses as if they are in a straight line rather than a circle. The `rob_linear` function iterates through the `nums` array, using the variables `rob1` and `rob2` to keep track of the maximum loot from the previous two houses and updating the maximum loot at each step.

Why this approach

This dynamic programming approach was chosen for its efficiency in dealing with the overlapping subproblems characteristic of the House Robber problems. By reusing solutions to subproblems, we avoid repeated calculations and achieve optimal runtime complexity.

Alternative approaches

An alternative approach might use recursion with memoization to solve the subproblems, but this typically requires additional space for the memoization

table and might not be as efficient as the iterative approach in terms of space complexity.

Similar problems to this one

Other dynamic programming problems with a focus on optimizing choices given constraints, such as the "House Robber" problem (the original version for a linear set of houses) and "Coin Change 2", share similarities with the "House Robber II" problem.

Things to keep in mind and tricks

One thing to keep in mind when approaching this type of problem is the importance of identifying the substructure of the problem. Breaking down the circle into two separate linear problems is key. Also, beware of edge cases, such as when there is only one house.

Corner and special cases to test when writing the code

Always test your implementation with corner cases, such as arrays with only one or two houses, and with special cases where the maximum loot might be obtained by skipping one or two houses in between.

Coin Change

The "Coin Change" problem is a well-known dynamic programming challenge that requires determining the minimum number of coins needed to make a certain amount of money given coin denominations.

Problem Statement

Given an array 'coins' of different denominations and an integer 'amount' representing a total amount of money, the aim is to compute the fewest number of coins required to make up that amount. If it is not possible to make the amount with any combination of the coins, the function should return -1.

Example 1:

Input: coins = [1, 2, 5], amount = 11

Output: 3

Explanation: $11 = 5 + 5 + 1$

Example 2:

Input: coins = [2], amount = 3

Output: -1

Example 3:

Input: coins = [1], amount = 0

Output: 0

LeetCode link: [Coin Change](#)

Algorithmic Approach

The solution utilizes dynamic programming to build a table that records the minimal number of coins necessary for each sub-amount up to the target amount. We use an array 'dp' with 'amount + 1' entries, where 'dp[i]' holds the minimum number of coins required for the value 'i'. Initially, 'dp[0]' is set to 0 (since zero coins are needed for amount zero), and the other entries are set to some maximum value to symbolize an unreachable state.

Complexities

- **Time Complexity:** The overall time complexity is $O(\text{amount} \times \text{number of coins})$ because we traverse through all sub-amounts for each coin.
- **Space Complexity:** The space complexity is $O(\text{amount})$ since we maintain a 1-dimensional DP array to store minimum coins for each amount up to the target amount.

Python Implementation

Below is the complete Python code for the ‘coinChange’ function, which finds the least number of coins needed given denominations to make up a certain amount:

```
def coinChange(coins, amount):
    # Initialize DP array with amount+1, which is an invalid
    #   ↪ value as it's
    #   ↪ bigger than the amount itself (since the maximum number of
    #   ↪ coins to
    # make any amount is amount (using all 1s)).
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0 # Base case: no coins needed for amount 0

    # Solve for all amounts from 1 to amount
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:
                # If coin is not greater than the amount
                dp[a] = min(dp[a], 1 + dp[a - coin])

    # If dp[amount] is still amount+1, then it was not possible
    #   ↪ to make change
    return dp[amount] if dp[amount] != amount + 1 else -1
```

This implementation first initializes the ‘dp’ array with a default value that’s not viable, signifying that for that amount, the minimum number of coins is still unknown. For every sub-amount up to the total amount, it tries each coin and determines if including that coin would result in a smaller number of coins to make the exact change.

Why this approach

The dynamic programming approach was chosen for its ability to break down the problem into subproblems and build up the answer efficiently. This characteristic of DP ensures that we do not recompute the minimum coins for the amounts we have already solved.

Alternative approaches

There are alternative approaches, such as a recursive solution or using a BFS approach, but these can be less efficient in terms of time complexity unless memoization is used with recursion. The greedy approach doesn’t work for all sets of coin denominations.

Similar problems to this one

This problem is similar to other classic DP problems like "Minimum Path Sum," "0/1 Knapsack," and "Longest Increasing Subsequence," where subproblems are solved to contribute to the overall solution.

Things to keep in mind and tricks

One of the key things

Longest Increasing Subsequence

The Longest Increasing Subsequence (LIS) problem is a classic problem in the field of computer science that comes under the dynamic programming category.

Problem Statement

The task is to find the length of the longest strictly increasing subsequence in an unsorted array of integers. A subsequence is a sequence that can be derived from the array by deleting some or no elements without changing the order of the remaining elements. A strictly increasing subsequence is one where each element is greater than the one before it.

For example:

Input: `nums = [10, 9, 2, 5, 3, 7, 101, 18]`

Output: 4

Explanation: The longest increasing subsequence is [2, 3, 7, 101], therefore the length is 4.

LeetCode link: [Longest Increasing Subsequence](#)

Algorithmic Approach

The solution to this problem can be approached by using dynamic programming to construct a table where the i -th element stores the length of the longest increasing subsequence ending with the i -th element. Alternatively, one can use patience sorting technique to optimize the time complexity further.

Complexities

- **Time Complexity:** The total time complexity of the dynamic programming approach is $O(n^2)$, where n is the length of the input array. Using a

binary search with patience sorting, the time complexity can be improved to $O(n \log n)$.

- **Space Complexity:** The space complexity is $O(n)$ due to the use of an additional array for storing the lengths of the subsequences.

Python Implementation

Below is the complete Python code for the 'lengthOfLIS' function to determine the length of the longest increasing subsequence:

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        if not nums:
            return 0

        LIS = [1] * len(nums)
        for i in range(1, len(nums)):
            for j in range(i):
                if nums[i] > nums[j]:
                    LIS[i] = max(LIS[i], LIS[j] + 1)
        return max(LIS)
```

This implementation constructs an array 'LIS', initialized with ones. It then iteratively updates the 'LIS' array by comparing each element with all the previous elements. If a valid increasing subsequence is found, the 'LIS' value is updated. Finally, the maximum value in the 'LIS' array is returned, which represents the length of the longest increasing subsequence.

Why this approach

The dynamic programming approach was chosen for its relatively straightforward implementation and reduced time complexity compared to a brute-force solution. It efficiently builds upon solutions to smaller problems, storing intermediate results to avoid redundant calculations.

Alternative approaches

An alternative approach involves the patience sorting technique coupled with binary search, reducing the overall time complexity. This approach takes advantage of the properties of the problem and demonstrates a more complex understanding of the underlying structures.

Similar problems to this one

Variants and extensions of the LIS problem include finding the actual longest increasing subsequence, determining the number of such subsequences, and solving the problem with modifications in the subsequence conditions (e.g., non-decreasing or with a fixed step size).

Things to keep in mind and tricks

- Always consider edge cases such as empty arrays or arrays with all identical elements. - Binary search is a significant optimization when the problem constraints allow. - It is crucial to thoroughly understand the subproblem overlap in dynamic programming to devise an optimal solution.

Corner and special cases to test when writing the code

- Arrays with only one element (should return 1). - Arrays with elements in strictly decreasing order (should return 1). - Arrays with all elements identical (should return 1). - Long arrays to evaluate performance and ensure that the implementation does not exceed time limits.

Longest Common Subsequence

The "Longest Common Subsequence" (LCS) problem is a classic algorithmic challenge that falls under the category of dynamic programming. The problem entails finding the length of the longest subsequence present in both given sequences, where a subsequence is defined as a sequence that maintains the original order but not necessarily contiguously present in both strings.

Problem Statement

Given two strings, 'text1' and 'text2', the task is to return the length of their longest common subsequence. A subsequence is a new string generated from the original string by deleting some characters (possibly none) without altering the remaining characters' relative order. A common subsequence of two strings is a subsequence that is common to both strings. If there is no common subsequence, the function should return 0.

Example:

Input:

text1 = "abcde"

text2 = "ace"

Output:

3

Explanation:

The longest common subsequence is "ace" and its length is 3.

LeetCode link: [Longest Common Subsequence](#)

Algorithmic Approach

The dynamic programming approach involves creating a 2D array 'dp' where 'dp[i][j]' represents the length of the longest common subsequence of sub-strings 'text1[0...i-1]' and 'text2[0...j-1]'. This table is filled in a bottom-up manner based on whether characters at current indices in both strings match or not, as defined by the recurrence relations:

- If $\text{text1}[i - 1]$ equals $\text{text2}[j - 1]$, include this character in LCS: $\text{dp}[i][j] = \text{dp}[i - 1][j - 1] + 1$.
- If $\text{text1}[i - 1]$ is not equal to $\text{text2}[j - 1]$, find the longer LCS by not including either the i -th or j -th character: $\text{dp}[i][j] = \max(\text{dp}[i - 1][j], \text{dp}[i][j - 1])$.

The initial condition is when at least one of the strings is empty, leading to a '0' longest common subsequence. The final answer resides in $\text{dp}[\text{length}(\text{text1})][\text{length}(\text{text2})]$, determining the length of the LCS of text1 and text2 .

Complexities

- **Time Complexity:** The time complexity is $O(m \times n)$, where m and n are the lengths of text1 and text2 respectively, owing to the nested loop to fill the dp table.
- **Space Complexity:** The space complexity is $O(m \times n)$ to store the dp table. It is possible to reduce the space complexity by using only two rows at a time since we only reference the row above the current one.

Python Implementation

Below is the complete Python code for computing the length of the longest common subsequence:

```
class Solution:
    def longestCommonSubsequence(self, text1, text2):
        m, n = len(text1), len(text2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m+1):
            for j in range(1, n+1):
                if text1[i-1] == text2[j-1]:
                    dp[i][j] = dp[i-1][j-1] + 1
                else:
                    dp[i][j] = max(dp[i-1][j], dp[i][j-1])

        return dp[m][n]
```

This code initializes a ‘dp’ table and then iteratively builds up the solution to larger subproblems using the previously mentioned recurrence relations. By the end, the ‘dp’ table’s last entry reflects the answer to the problem.

Why this approach

The dynamic programming approach is leveraged due to the LCS problem’s overlapping subproblems and optimal substructure properties. Specifically, solving larger problems efficiently requires solving various smaller, similar problems, which naturally lends itself to a dynamic programming solution. This approach minimizes redundant calculations compared to a naive recursive solution and is therefore chosen for its time efficiency.

Alternative approaches

An alternative approach includes the recursive

Bit Manipulation

Number of 1 Bits

The "Number of 1 Bits" problem, typically known as the "Hamming Weight" problem, is a classic bit manipulation challenge. This problem exercises one's understanding of how to work with individual bits and perform binary operations in programming.

Problem Statement

The task is to write a function that takes an unsigned integer as input and returns the number of '1' bits it has, which is also known as the function's Hamming weight.

For instance, given the 32-bit unsigned integer 11, its binary representation is 00000000000000000000000000001011, and the function should return '3', as there are three bits set to '1'.

Function signature for the `hammingWeight` function may look like this in C++:

```
int hammingWeight(uint32_t n);
```

The function should accept a 32-bit unsigned integer and return the number of 'Set bits' or '1' bits in its binary representation. LeetCode link: [Number of 1 Bits](#)

Python Implementation

Below is the complete Python code implementing the `hammingWeight` function:

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        count = 0
        while n:
            n &= n - 1 # Drops the lowest set bit of 'n'
            count += 1
        return count
```

This implementation takes advantage of the Brian Kernighan algorithm, which efficiently counts the number of '1' bits by repeatedly flipping the least significant '1' bit of the number to '0' until the number becomes zero.

Why this approach

The chosen approach uses Brian Kernighan's algorithm, which is highly efficient for counting set bits. The algorithm works in $O(k)$ time complexity, where k is the number of set bits, because it performs the operation `n &= n - 1` exactly k times before n becomes zero. This is typically faster than checking each of the 32 bits individually, especially for numbers with a smaller number of set bits relative to their possible range.

Alternative Approaches

There are alternative ways to count the number of '1' bits in a binary number. A straightforward approach is to iterate over each bit and increment a count if the bit is set. Another method is using lookup tables, which can be faster for higher ranges of numbers, at the cost of extra space.

Similar Problems

Problems that involve bit manipulation are quite common in competitive programming and technical interviews. Some similar problems include "Reverse Bits", "Single Number", and "Bitwise AND of Numbers Range".

Things to Keep in Mind and Tricks

When dealing with bit manipulation, it's crucial to remember how different bitwise operations work and be familiar with binary arithmetic. A trick often used is `x & (x - 1)`, which clears the lowest set bit; this is a key part of the Brian Kernighan's algorithm used in this solution.

Corner and Special Cases to Test

It's important to test cases where the input is 0, as it should return 0 since there are no set bits. Additionally, testing with the maximum unsigned 32-bit integer value, which is $2^{32} - 1$, ensures that the function correctly counts all 32 set bits. Cases with a single '1' bit at different positions should also be tested to ensure the bit counting is accurate regardless of the bit's position.

Reverse Bits

The "Reverse Bits" problem is a classic exercise in bit manipulation that asks for the reversal of bits in a given 32 bits unsigned integer.

Problem Statement

The task is to reverse the bits of a given 32 bits unsigned integer. The input is provided as an integer, and the output should also be an integer, representing the decimal value of the binary bits reversed.

LeetCode link: [Reverse Bits](#)

Algorithmic Approach

To reverse the bits in an integer, a bitwise approach is taken, shifting through each bit, and accumulating the result. The key operations involve bitwise shifts and bitwise OR.

Complexities

- **Time Complexity:** The algorithm runs in $O(1)$ time, as the number of bits to be processed is constant (32 bits).
- **Space Complexity:** The space complexity is $O(1)$, since no extra space is required other than the variables used for the bit manipulations.

Python Implementation

Below is the complete Python code to reverse the bits of a given 32 bits unsigned integer:

```
class Solution:
    def reverseBits(self, n: int) -> int:
        rev = 0
        for i in range(32):
            rev = (rev << 1) | (n & 1)
            n >>= 1
        return rev
```

This implementation is straightforward, using a loop to iterate through each of the 32 bits. It initially sets 'rev' to 0 and then, for each bit in the input 'n', shifts 'rev' one bit to the left, reads the least significant bit of 'n', and adds it to 'rev' using a bitwise OR. The input 'n' is then shifted one bit to the right to continue the process with the next bit until all bits have been reversed.

Why this approach

Bitwise manipulation is chosen for this problem due to its efficiency in handling binary operations at a low level. Since the problem relates to the individual bits of an integer, using bitwise operators is the most direct and fastest approach.

Alternative approaches

Though the problem could theoretically be solved by converting the integer to a binary string, reversing the string, and then converting back to an integer, this approach would not fulfil the constraints laid out in the problem statement where string manipulation is not allowed.

Similar problems to this one

Variations of bit manipulation problems could include counting set bits, finding single numbers in a duplicate array using XOR, and converting a number to hexadecimal.

Things to keep in mind and tricks

It is essential when performing bitwise operations to consider the size of the integers you are working with, especially when dealing with language-specific peculiarities related to signed and unsigned numbers.

Corner and special cases to test when writing the code

When implementing bitwise operations, testing edge cases including the maximum and minimum values the integer can have, such as 0, 1, $2^{31} - 1$, and 2^{31} can help ensure that the code correctly handles all possible bit configurations.

231. Power of Two

Determining if a given integer is a power of two is a common problem in computer science. This problem entails checking whether a number n can be expressed as 2^k where k is an integer.

Problem Statement

Given an integer n , the task is to write a function to determine if it is a power of two.

LeetCode link: [231. Power of Two](#)

Algorithmic Approach

To solve the problem, we will capitalize on the properties of binary representations of powers of two. An integer is a power of two if and only if it has exactly one '1' bit in its binary representation and it is not the sign bit. Therefore, the problem can be approached by checking the following conditions:

- **Positivity:** n must be positive, as negative numbers and zero cannot be powers of two.
- **Single '1' bit:** n must have exactly one '1' bit in its binary representation.

To check for a single '1' bit, we can use the following tricks and properties:

- **Bitwise AND:** When n is a power of two, $n \& (n - 1)$ will be zero since there is no overlap of '1' bits between n and $n - 1$.
- **Special Cases:** Since the problem's range is between -2^{31} to $2^{31} - 1$, we have to be careful when n is 2^{31} due to integer overflow issues.

Complexities

- **Time Complexity:** $O(1)$ - The time complexity is constant as the operation to check if the number is a power of two requires a fixed number of steps.

- **Space Complexity:** $O(1)$ - No extra space is required aside from the few variables used, so the space complexity is also constant.

Python Implementation

```
def isPowerOfTwo(n):
    return n > 0 and (n & (n - 1)) == 0
```

Explanation

The provided Python function checks two conditions: first, that the input integer n is greater than zero, and second, that n has exactly one '1' bit in its binary representation. The bitwise AND operation $n \& (n - 1)$ zeroes out the least significant '1' bit of n , and if after this operation the result is zero, then n was a power of two.

Why this approach

The bitwise approach is chosen for its efficiency. It avoids the need for looping, which would make the solution less efficient, especially for large values of n .

Alternative Approaches

An alternative approach would be to continually divide n by two until n equals one. If n becomes non-integer at any point in this process, it is not a power of two.

Similar Problems

A related problem is determining if a number is a power of any given number x , not just two. This would involve a similar approach of dividing by x until the result is one or a non-integer value.

Things to Keep in Mind and Tricks

- The special case of the maximum 32-bit integer (2^{31}) should be kept in mind.
- Remember to first check that n is positive before performing the bitwise check.
- The bitwise trick used here is that a power of two has a single '1' bit and all other bits are '0'.

Corner and Special Cases to Test

- Test the function with the input integers 0 and -1 to ensure that the condition of n being greater than zero is correctly handled.
- Also test the function with 1 and 2^{30} which are on the edge of being within the problem constraints.

Counting Bits

The "Counting Bits" problem ...

Problem Statement

Given an integer 'n', return an array 'ans' that represents the number of '1's in the binary representation of each number 'i' for all '0 <= i <= n'.

LeetCode link: [here](#)

Algorithmic Approach

The solution for counting the number of '1' bits in the binary representation of each number up to 'n' utilizes dynamic programming and bit manipulation. We can relate the number of '1' bits in a number to its half; if 'i' is even, 'ans[i]' is the same as 'ans[i/2]'. If 'i' is odd, then 'ans[i] = ans[i - 1] + 1'.

Complexities

- **Time Complexity:** The time complexity of this approach is $O(n)$, as we compute the number of '1' bits for each number up to 'n' once, using previously computed values.
- **Space Complexity:** The space complexity is $O(n)$, which is required for storing the output array.

Python Implementation

Below is the complete Python code that counts the number of '1' bits for all numbers up to 'n':

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i >> 1] + (i & 1)
        return ans
```

This implementation initializes an array `ans` of size $n + 1$ to store the number of 1 bits for each value from 0 to n . It then iterates from 1 to n , calculating each `ans[i]` based on the values already computed. The expression $i \gg 1$ corresponds to $i / 2$ and $i \& 1$ is 1 if i is odd and 0 if even.

Why this approach

This dynamic programming approach was chosen due to its efficiency over the brute force method. It effectively breaks down the problem by relating the number of bits in a number to the number of bits in a smaller number (specifically, its half), thereby reusing computations to build up the final result.

Alternative approaches

An alternative method could be the brute force strategy of iterating over each bit of every number using bit masking, but it would be less efficient, having a time complexity of $O(n \log n)$.

Similar problems to this one

Similar bit manipulation problems include "Number of 1 Bits", "Reverse Bits", and "Bitwise AND of Numbers Range". These problems also involve understanding the binary representation and manipulating bits.

Things to keep in mind and tricks

Key tricks include recognizing that for a given even number 'i', the number of '1' bits is the same as in $i / 2$, and that for an odd number 'i', it is one more than in the number $i - 1$. This understanding is crucial in formulating the dynamic programming relation.

Corner and special cases to test when writing the code

Special cases that should be tested include: - The lower bound where $n = 0$. - An odd number n , to ensure that the addition of the last bit is handled properly. - A large even number n , to confirm that the computation is efficiently scaled.

Counting Bits

The problem is to find the number of 1's in the binary representation of every number within a given range. The result should be an array containing the counts for all numbers from 0 to the input number.

Problem Statement

Leet code link: [Counting Bits Problem](#)

Algorithmic Approach

We will leverage the patterns of the binary representation to find the number of 1 bits for each number. This property can be described as follows: For any number x , the number of 1 bits in x is one more than the number of 1 bits in $x \& (x - 1)$, which removes the lowest-set bit from x .

An efficient approach is to use the previously calculated results to calculate the new result in a dynamic programming fashion. Given a number, if we right shift it by one bit and then add the last bit, it will give us the count of 1's in the number.

Complexities

- Time Complexity: $O(n)$ - where n is the input number. We traverse once from 0 to n and use previously computed values.
- Space Complexity: $O(n)$ - to store the result for each number from 0 to n .

Python Implementation

```
def countBits(num):  
    count_bits = [0] * (num + 1)  
    for i in range(1, num + 1):  
        count_bits[i] = count_bits[i >> 1] + (i & 1)  
    return count_bits
```

Explanation

In the code, we initialize an array called `count_bits` to store the counts of 1's for each number. We start a loop from 1 to `num` (since the count for 0 is always 0). Inside the loop, we right shift the current number by 1 to get the count of the previous number and add the last bit of the current number.

Why this approach

This approach makes use of dynamic programming principles to build up the solution as it progresses, avoiding redundant work and reaching an optimal time complexity. It utilizes the relation between bit patterns of consecutive numbers and previously calculated results.

Alternative approaches

A brute-force approach might be to check each bit of every number in the range `[0, num]`, but this would result in a worse time complexity and wouldn't fulfill the $O(n)$ requirement.

Similar problems to this one

Problems related to bit manipulation, such as finding a single number in a list where every other number appears twice, can be seen as similar since both involve understanding and manipulating binary representations.

Things to keep in mind and tricks

Remember that right shifting a number by one bit is equivalent to dividing it by two. The least significant bit can be obtained by bitwise AND operation with 1.

Corner and special cases to test when writing the code

Ensure to test cases with 'num' as 0 and also large values close to the upper constraint. It's also useful to check if each number's count of 1's correctly accumulates from the previous number's count.

Search and Sort

Valid Anagram

Problem Statement

Given two strings 's' and 't', write a function to determine if 't' is an anagram of 's'. An Anagram is a word or phrase formed by rearranging the letters of another word or phrase, typically using all the original letters exactly once.

For example, the word "anagram" can be rearranged into "nag a ram".

Note that two empty strings are considered to be anagrams of each other.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

Example 2:

Input: s = "rat", t = "car"

Output: false

LeetCode link: [Valid Anagram](#)

Algorithmic Approach

To determine if one string is an anagram of another, we can count the occurrences of each letter in both strings and then compare these counts. This can be accomplished with a hash table or an array of fixed size if the strings only consist of lowercase alphabetic characters.

Complexities

- **Time Complexity:** The time complexity of the algorithm is $O(n)$, where n is the length of the input strings, because we have to count each letter in both strings.
- **Space Complexity:** The space complexity is $O(1)$ because the additional space used by the algorithm depends only on the size of the alphabet used in the strings and not on the length of the strings themselves.

Python Implementation

Below is the complete Python code for the 'isAnagram' function to check if one string is an anagram of another:

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False

        count = [0] * 26 # Since the problem statement mentions
                        ↪ only lowercase alphabets

        for char in s:
            count[ord(char) - ord('a')] += 1
        for char in t:
            count[ord(char) - ord('a')] -= 1

        for c in count:
            if c != 0:
                return False

        return True
```

This implementation counts the frequency of each character in the string 's' and 't' by incrementing and decrementing the value at the respective indices in the 'count' array. After processing both strings, if all elements in the 'count' array are zeros, the strings are anagrams; otherwise, they aren't.

Why this approach

This approach was chosen due to its efficiency in both time and space complexity. By utilizing an array of fixed size to count character occurrences instead of a hash table, we ensure constant time operations for updating counts, which significantly reduces the overhead and maintains linear time complexity with respect to the size of the input.

Alternative approaches

An alternative approach involves sorting both strings and comparing them to check for equality. Although intuitive, this method has a higher time complexity of $O(n \log n)$, where n is the length of the strings due to the sorting operation.

Similar problems to this one

Similar problems may ask for permuting strings to match patterns, checking if a string can break another, or finding the minimum number of steps to make two strings anagrams. These variations usually apply a similar concept of character frequency counting but tweak the problem constraints or goals.

Things to keep in mind and tricks

Remember that direct array access is faster than using a hash table for counting when the set of possible characters is small and fixed, as is the case with lowercase English letters.

Corner and special cases to test when writing the code

Ensure that you consider and test for edge cases such as one or both strings being empty, strings of differing lengths (which should immediately return 'false'), and strings with the same characters but different frequencies.

Contains Duplicate

The "Contains Duplicate" problem is a common coding challenge that requires a straightforward solution to determine if an array contains any repeated elements.

Problem Statement

Given an integer array *nums*, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Example 1:

Input: *nums* = [1,2,3,1]

Output: `true`

Example 2:

Input: *nums* = [1,2,3,4]

Output: `false`

Example 3:

Input: *nums* = [1,1,1,3,3,4,3,2,4,2]

Output: `true`

Algorithmic Approach

There are several approaches to solve this problem:

1. Sorting: Sort the array and check for consecutive elements being equal.
2. Hashing: Use a hash set to keep track of visited elements.

The solution can be implemented using either approach, but using a hash set typically results in a time-efficient and straightforward solution. The follow-up question challenges the possibility of not using any extra space, which would then likely lead us to modify the array in-place, such as performing a sort operation.

Complexities

- **Time Complexity:** The time complexity for the sorting approach is $O(n \log n)$ due to the sort operation. For the hashing approach, it is $O(n)$ in the average case, as set operations have a constant time complexity on average.
- **Space Complexity:** The space complexity is $O(n)$ for the hashing approach as it requires storing up to n elements in the set. For sorting, if the in-place sort is used, then the space complexity can be $O(1)$.

Python Implementation

Below is the complete Python code implementing the hashing approach to determining if a list contains any duplicates:

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        seen = set()
        for number in nums:
            if number in seen:
                return True
            seen.add(number)
        return False
```

This implementation uses a set to store elements that have already been processed. As we iterate over the list, we check if each element is already present in the set, indicating a duplicate. If so, we return `true`; otherwise, we add the element to the set. If we finish iterating without finding duplicates, we return `false`.

Why this approach

The hashing approach was chosen because it provides an average-time complexity that is linear with respect to the number of elements in the array. It's also a simple and elegant solution that is easy to implement.

Alternative approaches

An alternative approach is to sort the array first and then look for consecutive equal elements. This removes the need for extra space (apart from the space needed to sort the array), complying with the follow-up requirement. However, sorting the array will increase the time complexity to $O(n \log n)$, making it less efficient than the hashing approach in terms of time.

Similar problems to this one

Similar problems often involve finding duplicates or checking for the existence of certain elements within a dataset. Problems such as "Find the Duplicate Number," "Find All Duplicates in an Array," and "Contains Duplicate II" are variations that pose additional constraints or slight modifications to the underlying problem of detecting duplicates.

Things to keep in mind and tricks

In problems like these, hashing is a common trick that leverages the high-speed access time of a hash table to quickly check for the existence of an element. It's important to consider the trade-off between time and space complexity when choosing the approach to implement.

Corner and special cases to test when writing the code

When implementing solutions, it is crucial to handle corner cases, such as empty arrays or arrays with a single element where no duplicates can exist. Additionally, for the hashing approach, one must consider the hash set's potential impact on the runtime if the dataset contains many elements, causing hash collisions and degrading performance to $O(n^2)$ in the worst case.

Group Anagrams

The "Group Anagrams" problem is an interesting question that requires understanding of hash tables and string manipulation. It is a typical interview question that checks the candidate's ability to handle and classify data based on specific rules, in this case, anagrams.

Problem Statement

Given an array of strings `strs`, the task is to group the anagrams together.

An **Anagram** is a word or phrase that is formed by rearranging the letters of a different word or phrase, using all the original letters exactly once.

LeetCode link: [Group Anagrams](#)

Algorithmic Approach

To solve this problem, we can categorize each string by its character count or sorted version. We will use a hash table to maintain a list of strings that, when sorted, yield the same string. Each key in the hash table will represent a sorted string, and its value will be a list of strings that are anagrams of each other.

Complexities

- **Time Complexity:** The time complexity is $O(NK \log K)$, where N is the length of `strs`, and K is the maximum length of a string in `strs`. This accounts for the sorting of each string.
- **Space Complexity:** The space complexity is $O(NK)$, to store the result in the hash table.

Python Implementation

Below is the complete Python code that implements the grouping of anagrams by using a hash table:

```
def groupAnagrams(strs):
    anagrams = {}
    for s in strs:
        sorted_s = ''.join(sorted(s))
        if sorted_s not in anagrams:
            anagrams[sorted_s] = [s]
        else:
            anagrams[sorted_s].append(s)
    return list(anagrams.values())
```

The function 'groupAnagrams' takes a list of strings and classifies them based on their sorted order, effectively grouping the anagrams together.

Why This Approach

This approach is chosen because it effectively utilizes the property that anagrams share the same sorted sequence of characters. Sorting the strings provides a consistent key to group the anagrams. Sorting each string takes $O(K \log K)$, and doing this for N strings results in the time complexity of $O(NK \log K)$.

Alternative Approaches

An alternative to sorting might be to use a character count as the key in the hash table, which could potentially reduce the time complexity to $O(NK)$ since calculating the character count of a string takes linear time with respect to the string length.

Similar Problems to This One

There are various problems that require categorization or grouping based on certain characteristics, such as "Valid Sudoku", which asks to categorize numbers based on their positions in a grid, or "Find Duplicate File in System", where files need to be grouped by their content.

Things to Keep in Mind and Tricks

- Utilizing sorting or counting as the way to identify anagrams.
- Being careful with how the hash table handles collisions and understanding how the lookups work.

Corner and Special Cases to Test When Writing the Code

- Input list containing empty strings. - Strings with varying lengths. - Ensuring that strings with the same characters but in different orders are grouped correctly.

Merge Intervals

The "Merge Intervals" problem is a common algorithmic challenge involving array manipulation and sorting. The task is to merge all overlapping intervals in a given array to produce a list of non-overlapping intervals that cover all the intervals in the input.

Problem Statement

Given an array of intervals where intervals

i

=

$start_i, end_i$

, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example:

Input: intervals = [[1,3],[2,6],[8,10],[15,18]]

Output: [[1,6],[8,10],[15,18]]

Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].

LeetCode link: [Merge Intervals](#)

Algorithmic Approach

The solution to this problem typically involves sorting the intervals by their start times, iterating through the sorted intervals, merging overlapping intervals, and adding non-overlapping intervals directly to the result list.

Python Implementation

Below is the complete Python code for merging overlapping intervals:

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        ↪ ]]:
        if not intervals:
            return []

        # Sort the intervals by the start of each interval
        intervals.sort(key=lambda x: x[0])

        merged = []
        for interval in intervals:
            # if the list of merged intervals is empty or if the
            ↪ current
            # interval does not overlap with the previous, simply
            ↪ append it.
            if not merged or merged[-1][1] < interval[0]:
                merged.append(interval)
            else:
                # otherwise, there is overlap, so we merge the
                ↪ current and previous
                # intervals by updating the end of the previous
                ↪ interval
                merged[-1][1] = max(merged[-1][1], interval[1])

        return merged
```

This implementation begins by checking if the intervals list is empty. It then sorts the intervals based on the start time to ensure that overlapping intervals are adjacent. As it iterates through the sorted list, it either merges overlapping intervals or appends non-overlapping intervals to the ‘merged’ list.

Why this approach

Sorting the intervals helps in identifying overlaps easily, which is vital because merging is only necessary when intervals overlap. Comparing the end of the last interval in the merged list with the start of the current interval allows us to determine whether an overlap exists.

Alternative approaches

An alternative approach could be using a tree data structure, like an interval tree, to manage overlapping intervals. However, this comes with more complex implementation and does not improve the worst-case time complexity.

Similar problems to this one

Other interval-related problems like "Insert Interval" or "Non-overlapping Intervals" use similar techniques for managing and comparing intervals, often also requiring the intervals to be sorted as a first step.

Things to keep in mind and tricks

Always consider edge cases where intervals might just touch each other, e.g.,

1, 2

and

2, 3

. The definition of overlap in the problem description determines how to handle these cases.

Corner and special cases to test when writing the code

It's important to test cases with a single interval, intervals that do not overlap at all, and intervals that overlap completely. Additionally, consider testing intervals with identical start and end times, which should be properly merged into a single interval.

Find Minimum in Rotated Sorted Array

The "Find Minimum in Rotated Sorted Array" problem entails locating the smallest element within a sorted array that has been rotated between 1 and n times. Given an increasingly sorted array that has undergone this transformation, the objective is to efficiently determine the minimal value considering that the array's sorted nature provides a strategic advantage for a binary search algorithm, as opposed to a linear search which would exceed the desired $O(\log n)$ time complexity.

Problem Statement

Given a rotated sorted array, the task is to identify the minimum element. A sorted array is considered rotated if it is shifted in such a way that some of the elements that were initially at the beginning of the array are moved to the end while preserving the overall order. It is important to note that the algorithm should run in $O(\log n)$ time complexity.

LeetCode link: [Find Minimum in Rotated Sorted Array](#)

Algorithmic Approach

To solve this problem, we will employ a binary search technique. Binary search enables us to divide the search space by half at each step. By comparing the middle element to the end element of the search space, we can determine whether the minimum element lies on the left or right side and continue searching in the correct half.

Complexities

- **Time Complexity:** Our binary search approach has a time complexity of $O(\log n)$ since at each step, we are halving the search space.
- **Space Complexity:** The space complexity is $O(1)$, as no additional space proportional to the input size is used.

Python Implementation

The Python code below illustrates the implementation of the binary search algorithm to find the minimum element in the rotated sorted array:

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > nums[right]:
                left = mid + 1
            else:
                right = mid
        return nums[left]
```

In this code, we maintain two pointers, 'left' and 'right', which indicate the current search space. We calculate the 'mid' point and use it to decide which half contains the minimum element. The loop continues until 'left' is the same as 'right', indicating that we have narrowed down to the minimum element.

Why this approach

The decision to apply binary search in this case is based on the fact that despite the array's rotation, it is still partially sorted. During each iteration, the binary search is able to establish which segment of the array (divided at the midpoint) is sorted and thus eliminate the opposite half from consideration, effectively narrowing the potential locations of the minimum element.

Alternative approaches

An alternate, less efficient method would be a linear search through the entire array to find the minimum element. However, since this would have a time complexity of $O(n)$, it does not meet the problem's requirement for $O(\log n)$ time complexity.

Similar problems to this one

This problem shares similarities with other problems that involve searching in a rotated sorted array, such as "Search in Rotated Sorted Array" where you are tasked with finding the position of a specific target value.

Things to keep in mind and tricks

When implementing binary search algorithms, it's important to be careful with the calculation of the middle index and the update steps for the search boundaries to avoid infinite loops or off-by-one errors. It's also necessary to ensure that the comparison logic is well thought out to determine the correct half of the search space.

Corner and special cases to test when writing the code

Make sure to test cases where the minimum element is at the beginning or the end of the array, as well as when the array is not rotated at all. These edge cases can help ensure that the binary search conditions are correctly handling all possible scenarios.

Search in Rotated Sorted Array

The "Search in Rotated Sorted Array" problem involves finding the index of a target value within an array that has been both sorted and rotated. A challenge here is to maintain a logarithmic $O(\log n)$ search time, indicating the need for a modified binary search approach, despite the array's rotation.

Problem Statement

Given an array *nums* that is originally sorted in ascending order and then possibly rotated, we need to find and return the index of a target value *target* in *nums*. If the target value is not found within the array, we should return -1.

The problem exemplifies a scenario where a sorted array like $[0, 1, 2, 4, 5, 6, 7]$ is rotated at a pivot unknown beforehand, resulting in an array like $[4, 5, 6, 7, 0, 1, 2]$. Despite the rotation, we need to ensure the search algorithm is efficient with a runtime complexity of $O(\log n)$.

LeetCode Link: [Search in Rotated Sorted Array](#)

Algorithmic Approach

To solve the problem, we can start by examining the array and determining how it might have been rotated. By comparing the middle element to the end of the array, we can decide which half is in proper ascending order and whether the target lies in that half. If not, we then search the other half accordingly. This approach is a subtle variation of the traditional binary search method, which is only performant on sorted arrays.

Complexities

- **Time Complexity:** Since the algorithm still fundamentally follows a binary search pattern, the time complexity is $O(\log n)$, where n is the number of elements in the input array.
- **Space Complexity:** The algorithm operates in constant space, hence space complexity is $O(1)$.

Python Implementation

The following Python code implements the search function for finding the target in a rotated sorted array:

```
class Solution(object):
    def search(self, nums, target):
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid

            # Left sorted portion
            if nums[left] <= nums[mid]:
                if nums[left] <= target <= nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            # Right sorted portion
            else:
                if nums[mid] <= target <= nums[right]:
                    left = mid + 1
                else:
                    right = mid - 1

        return -1
```

By maintaining two pointers, 'left' and 'right', that define the search space, the algorithm effectively halves this space at every step, discarding the half that can be confidently asserted to not contain the target.

Why this approach

This binary search approach was chosen due to its logarithmic time complexity, which aligns with the requirements of achieving $O(\log n)$ runtime. The problem's constraint that the array is originally sorted, even though it might be rotated, allows us to perform a modified binary search and makes this approach both elegant and efficient.

Alternative approaches

An alternative brute force approach would involve a linear search through the array to find the target, but this would not meet the logarithmic time complexity requirement. Another alternative could involve first finding the pivot point where the array is rotated and then performing a regular binary search in the appropriate subarray, but this is more complex and unnecessary.

Similar problems to this one

Similar problems that use binary search with a twist include "Find Minimum in Rotated Sorted Array", "Search in Rotated Sorted Array II" (where elements may be duplicated), and "Find Peak Element".

Things to keep in mind and tricks

When implementing the binary search for this problem, it's crucial to carefully handle the conditionals within the while loop to avoid off-by-one errors. Deciding when to move the 'left' or 'right' pointer requires a clear understanding of the sorted portions of the array after its rotation.

Corner and special cases to test when writing the code

Special cases that are essential to test include arrays with just one element, arrays where the target is the pivot point, and arrays where the target does not exist. Additionally, one should test cases where the target is at the beginning or end of the array.

“latex

Merge k Sorted Lists

The "Merge k Sorted Lists" problem is a classic algorithm problem that is often asked in coding interviews. The problem challenges the solver to efficiently combine multiple sorted data streams into a single sorted data stream, which has practical applications in various domains such as merging time-series data from multiple sources or combining sorted log files.

Problem Statement

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the k sorted linked-lists into one sorted linked-list and return it.

Example:

Consider the following k sorted linked lists:

List 1: $1 \rightarrow 4 \rightarrow 5$

List 2: $1 \rightarrow 3 \rightarrow 4$

List 3: $2 \rightarrow 6$

The merged list should be:

$1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4 \rightarrow 5 \rightarrow 6$

Input: The input consists of an array of k pointers to the head nodes of each of the k sorted linked lists.

Output: The output should be the head of the single sorted linked list that is the result of merging the k sorted lists.

LeetCode link: <https://leetcode.com/problems/merge-k-sorted-lists/>

Algorithmic Approach

The solution to this problem can be approached by using a min-heap or a priority queue to efficiently manage the current smallest nodes from each linked list. This method takes advantage of the fact that the heads of each linked list are the smallest elements remaining in each list, so we can perform a

similar operation to a merge in merge sort by always selecting the smallest head node.

Complexities

- **Time Complexity:** The total time complexity is $O(N \log k)$, where N is the total number of nodes and k is the number of linked lists.
- **Space Complexity:** The space complexity is $O(k)$ for storing the pointers in the heap at any given time.

Python Implementation

Below is the complete Python code for the ‘Solution’ class, which implements the ‘mergeKLists’ method to merge k sorted linked lists using a min-heap for efficient retrieval of the smallest node at any step:

```
from Queue import PriorityQueue

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def mergeKLists(self, lists):
        head = point = ListNode(0)
        pq = PriorityQueue()
        for l in lists:
            if l:
                pq.put((l.val, l))
        while not pq.empty():
            val, node = pq.get()
            point.next = ListNode(val)
            point = point.next
            node = node.next
            if node:
                pq.put((node.val, node))
        return head.next
```

This implementation initially sets up a dummy head and a point reference to track the merged list. The ‘PriorityQueue’ is used to store the tuple ‘(val, node)’ for each head of the linked lists. It ensures the heap property such that the smallest value is always at the top. By continuously extracting the smallest and inserting the next element from the same list, the algorithm merges all lists into one sorted linked list.

Why this approach

The min-heap approach was chosen for its optimal time complexity considering the need to frequently find and remove the smallest element from a collection of sorted arrays. This method is efficient because it maintains a heap of only k elements representing the heads of each list, and thus the extract-min and insert operations are $O(\log k)$.

Alternative approaches

Alternative approaches include the brute force method, which involves collecting all nodes into an array, sorting it, and then creating a new sorted list.

Another approach is to compare nodes one by one or use divide and conquer to merge lists in pairs.

Similar problems to this one

Similar problems include "Merge Two Sorted Lists" and "Sort List," where the principles of merging or handling sorted data structures are central to the solution.

Things to keep in mind and tricks

Keep

Miscellaneous

Happy Number

The problem is to determine whether a number is "happy". A happy number is one that follows a specific iterative process: starting with any positive integer, the number is replaced by the sum of the squares of its digits. This process is repeated until the number becomes 1, where it will remain as a happy number, or it enters an infinite loop which does not include 1.

Problem Statement

Leet code link: [Happy Number](#)

Algorithmic Approach

To solve this problem, we use two pointers approach commonly known as Floyd's Cycle detection algorithm. We move one pointer (the *slow* pointer) one step at a time, and the other (the *fast* pointer) two steps at a time. If at any point, the fast pointer reaches 1, we determine that the number is happy. However, if a cycle is detected signifying that the slow and fast pointers meet and the fast pointer is not 1, we conclude that the number is not happy.

Complexities

- **Time Complexity:** The time complexity is difficult to determine due to the nature of the problem. However, it can be said that once a cycle is detected, the function terminates, which prevents it from running indefinitely.
- **Space Complexity:** The space complexity is $O(1)$, as we are only using a constant amount of additional space beyond the input size.

Python Implementation

```

class Solution:
    def isHappy(self, n: int) -> bool:
        def get_next(number):
            total_sum = 0
            while number > 0:
                number, digit = divmod(number, 10)
                total_sum += digit ** 2
            return total_sum

        slow_runner = n
        fast_runner = get_next(n)

        while fast_runner != 1 and slow_runner != fast_runner:
            slow_runner = get_next(slow_runner)
            fast_runner = get_next(get_next(fast_runner))

        return fast_runner == 1

```

Explanation

The Python implementation defines a function `isHappy`, which accepts an integer `n` as a parameter. A helper function, `get_next`, is also defined to calculate the sum of squares of the digits for the given number.

Two pointers, `slow_runner` and `fast_runner`, are used to follow the sequence of number transformations. The slow runner moves to the next number in the sequence once, while the fast runner moves twice. If the fast runner reaches 1, the number is declared happy.

The loop continues until the fast runner equals 1, or the slow and fast runners meet, indicating a cycle. The function returns `True` if the fast runner is 1, and otherwise `False`, signaling the number is not happy.

Why this approach

Floyd's Cycle detection algorithm is chosen for its efficiency and simplicity. It avoids the use of extra space and also avoids the need to keep track of all the numbers that have already been processed.

Alternative approaches

An alternative approach could be using a hash set to keep track of all the numbers that have occurred to identify cycles. This approach has a higher space complexity but may be more straightforward to understand.

Similar problems to this one

Similar problems include detecting cycles in linked lists or any problem that requires the detection of repetition or cycles within a sequence of data.

Things to keep in mind and tricks

This problem serves as a great example of how the two pointers technique can solve problems that involve sequences and cycles without requiring additional memory for storage.

Corner and special cases to test when writing the code

When implementing the solution, ensure to test against cases where the starting number is very large or when the sequence converges to 1 after many iterations. Also consider edge cases like 0 or negative inputs, even if not explicitly allowed by the problem description.

933. Number of Recent Calls

The problem requires designing a class named `RecentCounter` to track the number of recent requests within a time frame of 3000 milliseconds. A request is represented by a single integer which indicates the time of the request in milliseconds. The class should support the following operations:

- `RecentCounter()`: Initializes the `RecentCounter` object.
- `ping(int t)`: Records a new request at time `t` and returns the number of requests that occurred in the past 3000 milliseconds, including the request just made at time `t`.

Problem Statement

LeetCode link: [933. Number of Recent Calls](#)

Algorithmic Approach

To keep track of the requests, we can use a queue to store the timestamps of all requests. When a new request comes in at time `t`, we add it to the queue. Then, we remove all requests from the front of the queue that are older than `t - 3000` milliseconds, because we only need to consider requests within the last 3000 milliseconds. Finally, we return the size of the queue, which represents the number of recent requests within the required time frame.

Complexities

The time complexity of the `ping` operation is $O(n)$, where `n` is the number of requests that are older than `t - 3000`. These need to be dequeued. However, since we are guaranteed that each `ping` call is made with a strictly larger value of `t`, each request will be added and removed from the queue at most once, leading to an amortized time complexity of $O(1)$ per operation.

The space complexity is $O(n)$, where `n` is the maximum number of requests that can appear within the time frame of 3000 milliseconds at any given time.

Python Implementation

```

from collections import deque

class RecentCounter:

    def __init__(self):
        self.requests = deque()

    def ping(self, t: int) -> int:
        self.requests.append(t)
        while self.requests[0] < t - 3000:
            self.requests.popleft()
        return len(self.requests)

# Example:
# obj = RecentCounter()
# param_1 = obj.ping(t)

```

Explanation

The `RecentCounter` class maintains a queue of timestamps, each representing a request. The `ping` method appends the current request time `t` to the queue and then continues to remove any requests from the front of the queue that are not within the 3000 milliseconds time frame. Finally, it returns the count of requests left in the queue.

Why this approach

This approach was chosen due to its simplicity and efficiency. Queues inherently follow a FIFO (First-In-First-Out) order, which is exactly what we need for maintaining the order of requests. Since we are only interested in a sliding window of 3000 milliseconds, older requests that fall out of this window can be discarded, and a queue efficiently allows the removal of these elements.

Alternative approaches

An alternative approach could involve keeping an array or list of all requests and iterating over it every time to count how many fall within the last 3000 milliseconds. However, this would result in a higher time complexity for the `ping` method, as it would require iterating over potentially many more elements than necessary.

Similar problems to this one

There are several other problems that deal with processing requests over a sliding time window or maintaining counts within a particular constraint, such as "362. Design Hit Counter" or "239. Sliding Window Maximum".

Things to keep in mind and tricks

One common trick in these types of problems is understanding that you do not need to keep track of all the historical information – only the information that is relevant for your current computation. Also, when processing elements in a sliding window, using a queue can be very helpful.

Corner and special cases to test when writing the code

Since the problem guarantees that each ping call has a strictly larger t than the previous one, we do not need to account for out-of-order or simultaneous requests. However, it is still important to test edge cases such as the first ping (to ensure the queue is handled correctly) and a ping exactly at $t - 3000$ (to check inclusivity of the time frame).

362. Design Hit Counter

The problem is to create a Hit Counter that can record hits and return the number of hits in the past five minutes. It operates on a system with two main functions: `hit(timestamp)` to record a hit at the specified timestamp in seconds and `getHits(timestamp)` to retrieve the count of hits in the last five minutes including the current timestamp. The design should ensure that timestamps are always increasing sequentially.

Problem Statement

LeetCode link: [Design Hit Counter](#)

Algorithmic Approach

The hit counter can be implemented using a queue or a circular buffer due to the nature of the problem that requires tracking of hits within a rolling time window of the last five minutes.

Complexities

- The time complexity of the `hit()` function is $O(1)$, because recording a new hit involves only an addition of the timestamp to the queue or updating the count at a specific index in the array.
- The time complexity of the `getHits()` function is $O(n)$ with a queue where n is the number of timestamps that fall outside the five-minute window, because we may need to remove these old timestamps from the queue. With a circular buffer, the complexity can be reduced to $O(1)$ since the `getHits()` function would only need to sum up the counts of valid time slots within the five-minute window, and the number of slots is constant.
- The space complexity is $O(n)$, where n is the number of potential hits that can occur within a five-minute window. With a queue, the space required grows with the number of hits recorded, while for an array, it is fixed to the size of the five-minute window in seconds (300 seconds).

Python Implementation

```

class HitCounter:
    def __init__(self):
        self.hits = []
        self.window_size = 300 # 5 minutes in seconds

    def hit(self, timestamp: int) -> None:
        self.hits.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        while self.hits and self.hits[0] <= timestamp - self.
            ↪ window_size:
            self.hits.pop(0)
        return len(self.hits)

```

Explanation

A simple queue is used to keep track of hit timestamps. On each hit, the timestamp is appended to the queue, and when the number of hits is queried, we remove timestamps from the beginning of the queue that are outside the five-minute window.

Why this approach

This approach was chosen for its simplicity and ease of understanding. The queue naturally handles the rolling window of hits by adding new hits to the end and removing old hits from the front.

Alternative approaches

An alternative approach could be to use a fixed-size array of 300 elements (one for each second in the five-minute window) to record hit counts. This would be particularly effective if the number of hits per second is very large as it avoids the overhead of queue operations.

Similar problems to this one

Similar problems involve tracking requests or events over a rolling time period, like counting unique visitors in the last 24 hours, or rate limiting API calls.

Things to keep in mind and tricks

When working with time series data, it's important to consider time complexity, especially when data might arrive at a high frequency. Using data structures that are efficient for adding/removing elements is key.

Corner and special cases to test when writing the code

Special cases to consider include handling very large numbers of hits at the same timestamp, ensuring that the system correctly handles the rolling window edge cases when timestamps are near the beginning or end of the window.

994. Rotting Oranges

Problem Statement

Given a grid where each cell can have values 0, 1, or 2, which represent an empty cell, a fresh orange, or a rotten orange, respectively, we are required to find the minimum number of minutes that must elapse until no cell has a fresh orange. An orange becomes rotten if it is adjacent (either up, down, left, or right) to a rotten orange. If it is impossible to rot all oranges, we should return -1.

LeetCode link: [994. Rotting Oranges](#)

Algorithmic Approach

The problem can be addressed using a Breadth-First Search (BFS) algorithm. The key idea is to start from all the rotten oranges at the beginning (time 0) and spread the rot to adjacent fresh oranges in each time step. Keep track of the time it takes for the last fresh orange to become rotten to determine the minimum number of minutes. It is crucial to check if there are any fresh oranges that are not adjacent to rotten oranges; if yes, then mark it as impossible by returning -1.

Complexities

- Time Complexity: $\mathcal{O}(N \times M)$, where N is the number of rows and M is the number of columns in the grid. This is because in the worst case, we have to process each cell in the grid.
- Space Complexity: $\mathcal{O}(N \times M)$ for the queue that is used for BFS if all oranges are rotten initially.

Python Implementation

```

from collections import deque

def orangesRotting(grid):
    rows, cols = len(grid), len(grid[0])
    rotten = deque()
    fresh_oranges = 0
    minutes = 0

    # Find all rotten oranges and count fresh oranges
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                rotten.append((r, c))
            elif grid[r][c] == 1:
                fresh_oranges += 1

    # Directions up, down, left, right
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    # Check if all oranges are rotten initially
    if fresh_oranges == 0:
        return max(0, minutes)

    # Process BFS
    while rotten and fresh_oranges > 0:
        minutes += 1
        for _ in range(len(rotten)):
            x, y = rotten.popleft()

            for dx, dy in directions:
                nx, ny = x + dx, y + dy

                if nx < 0 or ny < 0 or nx >= rows or ny >= cols:
                    continue

                if grid[nx][ny] == 1:
                    grid[nx][ny] = 2
                    fresh_oranges -= 1
                    rotten.append((nx, ny))

    return minutes if fresh_oranges == 0 else -1

```

Explanation

The solution uses a BFS approach to simulate the rot spread from rotten oranges to adjacent fresh ones. We start by identifying all the initial rotten oranges and counting the total number of fresh oranges. Then, for each minute,

we process all the current rotten oranges and update their adjacent cells. If all fresh oranges are infected by the end of the process, we return the time taken; otherwise, we return -1.

Why this approach

BFS is chosen for this problem because it allows for a uniformly spreading infection from every rotten orange at each time step. It also helps us keep track of the time needed for all oranges to rot, which is required by the problem statement.

Alternative approaches

An alternative could be using Depth-First Search (DFS) with backtracking to try all possible ways the rot can spread, but this would result in a less efficient solution than BFS for this problem.

Similar problems to this one

Similar problems include ones that deal with spreading or updating states in a grid, such as "Flood Fill", "Number of Islands", "Walls and Gates", and others involving BFS or grid-based state changes.

Things to keep in mind and tricks

Remember to check for cases where there are no fresh oranges at all or where all oranges are already rotten. Also, you should consider when oranges are isolated and cannot be rotted, which would require returning -1.

Corner and special cases to test when writing the code

- A grid with no oranges at all (all zeros). - A grid with only fresh oranges (should return -1 because they cannot rot on their own). - A grid with only rotten oranges. - Cases where some fresh oranges are isolated and cannot be reached by rotten oranges.

295. Find Median from Data Stream

The problem is to design a data structure that efficiently finds the median of the elements in a data stream. As data streams in, we want to be able to add a number and find the current median quickly.

Problem Statement

Leetcode Link: [295. Find Median from Data Stream](#)

Algorithmic Approach

To solve this problem, I will use two priority queues (also known as heaps in Python): one max-heap to store the smaller half of the numbers and one min-heap to store the larger half of the numbers. The max-heap allows us to efficiently get the largest of the small numbers, whereas the min-heap gives us quick access to the smallest of the large numbers. By keeping both heaps balanced (having the same number of elements or the max-heap having one more element), we can ensure that the median is either the top of the max-heap or the average of the tops of both heaps.

Complexities

- Time Complexity:
 - * `addNum()` - $O(\log n)$ because adding an element to a heap is $O(\log n)$ due to the need to maintain the heap invariant.
 - * `findMedian()` - $O(1)$ because it only requires accessing the top elements of the heaps.
- Space Complexity: $O(n)$ as we store all elements inserted into the data structure.

Python Implementation

```

import heapq

class MedianFinder:

    def __init__(self):
        # The large heap stores the larger half, the small heap
        #   ↳ stores the smaller half
        self.small = [] # Use max-heap (invert the values to use
        #   ↳ min-heap in Python)
        self.large = [] # Use min-heap

    def addNum(self, num):
        # Add to max heap
        heapq.heappush(self.small, -num)

        # Balance heaps - ensure each heappop operation is valid
        if self.small and self.large and (-self.small[0] > self.
        #   ↳ large[0]):
            val = -heapq.heappop(self.small)
            heapq.heappush(self.large, val)

        # Maintain size property - max-heap can have one more
        #   ↳ element than min-heap
        if len(self.small) > len(self.large) + 1:
            val = -heapq.heappop(self.small)
            heapq.heappush(self.large, val)
        elif len(self.large) > len(self.small):
            val = heapq.heappop(self.large)
            heapq.heappush(self.small, -val)

    def findMedian(self):
        if len(self.small) == len(self.large):
            return (-self.small[0] + self.large[0]) / 2.0
        return float(-self.small[0])

# Your MedianFinder object will be instantiated and called as
#   ↳ such:
# obj = MedianFinder()
# obj.addNum(num)
# param_2 = obj.findMedian()

```

Explanation

Initially, both heaps are empty. When a new number is added to the stream, it is first inserted into the max-heap. If after insertion, the max element of the max-heap is greater than the min element of the min-heap, we move this element from the max-heap to the min-heap to maintain the order property.

We also balance the heaps such that either both heaps have the same number of elements or the max-heap has one more element.

Why this approach

This approach ensures that finding the median operation can be done in constant time while adding a number operation takes logarithmic time. This is efficient considering the median needs to be recalculated each time a new number is introduced.

Alternative approaches

An alternative would be to maintain a sorted list and insert each number in the correct position, which would take $O(n)$ time for adding a number and $O(1)$ time for finding the median. This approach is not efficient for large lists.

Similar problems to this one

Similar problems deal with dynamic order statistics, such as finding the k -th largest element in a stream, which can also be solved using a dual heap approach.

Things to keep in mind and tricks

Keep both heaps balanced in size so the median can be found quickly. When using a min-heap in Python, to simulate a max-heap you can insert the negation of the values.

Corner and special cases to test when writing the code

- The stream has no elements or only one element.
- The number of elements is odd or even.
- Continuous numbers that would lead to imbalance heaps if not correctly managed.

Design Add and Search Words Data Structure

The problem described requires the creation of a data structure that supports both the addition of words and searching for words with the possibility of including wildcards represented by '.' characters, where each '.' can match any single letter.

LeetCode link: [Design Add and Search Words Data Structure](#)

Algorithmic Approach

To efficiently support the operations required by the problem, a trie (prefix tree) data structure is a suitable choice. A trie is a tree-like data structure used to store a dynamic set of strings, where keys are usually strings. Each node in the trie corresponds to a character of a string, making it a good fit for our word handling functionality.

For the `addWord(word)` operation, we insert a string into the trie by creating nodes for each character if they do not already exist.

For the `search(word)` operation with support for wildcards, we use recursion to represent different possibilities when a wildcard character is encountered.

Complexities

- For the `addWord(word)` operation, the time complexity is $O(m)$, where m is the length of the word to be inserted. This is because we process each character of the word once.
- The `search(word)` operation has a worst-case time complexity of $O(n \cdot 26^m)$, where n is the number of words added, and m is the length of the search word. This is because each wildcard character can be matched with any of the 26 possible characters, leading to a combinatorial explosion of possibilities. However, the average case is often much better, especially if there are a limited number of wildcards.

- The space complexity is $O(t)$, where t is the total number of nodes in the trie. In the worst case, when all words are of length m and have no common prefix, this complexity is $O(nm)$, where n is the number of inserted words.

Python Implementation

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class WordDictionary:

    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, word):
        def search_in_node(word, node):
            for i, char in enumerate(word):
                if char != '.':
                    if char not in node.children:
                        return False
                    node = node.children[char]
                else:
                    for child in node.children.values():
                        if search_in_node(word[i + 1:], child):
                            return True
                    return False
            return node.is_end_of_word

        return search_in_node(word, self.root)

```

Explanation

The core of the implemented solution consists of two parts: - The `TrieNode` class, which models the nodes of the trie, includes a dictionary child named `children` that maps characters to their child nodes, as well as a boolean flag `is_end_of_word` that indicates if a node is the end of a word. - The `WordDictionary` class, which implements the `addWord` and `search` methods.

The `addWord` method iterates through each character of the word, creating new nodes as required, and flags the end of the inserted word.

The `search` method is where the wildcard handling occurs. It uses a helper

function `search_in_node` that searches recursively. If it encounters a regular character, it simply traverses deeper into the trie. However, if a '.' character is encountered, the function must try all possible subpaths that could lead to a word match.

Why this approach

The trie was chosen for its efficiency and suitability in handling prefix-based search queries. Its structured approach to storing words allows for a quick and flexible search operation, including the ability to handle wildcards with a backtracking search technique.

Alternative Approaches

An alternative approach could have been to use a hash table storing all possible words and then checking for matches with wildcards using regular expressions. However, this would have been less space-efficient and potentially slower for wildcard searches due to the lack of structure in comparison to a trie.

Similar Problems

- Implementing a trie is a common problem itself.
- "Implement Magic Dictionary" is similar, as it involves adding a "search" functionality that can match words with one character difference.
- "Replace Words" uses a similar concept where shortest equivalents from a dictionary are used to replace words in a sentence.
- "Word Search II" involves finding words in a grid and can also be implemented with trie for more efficiency.

Things to Keep in Mind and Tricks

When implementing the trie:

- Recursion is often used in search functions, particularly when dealing with wildcards.
- It's important to carefully manage the `is_end_of_word` flag to correctly indicate when a word terminates.

Corner and Special Cases to Test

- The empty string as input.

- Cases with multiple wildcards in different positions (e.g. "..a", "a..", ".a.", "...").
- Repeated calls to `addWord` and `search` with the same word to check if previous insertions or searches interfere with later ones.

Kth Smallest Element in a BST

The "Kth Smallest Element in a BST" problem is a classic example of a tree traversal question. Given the root of a binary search tree, and an integer k , we need to find the k -th smallest value of all the values of the nodes in the tree.

Problem Statement

Given a binary search tree, write a function `kthSmallest` to find the k -th smallest element in it.

Example 1:

Input: `root = [3,1,4,null,2]`, `k = 1`

```
  3
 / \
1   4
 \
  2
```

Output: 1

Example 2:

Input: `root = [5,3,6,2,4,null,null,1]`, `k = 3`

```
  5
 / \
3   6
 / \
2   4
 /
1
```

Output: 3

LeetCode link: [Kth Smallest Element in a BST](#)

Algorithmic Approach

The solution to this problem hinges on the properties of a binary search tree (BST). A BST is a tree where each node has at most two children. It is structured in a way that the left child of a node contains a value less than the node itself, and the right child contains a value greater. Therefore, if one performs an in-order traversal on a BST, the nodes visited will be in ascending order. The algorithm will utilize this property to find the k -th smallest element.

Complexities

- **Time Complexity:** The total time complexity for this algorithm is $O(n)$, where n is the number of nodes in the tree. In the worst-case scenario, we visit each node once during the in-order traversal.
- **Space Complexity:** The space complexity is $O(h)$, where h is the height of the tree. This space is used on the recursive call stack for the in-order traversal. In the case of a balanced tree, the height is $O(\log n)$, but in the worst case of a skewed tree, the height can be $O(n)$.

Python Implementation

Below is the complete Python code for the `Solution` class, which implements the `kthSmallest` method to find the k -th smallest element in a BST:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def kthSmallest(self, root, k):
        # Helper function to perform in-order traversal.
        def inorder(r):
            return inorder(r.left) + [r.val] + inorder(r.right)
            ↪ if r else []

        # Perform in-order traversal and get the k-th smallest
        ↪ item.
        return inorder(root)[k - 1]

# The main function to execute the algorithm.
if __name__ == '__main__':
    # Example usage
    root = TreeNode(3)
    root.left = TreeNode(1)
    root.right = TreeNode(4)
    root.left.right = TreeNode(2)
    solution = Solution()
    k = 1
    print(f"{k}-th smallest element is: {solution.kthSmallest(
        ↪ root, k)}")
```

This implementation utilizes a helper function `inorder` that performs an in-order traversal of the BST. The `inorder` function leverages recursion to visit all nodes in ascending order and collect their values into a list. The `kthSmallest` method then simply returns the $k-1$ index from the list that `inorder` returns, since Python lists are zero-indexed.

Why this approach

The reason for choosing this approach is the intrinsic order provided by the in-order traversal on a BST. It is a direct and optimal method as it's using the BST's properties to our advantage.

Alternative Approaches

An alternative approach would be an iterative in-order traversal that stops once we've found the k -th smallest element, rather than traversing the entire tree.

This would still have a time complexity of $O(n)$ but can potentially reduce the number of visited nodes.

Similar Problems

Similar problems include finding the k -th largest element in a BST, validating if a binary tree is a BST, and constructing a BST from preorder or postorder traversal arrays.

Things to Keep in Mind and Tricks

- Understanding in-order, pre-order, and post-order traversals in a BST is essential for many tree-based interview problems.
- Always consider the properties of BSTs when designing your approach.
- Remember that the elements visited in an in-order traversal of a BST will be in sorted order.

Corner and Special Cases to Test

- The tree with a single node should return the value of that node for $k = 1$.
- A skewed tree (like a linked list) where all nodes are left or right children.
- Consider edge cases where k is outside the range of the number of nodes in the tree.